Algorithm Problems

Algorithm Time & Space Complexity Analysis

Algorithm complexity measures how the **resource requirements** (time and space) grow as the input size increases.

Why It Matters

- Predict algorithm performance
- Compare algorithms
- Scale applications
- Optimize code
- Pass coding interviews

Big O Notation (Asymptotic Analysis)

Measures worst-case growth rate as $n \to \infty$

Notation	Name	Description
O(1)	Constant	Time/space doesn't change with input
O(log n)	Logarithmic	Halves problem each step
O(n)	Linear	Grows proportionally with input
O(n log n)	Linearithmic	Common in efficient sorts
$O(n^2)$	Quadratic	Nested loops
$O(2^n)$	Exponential	Doubles with each addition
O(n!)	Factorial	All permutations

Basic Rules for Analysis

1. Drop Constants

```
// O(2n) → O(n)
for (int i = 0; i < n; i++) { }      // O(n)
for (int j = 0; j < n; j++) { }      // O(n)
// Total: O(2n) = O(n)
```

2. Drop Non-Dominant Terms

```
// O(n² + n) → O(n²)
for (int i = 0; i < n; i++) {        // O(n)
    for (int j = 0; j < n; j++) { }  // O(n²) - dominant
}
```

3. Different Inputs → Different Variables

```
// O(n × m) not O(n²)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) { }
}
```

Step-by-Step Analysis Process

1. **Identify input size `n`**
2. **Count primitive operations**
3. **Express as function f(n)**
4. **Find dominant term**
5. **Remove constants**
6. **Use Big O notation**

What Counts Towards Space?

- Variables
- Data structures (arrays, lists, trees)
- Function call stack (recursion)
- Auxiliary space (temporary variables)

Space Complexity Examples

```
// O(1) Space
public int Sum(int[] arr) {
    int total = 0;           // O(1)
    for (int i = 0; i < arr.Length; i++) {
        total += arr[i];     // O(1)
    }
    return total;
}

// O(n) Space
public int[] DoubleArray(int[] arr) {
    int[] result = new int[arr.Length];  // O(n)
    for (int i = 0; i < arr.Length; i++) {
        result[i] = arr[i] * 2;
    }
    return result;
}

// O(n) Space (Recursion)
public int Factorial(int n) {
    if (n <= 1) return 1;
    return n * Factorial(n - 1);  // O(n) call stack
}
```

Common Time Complexities

Complexity	Example Algorithms	Growth Rate
O(1)	Array access, Hash table lookup	Constant
O(log n)	Binary search, Balanced BST operations	Very slow growth
O(n)	Linear search, Counting sort	Proportional
O(n log n)	Merge sort, Heap sort, Quick sort (avg)	Almost linear
O(n^2)	Bubble sort, Selection sort, Insertion sort	Quadratic
O(2^n)	Fibonacci (naive), Subset generation	Exponential
O(n!)	Traveling salesman (brute force)	Factorial

Space Complexity Classes

Complexity	Description	Examples
O(1)	In-place algorithms	Swap, iterative factorial
O(log n)	Recursive divide & conquer	Binary search (recursive)
O(n)	Linear storage	Array copy, recursion depth
O(n^2)	2D matrices	Adjacency matrix

Common Patterns

Pattern 1: Single Loop

```
for (int i = 0; i < n; i++) { }
// Time: O(n)
// Space: O(1)
```

Pattern 2: Nested Loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { }
}
// Time: O(n²)
```

Pattern 3: Divide and Conquer

```
public void Process(int[] arr, int start, int end) {
    if (start >= end) return;
    int mid = (start + end) / 2;
    Process(arr, start, mid);     // T(n/2)
    Process(arr, mid + 1, end);   // T(n/2)
    Merge(arr, start, mid, end);  // O(n)
}
// Time: O(n log n)
```

Pattern 4: Backtracking

```
public void GenerateSubsets(int[] nums, int index, List<int> current) {
    if (index == nums.Length) {
        // Process subset
        return;
    }
    // Exclude current
    GenerateSubsets(nums, index + 1, current);
    // Include current
    current.Add(nums[index]);
    GenerateSubsets(nums, index + 1, current);
    current.RemoveAt(current.Count - 1);
}
// Time: O(2ⁿ) - binary choice at each level
// Space: O(n) - recursion depth
```

Time Complexity Growth Rates

```
Operations
10^12 ┤
10^9  ┤
10^6  ┤           O(2^n)
10^3  ┤       O(n^2)
100   ┤   O(n log n)
10    ┤ O(n)
1     ┼──────O(log n)──────O(1)──────────►
      1   10  100  1K   10K   100K   n
```

Comparison Table (Operations for n=1,000,000)

Complexity	Operations	Time (1µs/op)
O(1)	1	1 microsecond
O(log n)	~20	20 µs
O(n)	1,000,000	1 second
O(n log n)	20,000,000	20 seconds
O(n^2)	10^{12}	11.5 days
O(2^n)	$2^{1000000}$	10^{301056} years!

Graph: Time Complexity Comparison

```
Operations (log scale)
│
│                                    O(2^n)
│
│                        O(n^2)
│
```

```
        O(n log n)

    O(n)

O(log n)

└──────────►
  Input Size (n)
```

Array (Fixed Size)

Operation	Time Complexity	Space Complexity	Notes
Access by Index	**O(1)**	O(1)	Direct memory access
Search (Unsorted)	O(n)	O(1)	Linear search
Search (Sorted)	O(log n)	O(1)	Binary search
Insertion (Beginning)	O(n)	O(1)	Need to shift elements
Insertion (End)	O(1) (if space)	O(1)	If not resizing
Insertion (Middle)	O(n)	O(1)	Need to shift elements
Deletion (Beginning)	O(n)	O(1)	Need to shift elements
Deletion (End)	O(1)	O(1)	
Deletion (Middle)	O(n)	O(1)	Need to shift elements
Update	O(1)	O(1)	Direct assignment

ArrayList/Dynamic Array (C# List<T>)

Operation	Time Complexity	Space Complexity	Notes
Access by Index	**O(1)**	O(1)	
Search	O(n)	O(1)	
Add (End)	**O(1) amortized**	O(n)	May need to resize
Add (Beginning)	O(n)	O(1)	Need to shift
Insert (Middle)	O(n)	O(1)	Need to shift
Remove (End)	O(1)	O(1)	
Remove (Beginning/Middle)	O(n)	O(1)	Need to shift
Contains	O(n)	O(1)	

Linked List (Singly/Doubly)

Operation	Time Complexity	Space Complexity	Notes
Access by Index	O(n)	O(1)	Must traverse
Search	O(n)	O(1)	Must traverse
Insert (Beginning)	**O(1)**	O(1)	Update head
Insert (End)	O(1) with tail, O(n) without	O(1)	
Insert (Middle)	O(n) for search + O(1) insert	O(1)	
Delete (Beginning)	**O(1)**	O(1)	Update head
Delete (End)	O(n) singly, O(1) doubly	O(1)	
Delete (Middle)	O(n)	O(1)	Need to find node

C# Dictionary<TKey, TValue>

Operation	Time Complexity	Space Complexity	Notes
Insert/Add	**O(1) average**, O(n) worst	O(n)	Hash collision worst case
Search/ContainsKey	**O(1) average**, O(n) worst	O(1)	
Delete/Remove	**O(1) average**, O(n) worst	O(1)	
Update	**O(1) average**, O(n) worst	O(1)	
Iteration	O(n)	O(1)	
Get Keys/Values	O(n)	O(n)	Creates new collections

Binary Search Tree (BST) - Balanced

Operation	Time Complexity	Space Complexity	Notes
Search	**O(log n)**	O(log n) recursive, O(1) iterative	Height matters
Insert	**O(log n)**	O(log n) recursive, O(1) iterative	
Delete	**O(log n)**	O(log n) recursive, O(1) iterative	
Min/Max	O(log n)	O(log n)	
Traversal	O(n)	O(n) worst, O(log n) average	
Successor/Predecessor	O(log n)	O(1)	

BST - Unbalanced (Worst Case)

Operation	Time Complexity	Space Complexity	Notes
Search	O(n)	O(n)	Degenerates to linked list
Insert	O(n)	O(n)	
Delete	O(n)	O(n)	
Traversal	O(n)	O(n)	

AVL/Red-Black Trees (Self-balancing)

Operation	Time Complexity	Space Complexity	Notes
All operations	**O(log n) guaranteed**	O(log n)	Maintains balance

Binary Heap (Min/Max Heap)

Operation	Time Complexity	Space Complexity	Notes
Insert/Push	**O(log n)**	O(1)	Heapify-up
Extract Min/Max (Pop)	**O(log n)**	O(1)	Heapify-down
Peek (Get Min/Max)	**O(1)**	O(1)	Root element
Search	O(n)	O(1)	Not designed for search
Delete (arbitrary)	O(n) for find + O(log n)	O(1)	
Heapify (Build Heap)	**O(n)**	O(1)	Bottom-up construction
Heap Sort	O(n log n)	O(1)	In-place

C# Stack<T>

Operation	Time Complexity	Space Complexity	Notes
Push	**O(1)** average, O(n) worst	O(1)	Resizing if needed
Pop	**O(1)**	O(1)	
Peek/Top	**O(1)**	O(1)	
Search	O(n)	O(1)	Need to pop elements
Contains	O(n)	O(1)	Linear search
IsEmpty	O(1)	O(1)	Count == 0

C# Queue<T>

Operation	Time Complexity	Space Complexity	Notes
Enqueue	**O(1)** average, O(n) worst	O(1)	Resizing if needed
Dequeue	**O(1)**	O(1)	
Peek/Front	**O(1)**	O(1)	
Search	O(n)	O(1)	
Contains	O(n)	O(1)	Linear search
IsEmpty	O(1)	O(1)	Count == 0

Comparison Summary Table

Data Structure	Best For	Access	Search	Insert	Delete	Space
Array	Fixed size collections	O(1)	O(n)	O(n)	O(n)	O(n)
ArrayList/List	Dynamic arrays	O(1)	O(n)	O(1) end, O(n) middle	O(n)	O(n)
LinkedList	Frequent insert/delete at ends	O(n)	O(n)	O(1) ends	O(1) ends	O(n)
Hash Table	Fast lookups, no ordering	O(1) avg	O(1) avg	O(1) avg	O(1) avg	O(n)
BST (Balanced)	Ordered data, range queries	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Heap	Priority queue, min/max	O(1) peek	O(n)	O(log n)	O(log n)	O(n)
Stack	LIFO operations	O(1) top	O(n)	O(1) push	O(1) pop	O(n)
Queue	FIFO operations	O(1) front	O(n)	O(1) enqueue	O(1) dequeue	O(n)

Find Mod a number without modulo operator

Description: Find the remainder when dividing `a` by `b` without using the modulo operator.

Examples:

```
Input: a = 5, b = 3
Output: 2
Explanation: 5 % 3 = 2 (5 = 1*3 + 2)

Input: a = 10, b = 3
Output: 1
Explanation: 10 % 3 = 1 (10 = 3*3 + 1)

Input: a = 7, b = 7
Output: 0
Explanation: 7 % 7 = 0 (7 = 1*7 + 0)
```

Constraints

- Both numbers are positive and greater than zero
- Numbers within integer range (no overflow)
- Cannot use `%` operator

Brute Force Approach

Pseudocode:

```
FUNCTION Mod(a, b)
  // Use mathematical definition: a = (quotient × b) + remainder
  quotient ← INTEGER_DIVIDE(a, b)  // Integer division
  remainder ← a - (quotient × b)    // Subtraction
  RETURN remainder
END FUNCTION
```

Logic:

- If a=5, b=3: quotient=1, remainder=5-(1×3)=2
- Keep subtracting `b` from `a` until `a < b`

Code Implementation:

```
public static int Mod(int a, int b)
{
```

```
    // Both numbers are positive number and greater than zero.
    // Example: a = 5, b = 3
    // 5 / 3 = 1 (integer division)
    int quotient = a / b;
    // Calculate remainder: 5 - (1 * 3) = 2
    int remainder = a - quotient * b;
    // Return remainder: 2
    return remainder;
}
```

```
┌──────────────────────────────────────────────────┐
│                    Mod(5, 3)                     │
└──────────────────────────────────────────────────┘

        Step 1: Integer Division
        ┌──────────────────────────┐
        │ quotient = a / b         │
        │ quotient = 5 / 3         │
        │ quotient = 1             │
        └──────────────────────────┘
                │
                │ (Integer division truncates)
                ▼

        Step 2: Calculate Remainder
        ┌──────────────────────────┐
        │ remainder = a - quotient * b │
        │ remainder = 5 - (1 * 3)  │
        │ remainder = 5 - 3        │
        │ remainder = 2            │
        └──────────────────────────┘
                │
                ▼

        Step 3: Return Result
        Returns: 2 ✅
```

- **Time Complexity :** `O(1)`
- **Space Complexity :** `O(1)`

Divide numbers without division operator

Description: Find the quotient when dividing `a` by `b` without using the division operator.

Examples:

```
Input: a = 10, b = 3
Output: 3
Explanation: 10 / 3 = 3 (quotient only)

Input: a = 5, b = 2
Output: 2
Explanation: 5 / 2 = 2 (quotient only)

Input: a = 10, b = 1
Output: 10
Explanation: 10 / 1 = 10
```

Constraints

- Both numbers are positive and greater than zero
- Numbers within integer range (no overflow)
- Cannot use `/` operator

Brute Force Approach

Pseudocode:

```
FUNCTION Divide(a, b)
  count ← 0
  sum ← b
  WHILE sum ≤ a DO
    count ← count + 1
    sum ← sum + b
  END WHILE
  RETURN count
END FUNCTION
```

Logic:

- Keep adding `b` to `sum` until `sum` exceeds `a`
- Count how many times we added `b`

Code Implementation:

```
public static int Divide(int a, int b)
{
    // Both numbers are positive number and greater than zero.
    // Example: a = 5, b = 3

    int count = 0;
    int sum = b;
    while(sum <= a)
    {
        count++;
        sum += b;
    }

    return count;
}
```

```
Divide(10, 3)
       │
       ├── Initialize: count=0, sum=3
       │
       ├── Loop: sum <= 10?
```

```
    │   ├── sum=3 ≤10 ✓ → count=1, sum=6
    │   ├── sum=6 ≤10 ✓ → count=2, sum=9
    │   ├── sum=9 ≤10 ✓ → count=3, sum=12
    │   └── sum=12 ≤10 ✗ → STOP
    │
    └── Return: count=3
```

- **Time Complexity :** `O(a/b)`
  - Loop runs ceil(a/b) times
- **Space Complexity :** `O(1)`

Binary Search Approach

Pseudocode:

```
FUNCTION DivideOptimized(a, b):
    // Handle edge cases
    IF b = 0: RETURN ERROR (division by zero)
    IF a < b: RETURN 0

    left = 0
    right = a

    WHILE left <= right:
        mid = left + (right - left) / 2
        IF mid * b == a:
            RETURN mid
        ELSE IF mid * b < a:
            left = mid + 1
        ELSE:
            right = mid - 1

    RETURN right  // Floor division
END FUNCTION
```

Code Implementation:

```
public static int DivideBinarySearch(int a, int b)
{
    if (b == 0) throw new DivideByZeroException();
    if (a < b) return 0;

    int left = 0;
    int right = a;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        long product = (long)mid * b;

        if (product == a) return mid;
        if (product < a) left = mid + 1;
        else right = mid - 1;
    }

    return right; // Floor division
}
```

```
Search space: [0, 21]
mid = 10 → 10*4=40 > 21 → right=9
mid = 4 → 4*4=16 < 21 → left=5
mid = 7 → 7*4=28 > 21 → right=6
mid = 5 → 5*4=20 < 21 → left=6
mid = 6 → 6*4=24 > 21 → right=5
Stop: left=6 > right=5
Return right=5 ✅ (21/4 = 5 remainder 1)
```

- **Time Complexity :** `O(log a)`
- **Space Complexity :** `O(1)`

Bitwise Approach

Pseudocode:

```
FUNCTION DivideBitwise(a, b):
    IF b = 0: RETURN ERROR

    // Handle sign
    sign = 1
    IF (a < 0) XOR (b < 0):
        sign = -1

    // Work with absolute values
    dividend = ABS(a)
    divisor = ABS(b)

    quotient = 0

    // Start from most significant bit
    FOR i = 31 DOWNTO 0:
        IF (divisor << i) <= dividend:
            dividend = dividend - (divisor << i)
            quotient = quotient | (1 << i)

    // Apply sign
    quotient = quotient × sign

    RETURN quotient
END FUNCTION
```

```
public static int DivideBitwise(int a, int b)
{
    if (b == 0) throw new DivideByZeroException();

    // Handle signs using XOR
    bool isNegative = (a < 0) ^ (b < 0);
```

```
    long dividend = Math.Abs((long)a);
    long divisor = Math.Abs((long)b);

    long quotient = 0;

    // Start from most significant bit
    for (int i = 31; i >= 0; i--)
    {
        long shiftedDivisor = divisor << i;

        if (shiftedDivisor <= dividend)
        {
            dividend -= shiftedDivisor;
            quotient |= (1L << i);
        }
    }

    // Apply sign
    if (isNegative)
        quotient = -quotient;

    return (int)quotient;
}

a = 28, b = 5
Expected: 28 ÷ 5 = 5 remainder 3

Binary: 5 = 101₂
i=5: 5<<5 = 160 > 28 → skip
i=4: 5<<4 = 80 > 28 → skip
i=3: 5<<3 = 40 > 28 → skip
i=2: 5<<2 = 20 ≤ 28 → dividend=8, quotient=100₂=4
i=1: 5<<1 = 10 > 8 → skip
i=0: 5<<0 = 5 ≤ 8 → dividend=3, quotient=101₂=5

Result: quotient = 5 ✅
```

- **Time Complexity :** `O(1)`
- **Space Complexity :** `O(1)`

Add Two Numbers Without Arithmetic Operators

Description: Add two integers without using arithmetic operators (+, -), using only bitwise operations.

Examples:

```
Input: a = 1, b = 2
Output: 3
Explanation: 1 + 2 = 3 using XOR and AND operations

Input: a = 5, b = 3
Output: 8
Explanation: 5 + 3 = 8 (0101 + 0011 = 1000)

Input: a = 10, b = 20
Output: 30
Explanation: 10 + 20 = 30 using bitwise operations
```

Constraints

- Both inputs are positive integers greater than zero
- Cannot use +, -, *, / operators
- Must use bitwise operations (XOR, AND, shift)
- Result fits within integer range

One-Liner

```
FUNCTION AddOneLiner(a, b):
    // Add b to a by incrementing a, b times
    WHILE b > 0:
        a = a + 1  // Using ++ operator
        b = b - 1  // Using -- operator
    END WHILE

    RETURN a
END FUNCTION

public static int AddOneLiner(int a, int b)
{
    // Using ++ and -- operators (still arithmetic)
    while (b-- > 0) a++;
    return a;
}

Initial: a = 5, b = 3
```

Step	Condition	Operation	After Step
1	b-- > 0 (3 > 0 ✓)	b=3 → true a++ (a=6)	b=2, a=6
2	b-- > 0 (2 > 0 ✓)	b=2 → true a++ (a=7)	b=1, a=7
3	b-- > 0 (1 > 0 ✓)	b=1 → true a++ (a=8)	b=0, a=8
4	b-- > 0 (0 > 0 X)	b=0 → false loop ends	b=-1, a=8

```
Return: a = 8 ✅ (5 + 3 = 8)
```

- **Time Complexity :** `O(b)`
- **Space Complexity :** `O(1)`

Basic Bitwise Addition

Pseudocode:

```
FUNCTION AddWithoutPlus(a, b):
    // Keep adding until no carry left
    WHILE b ≠ 0:
        // Calculate carry (bits where both a and b have 1)
        carry = a AND b

        // Calculate sum without carry
        a = a XOR b

        // Shift carry left for next position
        b = carry LEFT SHIFT 1

    END WHILE

    RETURN a
END FUNCTION
```

```
public static int AddWithoutPlus(int a, int b)
{
    while (b != 0)
    {
        int carry = a & b;
        a = a ^ b;
        b = carry << 1;
    }
    return a;
}
```

```
  0101 (5)
+ 0011 (3)
  ----
  1000 (8)
```

```
Iteration | a (binary) | b (binary) | carry | a ^ b | carry << 1
----------|------------|------------|-------|-------|-----------
Initial   | 0101 (5)   | 0011 (3)   |       |       |
1         | 0101 (5)   | 0011 (3)   | 0001  | 0110  | 0010
After 1   | 0110 (6)   | 0010 (2)   |       |       |
2         | 0110 (6)   | 0010 (2)   | 0010  | 0100  | 0100
After 2   | 0100 (4)   | 0100 (4)   |       |       |
3         | 0100 (4)   | 0100 (4)   | 0100  | 0000  | 1000
After 3   | 0000 (0)   | 1000 (8)   |       |       |
4         | 0000 (0)   | 1000 (8)   | 0000  | 1000  | 0000
Final     | 1000 (8)   | 0000 (0)   |       |       |
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Prime number

Description: Determine if a number is prime or not.

Examples:

```
Input: num = 7
Output: true
Explanation: 7 is only divisible by 1 and itself

Input: num = 10
Output: false
Explanation: 10 is divisible by 2 and 5

Input: num = 1
Output: false
Explanation: 1 is not prime by definition
```

Constraints

- Input is a positive integer
- Prime numbers are greater than 1
- Only divisible by 1 and itself

Brute Force Approach

Pseudocode:

```
FUNCTION IsPrimeNumber(num)
  IF num ≤ 1 THEN RETURN false
  IF num = 2 THEN RETURN true
  FOR i = 2 TO num-1 DO
    IF num MOD i = 0 THEN
      RETURN false
    END IF
  END FOR
  RETURN true
END FUNCTION
```

Logic:

- Numbers ≤ 1 are not prime
- 2 is the only even prime
- Check all numbers from 2 to num-1
- If any number divides num evenly (remainder = 0), num is not prime
- If no divisors found, num is prime

Code Implementation:

```
public static bool IsPrimeNumber(int num)
{
    // Numbers less than or equal to 1 are not prime
```

```
    if (num <= 1)
    {
        return false;
    }

    // 2 is the only even prime number
    if (num == 2)
    {
        return true;
    }

    // Check all numbers from 2 to num-1
    for (int i = 2; i < num; i++)
    {
        if (num % i == 0)
        {
            return false; // Found a divisor
        }
    }

    return true; // No divisors found
}
```

```
Example 1: num = 7
┌──────────────────────────┐
│ i=2: 7 % 2 = 1 ≠ 0       │
│ i=3: 7 % 3 = 1 ≠ 0       │
│ i=4: 7 % 4 = 3 ≠ 0       │
│ i=5: 7 % 5 = 2 ≠ 0       │
│ i=6: 7 % 6 = 1 ≠ 0       │
└──────────────────────────┘
No divisors found → return true ✅

Example 2: num = 10
┌──────────────────────────┐
│ i=2: 10 % 2 = 0 ✓        │
│ Found divisor → return false │
└──────────────────────────┘
```

- **Time Complexity :** `O(n)`
  - Loop runs from 2 to n-1
- **Space Complexity :** `O(1)`

Prime Optimized

Pseudocode:

```
FUNCTION IsPrimeNumberOptimized(num):
    IF num ≤ 1 THEN RETURN false
    IF num = 2 THEN RETURN true
    IF num % 2 = 0 THEN RETURN false

    limit = floor(√num)

    FOR i = 3 TO limit STEP 2 DO
        IF num % i = 0 THEN
            RETURN false
        END IF
    END FOR

    RETURN true
END FUNCTION
```

Logic:

1. **Early rejections:**
   - Numbers ≤ 1 are not prime
   - 2 is the only even prime
   - All other even numbers are not prime
2. **Optimized checking range:**
   - Only check divisors up to √num
   - If num has a divisor > √num, it must have a corresponding divisor < √num
   - Example: For num=100, only need to check up to 10
3. **Skip even divisors:**
   - After checking 2, only check odd numbers (3, 5, 7, ...)
   - Reduces checks by half

Code Implementation:

```
public static bool IsPrimeNumberOptimized(int num)
{
    // Numbers less than or equal to 1 are not prime
    if (num <= 1)
    {
        return false;
    }

    // 2 is the only even prime number
    if (num == 2)
    {
        return true;
    }

    // All other even numbers are not prime
    if (num % 2 == 0)
    {
        return false;
    }

    // Check only odd divisors up to square root
    int limit = (int)Math.Sqrt(num);
```

```
    for (int i = 3; i <= limit; i += 2)
    {
        if (num % i == 0)
        {
            return false; // Found a divisor
        }
    }

    return true; // No divisors found
}
```

```
Example 1: num = 17
limit = √17 ≈ 4
Check: 3, 5
┌──────────────────────────┐
│ i=3: 17 % 3 = 2 ≠ 0      │
│ i=5: Skip (5 > 4)        │
└──────────────────────────┘
No divisors found → return true ✅

Example 2: num = 100
limit = √100 = 10
Check: 3, 5, 7, 9
┌──────────────────────────┐
│ i=3: 100 % 3 = 1 ≠ 0     │
│ i=5: 100 % 5 = 0 ✓       │
│ Found divisor → return false │
└──────────────────────────┘
```

- **Time Complexity :** `O(√n)`
  - Loop runs up to √n, checking only odd numbers
  - Much faster for large numbers
- **Space Complexity :** `O(1)`

Square root of a number

Description: Find the integer square root (floor) of a non-negative number `n`.

Examples:

```
Input: n = 9
Output: 3
Explanation: √9 = 3.0, floor = 3

Input: n = 25
Output: 5
Explanation: √25 = 5.0, floor = 5

Input: n = 8
Output: 2
Explanation: √8 ≈ 2.82, floor = 2
```

Constraints

- Input is a non-negative integer
- Return the floor of the square root
- Assume no floating point arithmetic needed

Brute Force Approach

Pseudocode:

```
FUNCTION Sqrt(n)
  IF n = 0 THEN RETURN 0
  i ← 1
  WHILE i × i ≤ n DO
    i ← i + 1
  END WHILE
  RETURN i - 1  // Floor of sqrt
END FUNCTION
```

Logic:

- Start from `1` and increment until `i² > n`
- Return `i-1` (last `i` where `i² ≤ n`)
- Finds floor of square root

Code Implementation:

```
public static int Sqrt(int n)
{
    // The number is a positive number and greater than zero.
    if (n == 0)
    {
        return 0;
    }

    int i = 1;
    while (i * i <= n)
    {
        i++;
    }

    return i - 1;  // Floor sqrt
}
```

```
n = 25
i = 1

Loop iterations:
┌──────────────────────────────────────┐
│ Iteration 1: i=1, 1*1=1 <=25 ✓ → i=2  │
│ Iteration 2: i=2, 4 <=25 ✓ → i=3      │
```

```
│ Iteration 3: i=3, 9 <=25 ✓ → i=4      │
│ Iteration 4: i=4, 16 <=25 ✓ → i=5     │
│ Iteration 5: i=5, 25 <=25 ✓ → i=6     │
│ Iteration 6: i=6, 36 <=25 ✗ → STOP    │
└────────────────────────────────────────┘

return i-1 = 6-1 = 5 ✅
```

- **Time Complexity :** `O(√n)`
  - Loop runs while `i² ≤ n` → `i ≤ √n`
- **Space Complexity :** `O(1)`

Sqrt Optimized

Pseudocode:

```
FUNCTION sqrt_optimized(n):
    IF n < 2:
        RETURN n

    left = 1
    right = n / 2

    WHILE left <= right:
        mid = left + (right - left) / 2
        square = mid * mid

        IF square == n:
            RETURN mid
        IF square < n:
            left = mid + 1
        ELSE:
            right = mid - 1

    RETURN right  // Floor sqrt
```

Logic:

- Use **binary search** between `1` and `n/2` (since √n ≤ n/2 for n ≥ 4)
- Repeatedly check middle value `mid`:
  - If `mid² = n`: Found exact square root, return `mid`
  - If `mid² < n`: Square root is larger, search **right half** (set `left = mid + 1`)
  - If `mid² > n`: Square root is smaller, search **left half** (set `right = mid - 1`)
- When search ends (`left > right`), `right` holds the **last valid `mid`** where `mid² ≤ n`
- Returns **floor of square root** (nearest integer ≤ √n)

Code Implementation:

```
public static int SqrtOptimized(int n)
{
    if (n < 2) return n;

    int left = 1, right = n / 2;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        long square = (long)mid * mid;

        if (square == n) return mid;
        if (square < n) left = mid + 1;
        else right = mid - 1;
    }

    return right;
}
```

```
Search space: 1 to 12
            Check mid=6 (36 > 25)
            /             \
       Too big            Too small
       [1..5]              [7..12] ✗

Check mid=3 (9 < 25)
        /           \
   Too small      Too big
   [4..5]          [1..2] ✗

Check mid=4 (16 < 25)
        /           \
   Too small      Too big
   [5]            [4] ✗

Check mid=5 (25 == 25) ✓ Found!
```

- **Time Complexity :** `O(log n)`
  - Reduces search space by half each iteration
- **Space Complexity :** `O(1)`

Power of two numbers

Description: Calculate the power: a^b (a raised to power b).

Examples:

```
Input: a = 2, b = 3
Output: 8
Explanation: 2^3 = 2 × 2 × 2 = 8

Input: a = 3, b = 2
Output: 9
Explanation: 3^2 = 3 × 3 = 9

Input: a = 2, b = 0
```

```
Output: 1
Explanation: Any number^0 = 1
```

Constraints

- Both numbers are positive integers greater than zero
- Numbers within integer range (no overflow/stack overflow)
- Base and exponent are non-negative

Brute Force Approach

Pseudocode:

```
FUNCTION Power(a, n):
    // Validate input
    IF n < 0:
        RETURN -1  // Error: negative exponent

    // Base case: a⁰ = 1
    IF n == 0:
        RETURN 1

    // Recursive case: aⁿ = a × aⁿ⁻¹
    smallerProblem = Power(a, n - 1)

    // Combine: multiply by a
    result = a × smallerProblem

    RETURN result
END FUNCTION
```

```
public static int Power(int a, int n)
{
    // Both numbers are positive number and greater than zero.
    // Handle edge cases
    if (n < 0)
    {
        // Exponent must be non-negative
        return -1;
    }

    // Base case
    if (n == 0)
    {
        return 1;
    }

    // Recursive case
    int remainingProblem = Power(a, n - 1);

    // Combine solutions
    return a * remainingProblem;
}
```

```
Power(3, 4)
  → Power(3, 3)
    → Power(3, 2)
      → Power(3, 1)
        → Power(3, 0)  // Base case
        ← returns 1
      ← returns 3 * 1 = 3
    ← returns 3 * 3 = 9
  ← returns 3 * 9 = 27
← returns 3 * 27 = 81
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

Binary Exponentiation

The naive approach makes n recursive calls for computing a^n. We can optimize this to O(log n) using **binary exponentiation** by exploiting the following property:

- If n is even: $a^n = (a^2)^{(n/2)}$
- If n is odd: $a^n = a \times a^{(n-1)} = a \times (a^2)^{((n-1)/2)}$

Pseudocode:

```
FUNCTION PowerBinaryExponentiation(a, n):
    // Validate input
    IF n < 0:
        RETURN -1  // Error: negative exponent

    // Base case: any number to power 0 is 1
    IF n == 0:
        RETURN 1

    // Recursively compute half power
    halfPower = PowerBinaryExponentiation(a, n DIV 2)

    // Recombine results using divide and conquer
    IF n MOD 2 == 0:          // n is even
        RETURN halfPower × halfPower
    ELSE:                     // n is odd
        RETURN a × halfPower × halfPower
    END IF
END FUNCTION
```

```
public static int PowerBinaryExponentiation(int a, int n)
{
    // Handle edge cases
    if (n < 0)
    {
        return -1;
    }

    // Base case
    if (n == 0)
```

```
    {
        return 1;
    }

    // Recursive case with binary exponentiation
    int halfPower = PowerBinaryExponentiation(a, n / 2);

    // If n is even: a^n = (a^(n/2))^2
    if (n % 2 == 0)
    {
        return halfPower * halfPower;
    }
    else
    {
        // If n is odd: a^n = a × (a^(n/2))^2
        return a * halfPower * halfPower;
    }
}
```

```
Power(3, 8) - Binary Exponentiation

Naive Approach:
3^8 = 3^7 = 3^6 = 3^5 = 3^4 = 3^3 = 3^2 = 3^1 = 3^0
(8 recursive calls)

Binary Exponentiation:
        PowerBinaryExp(3, 8)
        n=8 (even) → compute (a²)^(n/2) = (3²)^4 = 9^4
                    │
                    ├─ PowerBinaryExp(3, 4)
                    │  n=4 (even) → compute (a²)^(n/2) = (3²)^2 = 9^2
                    │            │
                    │            ├─ PowerBinaryExp(3, 2)
                    │            │  n=2 (even) → compute (a²)^(n/2) = (3²)^1 = 9^1
                    │            │            │
                    │            │            ├─ PowerBinaryExp(3, 1)
                    │            │            │  n=1 (odd) → 3 × (3^0)^2 = 3 × 1 = 3
                    │            │            │
                    │            │            ├─ halfPower = 3
                    │            │            └─ return 3^2 = 9
                    │            │
                    │            ├─ halfPower = 9
                    │            └─ return 9^2 = 81
                    │
                    ├─ halfPower = 81
                    └─ return 81^2 = 6561

Result: 3^8 = 6561 ✓
(Only 4 recursive calls instead of 8!)

Comparison:
Power(2, 16):
  Naive: 16 calls
  Binary Exponentiation: log₂(16) = 4 calls ✓

Power(2, 32):
  Naive: 32 calls
  Binary Exponentiation: log₂(32) = 5 calls ✓

Power(2, 1000):
  Naive: 1000 calls
  Binary Exponentiation: log₂(1000) ≈ 10 calls ✓
```

- **Time Complexity :** `O(log n)`
  - Each recursive call reduces n by half (n/2)
  - Maximum depth = $\log_2(n)$
  - Total calls = O(log n)
- **Space Complexity :** `O(log n)`
  - Call stack depth: `O(log n)` (depth of recursion tree)

Itrative Approach

Pseudocode:

```
FUNCTION PowerIterative(a, n):
    IF n < 0:
        RETURN -1  // Error: negative exponent

    result ← 1

    FOR i = 1 TO n DO:
        result ← result * a
    END FOR

    RETURN result
END FUNCTION
```

Code Implementation:

```
public static int PowerIterative(int a, int n)
{
    if (n < 0) return -1;

    int result = 1;
    for (int i = 0; i < n; i++)
    {
        result *= a;
    }
    return result;
}
```

```
Input: a = 3, n = 5
Expected: 3⁵ = 3 × 3 × 3 × 3 × 3 = 243

Initial: result = 1

┌──────────┬────┬──────────────────┬────────┐
│ Iteration │ i  │ Operation        │ result │
```

Start			1
1	0	result = 1 × 3	3
2	1	result = 3 × 3	9
3	2	result = 9 × 3	27
4	3	result = 27 × 3	81
5	4	result = 81 × 3	243

```
Loop ends (i = 5, n = 5, i < n is false)
Return result = 243 ✅
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Optimized with Bitwise

Pseudocode:

```
FUNCTION PowerOptimized(x, n):
    // Validate input
    IF n < 0:
        RETURN 0  // Or handle differently

    // Base case: x⁰ = 1
    IF n == 0:
        RETURN 1

    result = 1

    // Process n bit by bit
    WHILE n > 0:
        // If current bit (LSB) is 1 (n is odd)
        IF (n AND 1) == 1:
            result = result × x

        // Square x for next bit position
        x = x × x

        // Shift n right by 1 (divide by 2)
        n = n RIGHT SHIFT 1

    END WHILE

    RETURN result
END FUNCTION
```

```
public static int PowerOptimized(int x, int n)
{
    if (n < 0) return 0;  // Handle negative
    if (n == 0) return 1;

    int result = 1;

    while (n > 0)
    {
        // If n is odd, multiply result by x
        if ((n & 1) == 1)
        {
            result *= x;
        }

        // Square x and halve n
        x *= x;
        n >>= 1;  // n = n / 2
    }

    return result;
}
```

```
// 3^5 = 3^(101)₂
// 5=101, result=1
// n=101(odd) → result=3, x=9, n=10
// n=10(even)→ result=3, x=81, n=1
// n=1(odd) → result=243, x=6561, n=0
// Result=243 ✓
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Geometric Sum

Description: Calculate the geometric sum: $1 + 1/2 + 1/4 + 1/8 + ... + 1/2^n$

Examples:

```
Input: n = 0
Output: 1.0
Explanation: Sum = 1 (only first term)

Input: n = 1
Output: 1.5
Explanation: Sum = 1 + 0.5 = 1.5

Input: n = 3
Output: 1.875
Explanation: Sum = 1 + 0.5 + 0.25 + 0.125 = 1.875
```

Constraints

- Input n is non-negative integer
- Computes sum from 2^0 to 2^n terms
- Result is a floating-point double

Recursion Approach

Pseudocode:

```
FUNCTION GeometricSum(n)
  IF n == 0 THEN
    RETURN 1.0
  END IF
  RETURN GeometricSum(n - 1) + 1.0 / (2 ^ n)
END FUNCTION

public static double GeometricSum(int n)
{
    // Base case
    if (n == 0)
    {
        return 1;
    }

    // Inductive Hypothesis
    double smallResult = GeometricSum(n -1);

    //Inductive Step
    return smallResult + (1.0 / Math.Pow(2, n));
}

Call Stack (growing down):
GeometricSum(3) waiting, will add 1/2³
  GeometricSum(2) waiting, will add 1/2²
    GeometricSum(1) waiting, will add 1/2¹
      GeometricSum(0) → returns 1.0
    returns 1.0 + 0.5 = 1.5
  returns 1.5 + 0.25 = 1.75
returns 1.75 + 0.125 = 1.875
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(d)`

Iteration Approach

Pseudocode:

```
FUNCTION GeometricSum(n):
    // Initialize: 1/2⁰ = 1
    sum ← 1.0
    term ← 1.0

    // Loop from i=1 to n (terms 1/2¹ to 1/2ⁿ)
    FOR i = 1 TO n DO:
        // Each term is half of previous term
        term ← term / 2.0

        // Add term to running total
        sum ← sum + term
    END FOR

    RETURN sum
END FUNCTION

public static double GeometricSum(int n)
{
    double sum = 1.0;  // 1/2⁰ = 1
    double term = 1.0;

    for (int i = 1; i <= n; i++)
    {
        term /= 2.0;  // Each term is half of previous
        sum += term;
    }
    return sum;
}

Initial:
term = 1.0 = 1/2⁰
sum = 1.0 = 1/2⁰

i=1:
term = 1.0/2 = 0.5 = 1/2¹
sum = 1.0 + 0.5 = 1.5 = 1 + 1/2

i=2:
term = 0.5/2 = 0.25 = 1/2²
sum = 1.5 + 0.25 = 1.75 = 1 + 1/2 + 1/4

i=3:
term = 0.25/2 = 0.125 = 1/2³
sum = 1.75 + 0.125 = 1.875 = 1 + 1/2 + 1/4 + 1/8

i=4:
term = 0.125/2 = 0.0625 = 1/2⁴
sum = 1.875 + 0.0625 = 1.9375 = 1 + 1/2 + 1/4 + 1/8 + 1/16
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Count Zeroes

Description: Count the total number of zero digits in a positive integer.

Examples:

```
Input: n = 101010
Output: 3
Explanation: Digits are 1,0,1,0,1,0. Three zeros present.

Input: n = 1000
Output: 3
Explanation: Digits are 1,0,0,0. Three zeros present.

Input: n = 12345
```

```
Output: 0
Explanation: No zeros in the number.
```

Constraints

- Input is a positive integer
- Count total number of zero digits
- No negative numbers

Recursion Approch

Pseudocode:

```
FUNCTION CountZeroes(n)
  IF n <= 0 THEN
    RETURN 0
  END IF
  small ← CountZeroes(n / 10)
  IF (n MOD 10) == 0 THEN
    RETURN small + 1
  ELSE
    RETURN small
  END IF
END FUNCTION
```

```
public static int CountZeroes(int n)
{
    if (n <= 0)
    {
        return 0;
    }

    int smallResult = CountZeroes(n / 10);
    int lastDigit = n % 10;
    if (lastDigit == 0)
    {
        return smallResult + 1;
    }
    return smallResult;
}
```

```
Call Stack (growing down):
CountZeroes(100304) waiting, lastDigit=4≠0
  CountZeroes(10030) waiting, lastDigit=0=0
    CountZeroes(1003) waiting, lastDigit=3≠0
      CountZeroes(100) waiting, lastDigit=0=0
        CountZeroes(10) waiting, lastDigit=0=0
          CountZeroes(1) waiting, lastDigit=1≠0
            CountZeroes(0) → returns 0
          returns 0
        returns 0 + 1 = 1
      returns 1 + 1 = 2
    returns 2
  returns 2 + 1 = 3
returns 3
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(log d)`

Iteration Approach

Pseudocode:

```
FUNCTION CountZeroes(n)
    IF n <= 0 THEN RETURN 0
    count ← 0
    WHILE n > 0 DO
        IF n % 10 == 0 THEN count ← count + 1
        n ← n / 10
    END WHILE
    RETURN count
END FUNCTION
```

```
public static int CountZeroes(int n)
{
        if (n <= 0) return 0;
        int count = 0;
        while (n > 0)
    {
        if (n % 10 == 0) count++;
        n /= 10;
    }
    return count;
}
```

```
n=101010, count=0
Process digits: 0,1,0,1,0,1
Zeros at positions: 1st, 3rd, 5th from right
Result: 3 ✓
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Count digits

Description: Count the number of digits in a positive integer using recursion.

Examples:

```
Input: n = 123
Output: 3
Explanation: 123 has 3 digits

Input: n = 9
Output: 1
```

```
Explanation: Single digit number

Input: n = 1000
Output: 4
Explanation: 1000 has 4 digits
```

Constraints

- Input is a positive integer
- Count total number of digits in the number
- No negative numbers

Recursion Approach

Pseudocode:

```
FUNCTION CountDigits(n)
  IF n <= 0 THEN
    RETURN 0
  END IF
  RETURN 1 + CountDigits(n / 10)
END FUNCTION

public static int CountDigits(int n)
{
    if (n <= 0)
    {
        return 0;
    }

    return 1 + CountDigits(n / 10);
}

Call Stack (growing down):
CountDigits(1234)      → waiting for result
  CountDigits(123)     → waiting for result
    CountDigits(12)    → waiting for result
      CountDigits(1)   → waiting for result
        CountDigits(0) → returns 0
      returns 1 + 0 = 1
    returns 1 + 1 = 2
  returns 1 + 2 = 3
returns 1 + 3 = 4

CountDigits(1234) = 1 + CountDigits(123)
                  = 1 + (1 + CountDigits(12))
                  = 1 + (1 + (1 + CountDigits(1)))
                  = 1 + (1 + (1 + (1 + CountDigits(0))))
                  = 1 + (1 + (1 + (1 + 0)))
                  = 1 + (1 + (1 + 1))
                  = 1 + (1 + 2)
                  = 1 + 3
                  = 4
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(log n)`

Iteration Approach

Pseudocode:

```
FUNCTION CountDigitsIterative(n):
    // Handle edge cases
    IF n == 0:
        RETURN 1  // 0 has 1 digit

    // Make number positive
    number = ABS(n)

    count = 0

    // Count digits by repeatedly dividing by 10
    WHILE number > 0:
        count = count + 1
        number = number / 10  // Integer division

    END WHILE

    RETURN count
END FUNCTION

public static int CountDigitsIterative(int n)
{
    // Special case for 0
    if (n == 0)
        return 1;

    // Make number positive to handle negatives
    int number = Math.Abs(n);
    int count = 0;

    while (number > 0)
    {
        count++;
        number /= 10;  // Remove last digit
    }

    return count;
}

Number: 12345
Expected: 5 digits

Step-by-step:
┌─────────┬─────────────┬──────────────┬─────────────┐
│ Step    │ number value │ operation    │ count value │
├─────────┼─────────────┼──────────────┼─────────────┤
│ Start   │ 12345       │              │ 0           │
```

```
│ 1      │ 12345 > 0 ✓ │ count++      │ 1            │
│        │             │ number/=10   │ 1234         │
├────────┼─────────────┼──────────────┼──────────────┤
│ 2      │ 1234 > 0 ✓  │ count++      │ 2            │
│        │             │ number/=10   │ 123          │
├────────┼─────────────┼──────────────┼──────────────┤
│ 3      │ 123 > 0 ✓   │ count++      │ 3            │
│        │             │ number/=10   │ 12           │
├────────┼─────────────┼──────────────┼──────────────┤
│ 4      │ 12 > 0 ✓    │ count++      │ 4            │
│        │             │ number/=10   │ 1            │
├────────┼─────────────┼──────────────┼──────────────┤
│ 5      │ 1 > 0 ✓     │ count++      │ 5            │
│        │             │ number/=10   │ 0            │
├────────┼─────────────┼──────────────┼──────────────┤
│ 6      │ 0 > 0 ✗     │ Loop ends    │              │
└────────┴─────────────┴──────────────┴──────────────┘

Return: count = 5 ✅
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Count Number of 1 Bits

Description: Count the number of 1s in the binary representation of an integer.

Examples:

```
Input: n = 11 (binary: 1011)
Output: 3
Explanation: Three 1 bits in binary 1011

Input: n = 128 (binary: 10000000)
Output: 1
Explanation: Single 1 bit

Input: n = 7 (binary: 111)
Output: 3
Explanation: Three 1 bits
```

Constraints

- Input is a positive number greater than zero
- Count total number of 1 bits in binary representation
- Use efficient bit manipulation techniques

Iteration Approach

Pseudocode:

```
FUNCTION CountBits(n)
    count ← 0
    WHILE n > 0 DO
        count ← count + (n & 1)
        n ← n >> 1
    END WHILE
    RETURN count
END FUNCTION

public static int CountBits(int n)
{
    int count = 0;
    while(n > 0)
    {
        count += n & 1;
        n = n >> 1;
    }
    return count;
}

n = 13 (1101)
     ↓
Iteration 1: Check 1✓ → count=1, n=6(110)
Iteration 2: Check 0✗ → count=1, n=3(11)
Iteration 3: Check 1✓ → count=2, n=1(1)
Iteration 4: Check 1✓ → count=3, n=0

Return: 3
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Sum of digits

Description: Calculate the sum of all digits in a positive integer.

Examples:

```
Input: num = 123
Output: 6
Explanation: 1 + 2 + 3 = 6

Input: num = 9876
Output: 30
Explanation: 9 + 8 + 7 + 6 = 30

Input: num = 1000
Output: 1
Explanation: 1 + 0 + 0 + 0 = 1
```

Constraints

- Input is a positive integer greater than zero
- Sum all individual digits
- No negative numbers

Resursive Approach

Pseudocode:

```
FUNCTION SumOfDigits(n)
  IF n <= 0 THEN
    RETURN 0
  END IF
  RETURN SumOfDigits(n / 10) + (n MOD 10)
END FUNCTION
```

```
public static int SumOfDigits(int n)
{
    // Base cases
    if (n <= 0)
    {
        return 0;
    }

    // Inductive Hypothesis
    int smallResult = SumOfDigits(n / 10);

    // Inductive Step
    int lastDigit = n % 10;
    return smallResult + lastDigit;
}
```

```
Call Stack (growing down):
SumOfDigits(1234) waiting, lastDigit=4
  SumOfDigits(123) waiting, lastDigit=3
    SumOfDigits(12) waiting, lastDigit=2
      SumOfDigits(1) waiting, lastDigit=1
        SumOfDigits(0) → returns 0
      returns 0 + 1 = 1
    returns 1 + 2 = 3
  returns 3 + 3 = 6
returns 6 + 4 = 10
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(log d)`

Iteration Approach

Pseudocode:

```
FUNCTION SumOfDigits(num)
  sum ← 0
  WHILE num > 0 DO
    remainder ← num MOD 10      // Extract last digit
    sum ← sum + remainder       // Add to sum
    num ← num / 10              // Remove last digit
  END WHILE
  RETURN sum
END FUNCTION
```

Logic:

- Extract last digit using modulo 10
- Add to running sum
- Remove last digit by integer division
- Repeat until all digits processed

Code Implementation:

```
public static int SumOfDigits(int num)
{
    // The number is a positive number and greater than zero.
    int sum = 0;
    while(num > 0)
    {
        int reminder = num % 10;
        sum += reminder;
        num = num / 10;
    }

    return sum;
}
```

```
SumOfDigits(9876)
──────────────────
Iteration 1: num=9876 → remainder=6, sum=6, num=987
Iteration 2: num=987 → remainder=7, sum=13, num=98
Iteration 3: num=98 → remainder=8, sum=21, num=9
Iteration 4: num=9 → remainder=9, sum=30, num=0
Return: 30 ✓ (9+8+7+6=30)
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Reverse Integer

Description: Given a 32-bit signed integer, reverse its digits. If reversing causes the number to go outside the 32-bit signed integer range `[-2³¹, 2³¹-1]`, return `0`.

Examples:

```
Input: x = 123
Output: 321
Explanation: Digits reversed from 123 to 321

Input: x = -123
Output: -321
```

```
Explanation: Sign preserved, digits reversed

Input: x = 120
Output: 21
Explanation: Leading zero dropped (021 → 21)

Input: x = 1534236469
Output: 0
Explanation: Reversed number 9646324351 exceeds int.MaxValue

Input: x = -2147483648
Output: 0
Explanation: Absolute value can't be represented as positive int
```

Constraints

- Input is a 32-bit signed integer (range: -2^{31} to $2^{31}-1$)
- Must handle both positive and negative numbers
- Must detect overflow/underflow
- Leading zeros should be dropped
- Return 0 when result overflows

Mathematical with Overflow Check

Pseudocode:

```
FUNCTION Reverse(x)
    result ← 0

    WHILE x ≠ 0 DO
        digit ← x % 10
        result ← result * 10 + digit
        x ← x / 10
    END WHILE

    // Overflow checks before multiplication
    IF result > INT_MAX THEN
        RETURN 0
    END IF
    IF result < INT_MIN THEN
        RETURN 0
    END IF
    RETURN result
END FUNCTION
```

Code Implementation:

```
public int Reverse(int x) {
    long result = 0;

    while (x != 0) {
        int digit = x % 10;
        result = result * 10 + digit;
        x /= 10;
    }

    return (result > int.MaxValue || result < int.MinValue) ? 0 : (int)result;
}
```

Visualization

```
Input: x = 123

Initial: result = 0, x = 123

Iteration 1:
┌─────────────────────────────┐
│ digit = 123 % 10 = 3        │
│ Check: 0 ≤ 214748364 ✓      │
│ result = 0*10 + 3 = 3       │
│ x = 123/10 = 12             │
└─────────────────────────────┘

Iteration 2:
┌─────────────────────────────┐
│ digit = 12 % 10 = 2         │
│ Check: 3 ≤ 214748364 ✓      │
│ result = 3*10 + 2 = 32      │
│ x = 12/10 = 1               │
└─────────────────────────────┘

Iteration 3:
┌─────────────────────────────┐
│ digit = 1 % 10 = 1          │
│ Check: 32 ≤ 214748364 ✓     │
│ result = 32*10 + 1 = 321    │
│ x = 1/10 = 0                │
└─────────────────────────────┘

Final: 321 ✅
```

Overflow Example

```
Input: x = 1534236469

Iteration 9:
┌─────────────────────────────┐
│ result = 964632435          │
│ digit = 1                   │
│ Check: 964632435 > 214748364 │
│ RETURN 0                    │
└─────────────────────────────┘

Final: 0 ✅ (overflow detected)
```

Complexity Analysis

- **Time Complexity:** $O(\log_{10} n)$ - Number of digits in input
  - For 32-bit integers: maximum 10 iterations
  - Effectively constant time in practice
- **Space Complexity:** $O(1)$ - Only using a few variables

Recursive with Overflow Check

Pseudocode:

```
FUNCTION Reverse(x)
    RETURN ReverseHelper(ABS(x), 0) * SIGN(x)
END FUNCTION

FUNCTION ReverseHelper(num, result)
    // Base case
    IF num = 0 THEN
        RETURN result
    END IF

    digit ← num % 10
    newResult ← result * 10 + digit

    // Overflow check
    IF newResult > INT_MAX OR newResult < INT_MIN THEN
        RETURN 0
    END IF

    // Recursive call
    RETURN ReverseHelper(num / 10, newResult)
END FUNCTION
```

Code Implementation:

```
public int Reverse(int x) {
    if (x == 0) return 0;

    int sign = x > 0 ? 1 : -1;
    long num = Math.Abs((long)x);
    long result = ReverseHelper(num, 0);

    result *= sign;

    if (result < int.MinValue || result > int.MaxValue)
        return 0;

    return (int)result;
}

private long ReverseHelper(long num, long result) {
    if (num == 0)
        return result;

    long digit = num % 10;
    long newResult = result * 10 + digit;

    // Early overflow check
    if (newResult > int.MaxValue)
        return int.MaxValue + 1; // Signal overflow

    return ReverseHelper(num / 10, newResult);
}
```

Visualization

```
Input: x = 123

Call Stack:
┌─────────────────────────────────────────┐
│ ReverseHelper(123, 0)                   │
│   digit = 3, newResult = 3              │
│   → ReverseHelper(12, 3)                │
│       digit = 2, newResult = 32         │
│       → ReverseHelper(1, 32)            │
│           digit = 1, newResult = 321    │
│           → ReverseHelper(0, 321)       │
│               return 321                │
│           return 321                    │
│       return 321                        │
│   return 321                            │
└─────────────────────────────────────────┘

Apply sign: positive → 321
```

Complexity Analysis

- **Time Complexity:** $O(\log_{10} n)$ - Recursive depth equals number of digits
- **Space Complexity:** $O(\log_{10} n)$ - Call stack depth

Optimized with Bit Manipulation for Overflow

Pseudocode:

```
FUNCTION Reverse(x)
    result ← 0

    WHILE x ≠ 0 DO
        digit ← x % 10

        // Bit-level overflow detection
        // Check if result already has 31 bits set
        IF (result & 0xC0000000) ≠ 0 THEN
```

```
            // High bits set - potential overflow
            IF ABS(result) > 214748364 THEN
                RETURN 0
            END IF
        END IF

        result ← result * 10 + digit
        x ← x / 10
    END WHILE

    RETURN result
END FUNCTION
```

Code Implementation:

```
public int Reverse(int x) {
    int result = 0;

    while (x != 0) {
        int digit = x % 10;

        // Bit manipulation for early overflow detection
        // Check if result has any of the top 2 bits set (potential overflow)
        if ((result & 0xC0000000) != 0) {
            // If top bits are set, check more precisely
            if (Math.Abs(result) > int.MaxValue / 10) {
                return 0;
            }
        }

        // Edge cases at the boundary
        if (result == int.MaxValue / 10 && digit > 7) return 0;
        if (result == int.MinValue / 10 && digit < -8) return 0;

        result = result * 10 + digit;
        x /= 10;
    }

    return result;
}
```

Visualization

```
Binary representation insight:

int.MaxValue = 2147483647 = 0x7FFFFFFF (01111111 11111111 11111111 11111111)
int.MinValue = -2147483648 = 0x80000000 (10000000 00000000 00000000 00000000)

0xC0000000 = 11000000 00000000 00000000 00000000
            (top 2 bits set - indicates number close to overflow)
```

Complexity Analysis

- **Time Complexity:** $O(\log_{10} n)$ - Still processes each digit
- **Space Complexity:** $O(1)$ - Constant space

Reverse Bits

Description: Reverse the bits of a 32-bit unsigned integer. The function should take an unsigned integer and return the unsigned integer with its bits reversed.

Examples:

```
Input: n = 43261596 (00000010100101000001111010011100)
Output: 964176192 (00111001011110000010100101000000)
Explanation: The input binary string represents the unsigned integer 43261596,
so return 964176192 which has reversed bits

Input: n = 4294967293 (11111111111111111111111111111101)
Output: 3221225471 (10111111111111111111111111111111)
Explanation: The input binary string represents the unsigned integer 4294967293,
so return 3221225471 which has reversed bits

Input: n = 0 (00000000000000000000000000000000)
Output: 0 (00000000000000000000000000000000)

Input: n = 1 (00000000000000000000000000000001)
Output: 2147483648 (10000000000000000000000000000000)
```

Constraints

- Input is a 32-bit unsigned integer (range: 0 to 2^{32}-1)
- Must process all 32 bits (including leading zeros)
- Output is also a 32-bit unsigned integer
- No overflow concerns (unsigned integers wrap naturally)

Bit by Bit (Basic)

Pseudocode:

```
FUNCTION ReverseBits(n)
    result ← 0

    FOR i FROM 0 TO 31 DO
        // Shift result left to make room for next bit
        result ← result << 1

        // Get the least significant bit of n
        bit ← n & 1

        // Add the bit to result
        result ← result | bit

        // Shift n right to process next bit
        n ← n >> 1
```

```
    END FOR

    RETURN result
END FUNCTION
```

Code Implementation:

```
public uint ReverseBits(uint n) {
    uint result = 0;

    for (int i = 0; i < 32; i++) {
        // Make room for the new bit
        result <<= 1;

        // Add the least significant bit of n
        result |= (n & 1);

        // Move to the next bit
        n >>= 1;
    }

    return result;
}
```

Visualization

```
Input: n = 43261596 (binary: 00000010100101000001111010011100)

Bit-by-bit reversal process:

┌──────────────────────────────────────────────────────────────────┐
│ Iteration    n (last 8 bits)    bit    result (building)         │
├──────────────────────────────────────────────────────────────────┤
│ Start        ...10011100         -     00000000000000000000000000000000
│ 1            ...1001110(0)       0     00000000000000000000000000000000
│ 2            ...100111(00)       0     00000000000000000000000000000000
│ 3            ...10011(100)       1     00000000000000000000000000000001
│ 4            ...1001(1100)       0     00000000000000000000000000000010
│ 5            ...100(11100)       1     00000000000000000000000000000101
│ 6            ...10(011100)       1     00000000000000000000000000001011
│ 7            ...1(0011100)       1     00000000000000000000000000010111
│ 8            ...(10011100)       1     00000000000000000000000000101111
│ ...          ...                ...    ...
│ 32           00000000(..)        0     00111001011110000010100101000000
└──────────────────────────────────────────────────────────────────┘

Final result: 00111001011110000010100101000000 = 964176192
```

Complexity Analysis

- **Time Complexity:** `O(32)` = `O(1)` - Fixed 32 iterations
- **Space Complexity:** `O(1)` - Only using a few variables

Using a Lookup Table (Optimized)

Pseudocode:

```
// Precompute reversed bits for all 8-bit values
lookup ← ARRAY[256] OF BYTE
FOR i FROM 0 TO 255 DO
    lookup[i] = REVERSE_8BITS(i)
END FOR

FUNCTION ReverseBits(n)
    result ← 0

    // Process 4 bytes (8 bits each)
    FOR i FROM 0 TO 3 DO
        // Get current byte
        byte ← (n >> (8 * i)) & 0xFF

        // Get reversed byte from lookup table
        reversedByte ← lookup[byte]

        // Place reversed byte in correct position
        result ← result | (reversedByte << (8 * (3 - i)))
    END FOR

    RETURN result
END FUNCTION
```

Code Implementation:

```
public class Solution {
    // Lookup table for 8-bit reversal
    private static readonly uint[] LookupTable = new uint[256];

    static Solution() {
        // Precompute reversed bits for all possible bytes
        for (uint i = 0; i < 256; i++) {
            LookupTable[i] = Reverse8Bits(i);
        }
    }

    private static uint Reverse8Bits(uint b) {
        uint result = 0;
        for (int i = 0; i < 8; i++) {
            result = (result << 1) | (b & 1);
            b >>= 1;
        }
        return result;
    }

    public uint ReverseBits(uint n) {
        uint result = 0;
```

```
        // Process 4 bytes
        result |= LookupTable[n & 0xFF] << 24;        // Byte 0 → position 3
        result |= LookupTable[(n >> 8) & 0xFF] << 16; // Byte 1 → position 2
        result |= LookupTable[(n >> 16) & 0xFF] << 8; // Byte 2 → position 1
        result |= LookupTable[(n >> 24) & 0xFF];       // Byte 3 → position 0

        return result;
    }
}
```

Visualization

```
Input: n = 43261596

Step 1: Split into 4 bytes
┌─────────────────────────────────────────────────────────┐
│ n = 00000010 10010100 00011110 10011100                 │
│      └─B3──┘ └─B2──┘ └─B1──┘ └─B0──┘                   │
│                                                         │
│ Byte 0 (B0): 10011100 = 156                             │
│ Byte 1 (B1): 00011110 = 30                              │
│ Byte 2 (B2): 10010100 = 148                             │
│ Byte 3 (B3): 00000010 = 2                               │
└─────────────────────────────────────────────────────────┘

Step 2: Reverse each byte using lookup table
┌─────────────────────────────────────────────────────────┐
│ Lookup[156] = 00111001 (57)    ← 10011100 reversed      │
│ Lookup[30]  = 01111000 (120)   ← 00011110 reversed      │
│ Lookup[148] = 00101001 (41)    ← 10010100 reversed      │
│ Lookup[2]   = 01000000 (64)    ← 00000010 reversed      │
└─────────────────────────────────────────────────────────┘

Step 3: Reassemble in reverse order
┌─────────────────────────────────────────────────────────┐
│ Original: B3 B2 B1 B0                                   │
│ Reversed: B0 B1 B2 B3 (reversed order)                  │
│                                                         │
│ result = Lookup[B0] << 24  |  00111001 00000000 00000000 00000000 │
│        | Lookup[B1] << 16  |  00111001 01111000 00000000 00000000 │
│        | Lookup[B2] << 8   |  00111001 01111000 00101001 00000000 │
│        | Lookup[B3]        |  00111001 01111000 00101001 01000000 │
└─────────────────────────────────────────────────────────┘

Final: 00111001 01111000 00101001 01000000 = 964176192
```

Complexity Analysis

- **Time Complexity:** `O(1)` - Only 4 table lookups and bit operations
- **Space Complexity:** `O(256)` = `O(1)` - Fixed-size lookup table

Divide and Conquer (Masking)

Pseudocode:

```
FUNCTION ReverseBits(n)
    // Swap odd and even bits
    n ← ((n >> 1) & 0x55555555) | ((n & 0x55555555) << 1)

    // Swap consecutive pairs
    n ← ((n >> 2) & 0x33333333) | ((n & 0x33333333) << 2)

    // Swap nibbles (4-bit chunks)
    n ← ((n >> 4) & 0x0F0F0F0F) | ((n & 0x0F0F0F0F) << 4)

    // Swap bytes (8-bit chunks)
    n ← ((n >> 8) & 0x00FF00FF) | ((n & 0x00FF00FF) << 8)

    // Swap 16-bit halves
    n ← (n >> 16) | (n << 16)

    RETURN n
END FUNCTION
```

Code Implementation:

```
public uint ReverseBits(uint n) {
    // Step 1: Swap adjacent bits (1-bit chunks)
    n = ((n >> 1) & 0x55555555) | ((n & 0x55555555) << 1);

    // Step 2: Swap adjacent 2-bit chunks
    n = ((n >> 2) & 0x33333333) | ((n & 0x33333333) << 2);

    // Step 3: Swap adjacent 4-bit chunks (nibbles)
    n = ((n >> 4) & 0x0F0F0F0F) | ((n & 0x0F0F0F0F) << 4);

    // Step 4: Swap adjacent 8-bit chunks (bytes)
    n = ((n >> 8) & 0x00FF00FF) | ((n & 0x00FF00FF) << 8);

    // Step 5: Swap 16-bit halves
    n = (n >> 16) | (n << 16);

    return n;
}
```

Visualization

```
Input: n = 43261596 (00000010 10010100 00011110 10011100)

Let's track the transformation step by step:

Original:
00000010 10010100 00011110 10011100

Step 1: Swap adjacent bits (1-bit chunks)
┌─────────────────────────────────────────────────────────┐
```

```
Mask 0x55555555 = 01010101 01010101 01010101 01010101

Original:   00 00 00 10   10 01 01 00   00 01 11 10   10 01 11 00
Bit pairs:  (0,0) (0,0) (0,1) (1,0) ...
After swap: 00 00 01 00   01 10 10 00   00 10 11 01   01 10 10 00
Result:     00000100 01101000 00101101 01101000
```

```
Step 2: Swap adjacent 2-bit chunks
Mask 0x33333333 = 00110011 00110011 00110011 00110011

Current:    00 00 01 00   01 10 10 00   00 10 11 01   01 10 10 00
2-bit chunks: (00,00) (01,00) (01,10) (10,00) (00,10) (11,01) (01,10) (10,00)
After swap:  00 00 00 01   10 01 00 10   10 00 01 11   10 01 00 10
Result:     00000001 10010010 10000111 10010010
```

```
Step 3: Swap adjacent 4-bit chunks (nibbles)
Mask 0x0F0F0F0F = 00001111 00001111 00001111 00001111

Current:    00000001 10010010 10000111 10010010
Nibbles:    (0000,0001) (1001,0010) (1000,0111) (1001,0010)
After swap: (0001,0000) (0010,1001) (0111,1000) (0010,1001)
Result:     00010000 00101001 01111000 00101001
```

```
Step 4: Swap adjacent 8-bit chunks (bytes)
Mask 0x00FF00FF = 00000000 11111111 00000000 11111111

Current:    00010000 00101001 01111000 00101001
Bytes:      (00010000,00101001) (01111000,00101001)
After swap: (00101001,00010000) (00101001,01111000)
Result:     00101001 00010000 00101001 01111000
```

```
Step 5: Swap 16-bit halves
Current:    0010100100010000 0010100101111000
            └──high 16──┘ └──low 16──┘
After swap: 0010100101111000 0010100100010000
Result:     0010100101111000 0010100100010000
```

```
Final: 00111001 01111000 00101001 01000000 = 964176192 ✅
```

Complexity Analysis

- **Time Complexity:** `O(1)` - Fixed number of operations
- **Space Complexity:** `O(1)` - In-place bit manipulation
- **Operations:** Only 5 steps regardless of input

Using Long (Simplified for Readability)

Pseudocode:

```
FUNCTION ReverseBits(n)
    result ← 0 AS LONG

    FOR i FROM 0 TO 31 DO
        // Get the i-th bit from original
        bit ← (n >> i) & 1

        // Place it in reversed position
        result ← result | (bit << (31 - i))
    END FOR

    RETURN (UINT)result
END FUNCTION
```

Code Implementation:

```
public uint ReverseBits(uint n) {
    ulong result = 0;

    for (int i = 0; i < 32; i++) {
        // Get bit from position i in original
        ulong bit = (ulong)(n >> i) & 1;

        // Place it at position 31-i in result
        result |= bit << (31 - i);
    }

    return (uint)result;
}
```

Visualization

```
Input: n = 43261596 (binary: 00000010 10010100 00011110 10011100)

Bit positions (0 = LSB, 31 = MSB):

Original:
Position: 31                  0
          ↓                   ↓
          00000010 10010100 00011110 10011100

Process:
i=0: bit = bit0 = 0 → place at pos31: 0<<31 = 0
i=1: bit = bit1 = 0 → place at pos30: 0<<30 = 0
i=2: bit = bit2 = 1 → place at pos29: 1<<29 = 0x20000000
i=3: bit = bit3 = 1 → place at pos28: 1<<28 = 0x10000000
...
i=31: bit = bit31 = 0 → place at pos0: 0<<0 = 0
```

```
Result accumulates:
0x00000000
| 0x20000000 = 0x20000000
| 0x10000000 = 0x30000000
| ... (all bits accumulate in reversed order)

Final: 0x3982A940 = 964176192
```

Complexity Analysis

- **Time Complexity:** `O(32)` = `O(1)` - Fixed iterations
- **Space Complexity:** `O(1)` - Using `ulong` for safety

Recursive (Educational)

Pseudocode:

```
FUNCTION ReverseBits(n, pos)
    IF pos = 32 THEN
        RETURN 0
    END IF

    // Get current bit from original at position 'pos'
    bit ← (n >> pos) & 1

    // Recursively get reversed bits for remaining positions
    remaining ← ReverseBits(n, pos + 1)

    // Place current bit at position (31-pos)
    RETURN remaining | (bit << (31 - pos))
END FUNCTION
```

Code Implementation:

```
public uint ReverseBits(uint n) {
    return ReverseBitsHelper(n, 0);
}

private uint ReverseBitsHelper(uint n, int pos) {
    if (pos == 32) {
        return 0;
    }

    // Get bit at current position
    uint bit = (n >> pos) & 1;

    // Recursively process higher positions
    uint remaining = ReverseBitsHelper(n, pos + 1);

    // Place current bit at reversed position
    return remaining | (bit << (31 - pos));
}
```

Visualization

```
Input: n = 5 (binary: 00000000 00000000 00000000 00000101)

Call Stack (growing down):
┌─────────────────────────────────────────────────────────┐
│ ReverseBitsHelper(n, 0)                                 │
│   bit = bit0 = 1                                        │
│   waiting for ReverseBitsHelper(n, 1)                   │
│   ↓                                                     │
│   ReverseBitsHelper(n, 1)                               │
│     bit = bit1 = 0                                      │
│     waiting for ReverseBitsHelper(n, 2)                 │
│     ↓                                                   │
│     ReverseBitsHelper(n, 2)                             │
│       ...                                               │
│       ↓                                                 │
│       ReverseBitsHelper(n, 31)                          │
│         bit = bit31 = 0                                 │
│         ReverseBitsHelper(n, 32) → returns 0            │
│         returns 0 | (0<<0) = 0                         │
│       returns prev | (bit30<<1) = 0 | 0 = 0             │
│       ...                                               │
│     returns 0 | (bit2<<29) = (0 | 0)                   │
│   returns 0 | (bit1<<30) = 0                            │
│ returns 0 | (bit0<<31) = (1 << 31) = 0x80000000         │
└─────────────────────────────────────────────────────────┘

Final: 0x80000000 = 2147483648 ✅
```

Complexity Analysis

- **Time Complexity:** `O(32)` = `O(1)` - 32 recursive calls
- **Space Complexity:** `O(32)` = `O(1)` - Call stack depth (fixed)

Parity of a 32-bit Unsigned Integer

Description: Compute the parity of a 32-bit unsigned integer. Parity is 1 if the number of 1 bits is odd, and 0 if the number of 1 bits is even.

Examples:

```
Input: x = 5 (binary: 00000000 00000000 00000000 00000101)
Output: 0
Explanation: 5 has two 1 bits (positions 0 and 2) → even → parity 0

Input: x = 7 (binary: 00000000 00000000 00000000 00000111)
Output: 1
Explanation: 7 has three 1 bits → odd → parity 1
```

```
Input: x = 0 (binary: 00000000 00000000 00000000 00000000)
Output: 0
Explanation: 0 has zero 1 bits (even) → parity 0

Input: x = 0xFFFFFFFF (binary: all 32 bits are 1)
Output: 0
Explanation: 32 is even → parity 0

Input: x = 0x80000001 (binary: 10000000 00000000 00000000 00000001)
Output: 0
Explanation: Two 1 bits → even → parity 0

Input: x = 0x80000000 (binary: 10000000 00000000 00000000 00000000)
Output: 1
Explanation: One 1 bit → odd → parity 1
```

Constraints

- Input is a 32-bit unsigned integer (range: 0 to 2^{32}-1)
- Output is a single bit (0 for even parity, 1 for odd parity)
- Must handle all 32 bits efficiently

Brute Force (Bit Counting)

Pseudocode:

```
FUNCTION Parity(x)
    count ← 0

    FOR i FROM 0 TO 31 DO
        IF (x & 1) = 1 THEN
            count ← count + 1
        END IF
        x ← x >> 1
    END FOR

    RETURN count % 2
END FUNCTION
```

Code Implementation:

```
public int Parity(uint x) {
    int count = 0;

    for (int i = 0; i < 32; i++) {
        // Check least significant bit
        if ((x & 1) == 1) {
            count++;
        }
        // Shift right to check next bit
        x >>= 1;
    }

    return count % 2;
}
```

Visualization

```
Input: x = 7 (00000000 00000000 00000000 00000111)

Bit-by-bit counting:
```

Iteration	x (LSB first)	bit	count
Start	...00000111	-	0
1	...0000011(1)	1	1
2	...000001(11)	1	2
3	...00000(111)	1	3
4	...0000(0111)	0	3
5	...000(00111)	0	3
...	...	...	...
32	00000000(..)	0	3

```
count = 3 → 3 % 2 = 1 → parity = 1 ✅
```

Complexity Analysis

- **Time Complexity:** `O(32)` = `O(1)` - Fixed 32 iterations
- **Space Complexity:** `O(1)` - Only using count variable

Brian Kernighan's Algorithm

Pseudocode:

```
FUNCTION Parity(x)
    count ← 0

    WHILE x ≠ 0 DO
        // Clear the lowest set bit
        x ← x & (x - 1)
        count ← count + 1
    END WHILE

    RETURN count % 2
END FUNCTION
```

Code Implementation:

```
public int Parity(uint x) {
    int count = 0;
```

```
    while (x != 0) {
        // Clear the lowest set bit
        x &= (x - 1);
        count++;
    }

    return count % 2;
}
```

Visualization

```
Input: x = 7 (00000000 00000000 00000000 00000111)

Step-by-step bit clearing:

┌──────────────────────────────────────────────────────────┐
│ Step    x (binary)          x-1          x & (x-1)       │
├──────────────────────────────────────────────────────────┤
│ 1       00000111           00000110     00000110         │
│         ↑ clears bit 0                  count = 1        │
│                                                          │
│ 2       00000110           00000101     00000100         │
│         ↑ clears bit 1                  count = 2        │
│                                                          │
│ 3       00000100           00000011     00000000         │
│         ↑ clears bit 2                  count = 3        │
│                                                          │
│ Loop ends: x = 0                                         │
└──────────────────────────────────────────────────────────┘

count = 3 → 3 % 2 = 1 → parity = 1 ✅
```

Complexity Analysis

- **Time Complexity:** `O(k)` where k is number of 1 bits
  - Best case: O(1) (when x = 0)
  - Worst case: O(32) (when all bits are 1)
- **Space Complexity:** `O(1)`

XOR Folding (Divide and Conquer)

Pseudocode:

```
FUNCTION Parity(x)
    // Fold 32 bits → 16 bits
    x ← x ^ (x >> 16)

    // Fold 16 bits → 8 bits
    x ← x ^ (x >> 8)

    // Fold 8 bits → 4 bits
    x ← x ^ (x >> 4)

    // Fold 4 bits → 2 bits
    x ← x ^ (x >> 2)

    // Fold 2 bits → 1 bit
    x ← x ^ (x >> 1)

    // Return only the least significant bit
    RETURN (int)(x & 1)
END FUNCTION
```

Code Implementation:

```
public int Parity(uint x) {
    // XOR folding technique
    x ^= x >> 16;
    x ^= x >> 8;
    x ^= x >> 4;
    x ^= x >> 2;
    x ^= x >> 1;

    // Return only the LSB (the parity bit)
    return (int)(x & 1);
}
```

Visualization

```
Input: x = 7 (00000000 00000000 00000000 00000111)

Step-by-step XOR folding:

Step 1: Fold 32 → 16 bits

┌──────────────────────────────────────────────────────────┐
│ x       = 00000000 00000000 00000000 00000111            │
│ x >> 16 = 00000000 00000000 00000000 00000000            │
│ XOR     = 00000000 00000000 00000000 00000111            │
└──────────────────────────────────────────────────────────┘

Step 2: Fold 16 → 8 bits

┌──────────────────────────────────────────────────────────┐
│ x       = 00000000 00000000 00000000 00000111            │
│ x >> 8  = 00000000 00000000 00000000 00000000            │
│ XOR     = 00000000 00000000 00000000 00000111            │
└──────────────────────────────────────────────────────────┘

Step 3: Fold 8 → 4 bits

┌──────────────────────────────────────────────────────────┐
│ x       = 00000000 00000000 00000000 00000111            │
│ x >> 4  = 00000000 00000000 00000000 00000000            │
│ XOR     = 00000000 00000000 00000000 00000111            │
└──────────────────────────────────────────────────────────┘

Step 4: Fold 4 → 2 bits

┌──────────────────────────────────────────────────────────┐
```

```
x       = 00000000 00000000 00000000 00000111
x >> 2  = 00000000 00000000 00000000 00000001
XOR     = 00000000 00000000 00000000 00000110
```

```
Step 5: Fold 2 → 1 bit
```

```
x       = 00000000 00000000 00000000 00000110
x >> 1  = 00000000 00000000 00000000 00000011
XOR     = 00000000 00000000 00000000 00000101
```

```
Result: x & 1 = 00000101 & 1 = 1 → parity = 1 ✅
```

Complexity Analysis

- **Time Complexity:** `O(1)` - Fixed 5 operations regardless of input
- **Space Complexity:** `O(1)` - In-place computation

Lookup Table (8-bit chunks)

Pseudocode:

```
// Precompute parity for all 8-bit values
lookup ← ARRAY[256] OF BYTE
FOR i FROM 0 TO 255 DO
    lookup[i] = PARITY_8BITS(i)
END FOR

FUNCTION Parity(x)
    // XOR parity of all 4 bytes
    result ← lookup[x & 0xFF]
    result ← result ^ lookup[(x >> 8) & 0xFF]
    result ← result ^ lookup[(x >> 16) & 0xFF]
    result ← result ^ lookup[(x >> 24) & 0xFF]

    RETURN result
END FUNCTION
```

Code Implementation:

```
public class Solution {
    // Lookup table for 8-bit parity
    private static readonly byte[] ParityTable = new byte[256];

    static Solution() {
        // Precompute parity for all possible bytes
        for (int i = 0; i < 256; i++) {
            ParityTable[i] = ComputeParity8((uint)i);
        }
    }

    private static byte ComputeParity8(uint b) {
        byte parity = 0;
        for (int i = 0; i < 8; i++) {
            parity ^= (byte)(b & 1);
            b >>= 1;
        }
        return parity;
    }

    public int Parity(uint x) {
        // XOR parity of all 4 bytes
        byte result = ParityTable[x & 0xFF];
        result ^= ParityTable[(x >> 8) & 0xFF];
        result ^= ParityTable[(x >> 16) & 0xFF];
        result ^= ParityTable[(x >> 24) & 0xFF];

        return result;
    }
}
```

Visualization

```
Input: x = 7 (00000000 00000000 00000000 00000111)

Step 1: Split into 4 bytes
```

```
x = 00000000 00000000 00000000 00000111
     └─B3──┘ └─B2──┘ └─B1──┘ └─B0──┘

Byte 0 (B0): 00000111 = 7
Byte 1 (B1): 00000000 = 0
Byte 2 (B2): 00000000 = 0
Byte 3 (B3): 00000000 = 0
```

```
Step 2: Lookup parity for each byte
```

```
ParityTable[7] = 1 (since 7 has three 1s)
ParityTable[0] = 0
ParityTable[0] = 0
ParityTable[0] = 0
```

```
Step 3: XOR all parities
```

```
result = 1 ^ 0 ^ 0 ^ 0 = 1
```

```
Final: parity = 1 ✅
```

Complexity Analysis

- **Time Complexity:** `O(1)` - Only 4 table lookups and XORs

- **Space Complexity:** `O(256)` = `O(1)` - Fixed-size lookup table

Recursive (Educational)

Pseudocode:

```
FUNCTION Parity(x)
    // Base case: single bit
    IF x = 0 OR x = 1 THEN
        RETURN (int)x
    END IF

    // Split into two halves and XOR their parities
    RETURN Parity(x >> 16) ^ Parity(x & 0xFFFF)
END FUNCTION
```

Code Implementation:

```
public int Parity(uint x) {
    // Base cases
    if (x == 0) return 0;
    if (x == 1) return 1;

    // Split into two halves and recurse
    // XOR the parity of upper and lower halves
    return Parity(x >> 16) ^ Parity(x & 0xFFFF);
}
```

Visualization

```
Input: x = 7 (00000000 00000000 00000000 00000111)

Recursion tree:
```

```
Parity(7)
 ├─ Split: high=0, low=7
 ├─ Parity(0) ^ Parity(7)
 │   ├─ Parity(0) = 0
 │   └─ Parity(7)
 │       ├─ Split: high=0, low=7
 │       ├─ Parity(0) ^ Parity(7)
 │       │   ├─ Parity(0) = 0
 │       │   └─ Parity(7)
 │       │       └─ ... (continues until base)
 │       └─ (builds up results)
 └─ Final: 0 ^ 1 = 1
```

```
Final: parity = 1 ✅
```

Complexity Analysis

- **Time Complexity:** `O(log n)` - Recursive depth is $\log_2(32) = 5$
- **Space Complexity:** `O(log n)` - Call stack depth

Itrative, number, bits

Excel Sheet Column Number

Description: Given a string columnTitle that represents the column title as appears in an Excel sheet, return its corresponding column number.

Examples:

```
Input: columnTitle = "A"
Output: 1
Explanation: A is the 1st column

Input: columnTitle = "Z"
Output: 26
Explanation: Z is the 26th column

Input: columnTitle = "AB"
Output: 28
Explanation: A=1, B=2 → AB = 1*26 + 2 = 28

Input: columnTitle = "ZY"
Output: 701
Explanation: Z=26, Y=25 → ZY = 26*26 + 25 = 701
```

Constraints

- Column title contains uppercase English letters
- Title is not empty
- Result is within integer range
- Excel columns: A (1), B (2), ..., Z (26), AA (27), ...

Itrative Approach

Pseudocode:

```
FUNCTION TitleToNumber(columnTitle)
    result ← 0
    FOR i FROM 0 TO len(columnTitle)-1 DO
        result ← result * 26 + (columnTitle[i] - 'A' + 1)
    END FOR
    RETURN result
END FUNCTION

public static int TitleToNumber(string columnTitle)
{
    int result = 0;
    if (string.IsNullOrEmpty(columnTitle))
```

```
    {
        return result;
    }

    for (int i = 0; i < columnTitle.Length; i++)
    {
        // convert to integer
        result = result * 26 + (columnTitle[i] - 'A' + 1);
    }

    return result;
}
```

Step	Character	Char Value	Calculation	Result
Start				0
1	A (i=0)	1	0×26 + 1	1
2	B (i=1)	2	1×26 + 2	28
3	C (i=2)	3	28×26 + 3	731

Return: 731 ✅

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Factorial of a Number

Description: Calculate the factorial of a positive integer n (n! = n × (n-1) × ... × 1).

Examples:

```
Input: n = 5
Output: 120
Explanation: 5! = 5 × 4 × 3 × 2 × 1 = 120

Input: n = 0
Output: 1
Explanation: 0! = 1 (by definition)

Input: n = 3
Output: 6
Explanation: 3! = 3 × 2 × 1 = 6
```

Constraints

- Input is a positive number greater than zero
- Numbers within integer range (no overflow)
- No negative numbers

Recursion Approach

Pseudocode:

```
FUNCTION Factorial(n)
    IF n == 0 THEN RETURN 1
    RETURN n * Factorial(n-1)
END FUNCTION

public static int FactorialRecursive(int n)
{
        // Base case: 0! = 1
        if (n == 0) return 1;

        // Recursive case: n! = n × (n-1)!
        return n * FactorialRecursive(n - 1);
}

FactorialRecursive(5)
  → 5 * FactorialRecursive(4)
    → 4 * FactorialRecursive(3)
      → 3 * FactorialRecursive(2)
        → 2 * FactorialRecursive(1)
          → 1 * FactorialRecursive(0)
            → Base case: return 1
          ← returns 1 * 1 = 1
        ← returns 2 * 1 = 2
      ← returns 3 * 2 = 6
    ← returns 4 * 6 = 24
  ← returns 5 * 24 = 120
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

Iteration Approach

Pseudocode:

```
FUNCTION FactorialIterative(n):
    // Handle edge cases
    IF n < 0:
        RETURN ERROR  // Factorial not defined for negative numbers

    // Base case: 0! = 1
    IF n == 0:
        RETURN 1

    result = 1

    // Multiply from 1 to n
    FOR i = 1 TO n:
        result = result × i
    END FOR
```

```
    RETURN result
END FUNCTION

public static int FactorialIterative(int n)
{
    if (n < 0)
        throw new ArgumentException("Factorial not defined for negative numbers");

    if (n == 0)
        return 1;

    int result = 1;

    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }

    return result;
}

n = 5, Expected: 5! = 120
```

Step	i	Operation	result
Start			1
1	1	1 × 1 = 1	1
2	2	1 × 2 = 2	2
3	3	2 × 3 = 6	6
4	4	6 × 4 = 24	24
5	5	24 × 5 = 120	120

Return: 120 ✅

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Fibonacci Series

Description: Generate the Fibonacci number at position n (F(n) = F(n-1) + F(n-2), F(0)=0, F(1)=1).

Examples:

```
Input: n = 0
Output: 0
Explanation: F(0) = 0

Input: n = 5
Output: 5
Explanation: Sequence: 0,1,1,2,3,5... F(5) = 5

Input: n = 6
Output: 8
Explanation: Sequence: 0,1,1,2,3,5,8... F(6) = 8
```

Constraints

- Input is a non-negative integer
- Generate Fibonacci sequence starting from F(0)=0, F(1)=1
- Numbers within reasonable range (no overflow for typical use cases)

The number is a positive number and greater than zero.

Brute Force - Recursive

Pseudocode:

```
FUNCTION Fibonacci(n)
    IF n <= 0 THEN RETURN 0
    IF n == 1 THEN RETURN 1
    RETURN Fibonacci(n-1) + Fibonacci(n-2)
END FUNCTION

public static int FibonacciRecursive(int n)
{
    // Base cases
    if (n <= 0) return 0;
    if (n == 1) return 1;

    // Recursive case: F(n) = F(n-1) + F(n-2)
    return FibonacciRecursive(n - 1) + FibonacciRecursive(n - 2);
}

F(0) = 0
F(1) = 1
F(2) = F(1) + F(0) = 1 + 0 = 1
F(3) = F(2) + F(1) = 1 + 1 = 2
F(4) = F(3) + F(2) = 2 + 1 = 3
F(5) = F(4) + F(3) = 3 + 2 = 5

               Fibonacci(5)
               /          \
      Fibonacci(4)      Fibonacci(3)
       /       \          /       \
  Fib(3)     Fib(2)    Fib(2)   Fib(1)
   /   \      /   \     /   \      1
Fib(2) Fib(1) Fib(1) Fib(0) Fib(1) Fib(0)
 /   \    1     1      0      1      0
Fib(1) Fib(0)
   1      0

Calculation:
Fib(2) = Fib(1) + Fib(0) = 1 + 0 = 1
Fib(3) = Fib(2) + Fib(1) = 1 + 1 = 2
```

```
Fib(4) = Fib(3) + Fib(2) = 2 + 1 = 3
Fib(5) = Fib(4) + Fib(3) = 3 + 2 = 5
```

- **Time Complexity :** `O(2ⁿ)`
- **Space Complexity :** `O(n)`

Memoization Approach

We can optimize by storing already computed results using **memoization**.

Pseudocode:

```
FUNCTION FibonacciMemoization(n, memo = null):
    // Initialize memo dictionary if not provided
    IF memo == null:
        memo = NEW Dictionary<int, int>()

    // Base cases
    IF n <= 0:
        RETURN 0
    IF n == 1:
        RETURN 1

    // Check if result already computed
    IF memo CONTAINS KEY n:
        RETURN memo[n]

    // Recursive calculation
    fib_n_minus_1 = FibonacciMemoization(n - 1, memo)
    fib_n_minus_2 = FibonacciMemoization(n - 2, memo)
    result = fib_n_minus_1 + fib_n_minus_2

    // Store result in memo for future use
    memo[n] = result

    RETURN result
END FUNCTION

public static int FibonacciMemoization(int n, Dictionary<int, int> memo = null)
{
    if (memo == null) memo = new Dictionary<int, int>();
    // Base cases
    if (n <= 0) return 0;
    if (n == 1) return 1;

    if (memo.ContainsKey(n))
    {
        return memo[n];
    }

    // Recursive case
    int result = FibonacciMemoization(n - 1, memo) + FibonacciMemoization(n - 2, memo);

    // Store the result in the memo dictionary
    memo[n] = result;
    return result;
}
```

```
                        Fib(5) → computes Fib(4) + Fib(3)
                        /                     \
                       /                       \
              needs Fib(4)                Fib(3) FROM MEMO!
              /           \                   (already computed)
             /             \
      needs Fib(3)     Fib(2) FROM MEMO!
     /           \       (already computed)
    /             \
needs Fib(2)   Fib(1) FROM MEMO!
   /     \      (already computed)
  /       \
Fib(1)  Fib(0) FROM MEMO!
(base)   (base)
```

- **Time Complexity :** `O(n)`
  - Each Fibonacci number computed once
- **Space Complexity :** `O(n)`
  - For memo dictionary + call stack

Dynamic Programming - Tabulation

The brute force recursive approach has exponential time complexity due to redundant calculations. We can optimize using **dynamic programming (tabulation)** - build up the solution from bottom-up.

The brute force recursive approach has exponential time complexity due to redundant calculations. We can optimize using **dynamic programming (tabulation)** - build up the solution from bottom-up.

Pseudocode:

```
FUNCTION FibonacciTabulation(n):
    // Handle base cases
    IF n <= 0:
        RETURN 0
    IF n == 1:
        RETURN 1

    // Create dynamic programming table
    dp = ARRAY[0..n] of integers

    // Initialize base cases in table
    dp[0] = 0
    dp[1] = 1

    // Fill table iteratively from smallest to largest
    FOR i = 2 TO n DO:
        dp[i] = dp[i-1] + dp[i-2]
    END FOR
```

```
    // Return the nth Fibonacci number
    RETURN dp[n]
END FUNCTION

public static int FibonacciTabulation(int n)
{
    // Handle base cases
    if (n <= 0) return 0;
    if (n == 1) return 1;

    // Create a table to store Fibonacci values
    int[] dp = new int[n + 1];

    // Initialize base cases
    dp[0] = 0;
    dp[1] = 1;

    // Fill the table bottom-up
    for (int i = 2; i <= n; i++)
    {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}

Computing Fibonacci(5) with Tabulation:

Step 1: Initialize array
  dp = [0, 0, 0, 0, 0, 0]
       [0, 1, ?, ?, ?, ?] (after setting base cases)

Step 2: Build bottom-up
  i=2: dp[2] = dp[1] + dp[0] = 1 + 0 = 1
       dp = [0, 1, 1, ?, ?, ?]

  i=3: dp[3] = dp[2] + dp[1] = 1 + 1 = 2
       dp = [0, 1, 1, 2, ?, ?]

  i=4: dp[4] = dp[3] + dp[2] = 2 + 1 = 3
       dp = [0, 1, 1, 2, 3, ?]

  i=5: dp[5] = dp[4] + dp[3] = 3 + 2 = 5
       dp = [0, 1, 1, 2, 3, 5]

Step 3: Return dp[5] = 5 ✓

Computation trace:
  dp[0]=0  (base case)
  dp[1]=1  (base case)
  dp[2]=1  (computed once)
  dp[3]=2  (computed once)
  dp[4]=3  (computed once)
  dp[5]=5  (computed once)

No redundant calculations!
```

- **Time Complexity :** `O(n)`
  - Each Fibonacci number computed once
- **Space Complexity :** `O(n)`
  - For integer array

Space-Optimized Fibonacci (Tabulation)

We can further optimize space by only keeping track of the last two values:

Pseudocode:

```
FUNCTION FibonacciSpaceOptimized(n):
    IF n < 2:
        RETURN n

    a ← 0   // F(0)
    b ← 1   // F(1)

    FOR i = 2 TO n:
        c ← a + b
        a ← b
        b ← c
    END FOR

    RETURN b
END FUNCTION

public static int FibonacciSpaceOptimized(int n)
{
    // Handle base cases
    if (n <= 0) return 0;
    if (n == 1) return 1;

    int a = 0;   // F(0)
    int b = 1;   // F(1)

    // Compute from bottom-up
    for (int i = 2; i <= n; i++)
    {
        int c = a + b;

        // Shift values for next iteration
        a = b;
        b = c;
    }

    return b;
}

Computing Fibonacci(5) with Space Optimization:

Initial state:
```

```
  prev2=0 (F(0)), prev1=1 (F(1))

i=2: current = 1+0 = 1, prev2=1, prev1=1 (F(2)=1)
i=3: current = 1+1 = 2, prev2=1, prev1=2 (F(3)=2)
i=4: current = 2+1 = 3, prev2=2, prev1=3 (F(4)=3)
i=5: current = 3+2 = 5, prev2=3, prev1=5 (F(5)=5)

Return: 5 ✓

No array needed, only two variables!
```

- **Time Complexity :** `O(n)`
  - Each Fibonacci number computed once
- **Space Complexity :** `O(1)`

String to Integer

Description: Convert a numeric string to its integer representation.

Examples:

```
Input: s = "123"
Output: 123
Explanation: String "123" converts to integer 123

Input: s = "5678"
Output: 5678
Explanation: String "5678" converts to integer 5678

Input: s = "0"
Output: 0
Explanation: String "0" converts to integer 0
```

Constraints

- Input string is not null or empty
- String contains only digit characters
- Result fits within integer range
- No need to handle sign (positive only)

Substring Approch

Pseudocode:

```
FUNCTION StringToInt(s)
  IF s IS EMPTY THEN
    RETURN 0
  END IF
  last ← LAST_CHAR(s)
  rest ← ALL_BUT_LAST(s)
  RETURN StringToInt(rest) * 10 + (last - '0')
END FUNCTION

public int StringToInt(string s)
{
    // Base case
    if (s.Length == 0)
    {
        return 0;
    }

    // Inductive Hypothesis
    char lastChar = s[s.Length - 1];
    string restOfString = s.Substring(0, s.Length - 1);
    int smallResult = StringToInt(restOfString);

    // Inductive Step
    int lastDigit = lastChar - '0';
    return smallResult * 10 + lastDigit;
}


Call Stack (growing down):
StringToInt("123") waiting, lastDigit=3
  StringToInt("12") waiting, lastDigit=2
    StringToInt("1") waiting, lastDigit=1
      StringToInt("") → returns 0
    returns 0*10 + 1 = 1
  returns 1*10 + 2 = 12
returns 12*10 + 3 = 123
```

- **Time Complexity :** `O(n²)`
  - Recursive calls: `n+1` calls for string of length `n`
  - Each call: `Substring(0, s.Length-1)` is `O(n)` operation
- **Space Complexity :** `O(d²)`
  - Call stack depth: `d` (when going down one branch)
  - String operations create new strings: `O(d²)` total space

Substring Approch with Memoization

Pseudocode:

```
FUNCTION StringToInt(s, index)
  IF index < 0 THEN
    RETURN 0
  END IF
  digit ← s[index] - '0'
  remainingValue ← StringToInt(s, index - 1)
  RETURN remainingValue * 10 + digit
END FUNCTION

public static int StringToInt(string s, int index)
{
    if (index < 0)
```

```
        return 0;

    int digit = s[index] - '0';

    int remainingValue = StringToInt(s, index - 1);
    return remainingValue * 10 + digit;
}

Call Stack (growing down):
StringToInt("123") waiting, lastDigit=3
  StringToInt("12") waiting, lastDigit=2
    StringToInt("1") waiting, lastDigit=1
      StringToInt("") → returns 0
    returns 0*10 + 1 = 1
  returns 1*10 + 2 = 12
returns 12*10 + 3 = 123
```

- **Time Complexity :** `O(n)`
  - Each number computed once
- **Space Complexity :** `O(n)`
  - For call stack

Basic Conversion Approach

Pseudocode:

```
FUNCTION StringToInt(s)
    IF s IS EMPTY THEN RETURN 0
    result ← 0
    FOR each char c IN s DO
        digit ← c - '0'
        result ← result * 10 + digit
    END FOR
    RETURN result
END FUNCTION

public int StringToInt(string s)
{
    int result = 0;
    for (int i = 0; i < s.Length; i++)
    {
        result = result * 10 + (s[i] - '0');
    }
    return result;
}

"d₀d₁d₂...dₙ₋₁" = (((...((d₀×10 + d₁)×10 + d₂)×10 + ...)×10 + dₙ₋₂)×10 + dₙ₋₁

input = "123"

0×10 + 1 = 1
1×10 + 2 = 12
12×10 + 3 = 123
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Loop Unrolling Optimization

Pseudocode:

```
FUNCTION StringToIntUnrolled(s):
    IF s is null or empty:
        RETURN 0

    result = 0
    length = s.Length
    i = 0

    // Process 4 digits at a time (loop unrolling)
    WHILE i + 3 < length:
        batch = (s[i] - '0') × 1000 +
                (s[i+1] - '0') × 100 +
                (s[i+2] - '0') × 10 +
                (s[i+3] - '0')

        result = result × 10000 + batch
        i = i + 4
    END WHILE

    // Process remaining digits (1-3 digits)
    WHILE i < length:
        result = result × 10 + (s[i] - '0')
        i = i + 1
    END WHILE

    RETURN result
END FUNCTION
```

Logic:

- Group processing: Handle 4 digits together as a batch
- Reduce multiplications: Instead of 4 multiplications, process batch with one
- Minimize loop overhead: Fewer iterations = less loop control logic
- Handle leftovers: Process remaining 1-3 digits normally

Code Implementation:

```
public static int StringToIntUnrolled(string s)
{
    if (string.IsNullOrEmpty(s)) return 0;

    int result = 0;
    int i = 0;
    int length = s.Length;

    // Process 4 digits at a time (loop unrolling)
```

```
    while (i + 3 < length)
    {
        result = result * 10000
               + (s[i] - '0') * 1000
               + (s[i+1] - '0') * 100
               + (s[i+2] - '0') * 10
               + (s[i+3] - '0');
        i += 4;
    }

    // Process remaining digits
    while (i < length)
    {
        result = result * 10 + (s[i] - '0');
        i++;
    }

    return result;
}
```

```
Input: s = "12345678" (length = 8)

Step 1: First 4-digit batch (i=0, i+3=3)
┌──────────────────────────────────────────────────────────┐
 batch = ('1'-'0')*1000 + ('2'-'0')*100 +
          ('3'-'0')*10 + ('4'-'0')
        = 1*1000 + 2*100 + 3*10 + 4
        = 1000 + 200 + 30 + 4 = 1234
 result = 0*10000 + 1234 = 1234
 i = 0 + 4 = 4
└──────────────────────────────────────────────────────────┘

Step 2: Second 4-digit batch (i=4, i+3=7)
┌──────────────────────────────────────────────────────────┐
 batch = ('5'-'0')*1000 + ('6'-'0')*100 +
          ('7'-'0')*10 + ('8'-'0')
        = 5*1000 + 6*100 + 7*10 + 8
        = 5000 + 600 + 70 + 8 = 5678
 result = 1234*10000 + 5678
        = 12,340,000 + 5,678 = 12,345,678
 i = 4 + 4 = 8 (end of string)
└──────────────────────────────────────────────────────────┘

Final result: 12,345,678 ✅
```

- **Time Complexity :** `O(n)`
  - Still linear with input size but with smaller constant factor than basic approach
- **Space Complexity :** `O(1)`

Convert Roman to Integer

Description: Convert a Roman numeral string to its integer equivalent.

Examples:

```
Input: s = "III"
Output: 3
Explanation: I + I + I = 1 + 1 + 1 = 3

Input: s = "IV"
Output: 4
Explanation: V - I = 5 - 1 = 4 (subtractive case)

Input: s = "MCMXCIV"
Output: 1994
Explanation: M + (M-C) + (C-X) + (V-I) = 1000 + 900 + 90 + 4 = 1994
```

Constraints

- Input string contains valid Roman numerals
- Roman string is not null or empty
- Range: I(1) to MMMCMXCIX(3999)
- Subtractive notation: IV(4), IX(9), XL(40), XC(90), CD(400), CM(900)

Given a Roman numeral, convert it to an integer. Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

```
Symbol  Value
I       1
V       5
X       10
L       50
C       100
M       1000
```

Brute Force Approach

Pseudocode:

```
FUNCTION RomanToInt(s)
    map ← {I:1,V:5,X:10,L:50,C:100,D:500,M:1000}
    total ← 0
    FOR i FROM 0 TO len(s)-1 DO
        IF i+1 < len(s) AND map[s[i]] < map[s[i+1]] THEN
            total ← total - map[s[i]]
        ELSE
            total ← total + map[s[i]]
        END IF
    END FOR
    RETURN total
END FUNCTION
```

Rules:

- I can be placed before V (5) and X (10) to make 4 and 9
- X can be placed before L (50) and C (100) to make 40 and 90

- C can be placed before D (500) and M (1000) to make 400 and 900

Code Implementation:

```
public static int RomanToIntBruteForce(string s)
{
    // Brute force: use if-else chains to handle subtraction cases
    Dictionary<char, int> romanValues = new()
    {
        { 'I', 1 },
        { 'V', 5 },
        { 'X', 10 },
        { 'L', 50 },
        { 'C', 100 },
        { 'D', 500 },
        { 'M', 1000 }
    };

    int result = 0;
    for (int i = 0; i < s.Length; i++)
    {
        // If current value is less than next value, subtract (cases like IV, IX, XL, etc.)
        if (i + 1 < s.Length && romanValues[s[i]] < romanValues[s[i + 1]])
        {
            result -= romanValues[s[i]];
        }
        else
        {
            result += romanValues[s[i]];
        }
    }

    return result;
}
```

```
Example: "MCMXCIV" = 1994

M    → 1000 (next is C, 100 > 1000? NO) → sum = 1000
C    → 100  (next is M, 1000 > 100? YES, subtract) → sum = 1000 - 100 = 900
M    → 1000 (next is X, 10 > 1000? NO) → sum = 1900
X    → 10   (next is C, 100 > 10? YES, subtract) → sum = 1900 - 10 = 1890
C    → 100  (next is I, 1 > 100? NO) → sum = 1990
I    → 1    (next is V, 5 > 1? YES, subtract) → sum = 1990 - 1 = 1989
V    → 5    (no next) → sum = 1989 + 5 = 1994 ✓
```

- **Time Complexity:** `O(n)` - single pass through string
- **Space Complexity:** `O(1)` - dictionary of fixed size 7

Integer to Roman

Description: Convert an integer to its Roman numeral representation (1-3999).

Examples:

```
Input: num = 3
Output: "III"
Explanation: I + I + I = 1 + 1 + 1 = 3

Input: num = 58
Output: "LVIII"
Explanation: L=50, V=5, III=3 → 50+5+3=58

Input: num = 1994
Output: "MCMXCIV"
Explanation: M=1000, CM=900, XC=90, IV=4 → 1000+900+90+4=1994

Input: num = 3749
Output: "MMMDCCXLIX"
Explanation: MMM=3000, DCC=700, XL=40, IX=9 → 3749
```

Constraints

- Input is integer in range [1, 3999]
- Output is valid Roman numeral string
- Must use subtractive notation for 4,9,40,90,400,900

Convert an integer to its Roman numeral representation.

```
Example:
- 3 → "III"
- 58 → "LVIII"
- 1994 → "MCMXCIV"
```

Brute Force Approach

Pseudocode:

```
FUNCTION IntToRoman(num)
    values = [1000,900,500,400,100,90,50,40,10,9,5,4,1]
    symbols = ["M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"]
    res ← ""
    FOR i FROM 0 TO len(values)-1 DO
        WHILE num >= values[i] DO
            res ← res + symbols[i]
            num ← num - values[i]
        END WHILE
    END FOR
    RETURN res
END FUNCTION
```

Code Implementation

```
public static string IntToRoman(int num)
{
    // Optimized: pre-computed array of values in descending order
```

```
    // Includes all subtraction cases (4, 9, 40, 90, 400, 900)
    int[] values = { 1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1 };
    string[] symbols = { "M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I" };

    string result = "";
    for (int i = 0; i < values.Length; i++)
    {
        while (num >= values[i])
        {
            result += symbols[i];
            num -= values[i];
        }
    }
    return result;
}
```

```
Example: num = 1994

Step-by-step process:
┌──────────────────────────────────────────────────────────────────────────┐
│ values:  [1000, 900, 500, 400, 100,  90,  50,  40,  10,  9,  5,  4,  1] │
│ symbols: ["M", "CM","D","CD", "C","XC","L","XL","X","IX","V","IV","I"] │
└──────────────────────────────────────────────────────────────────────────┘

Initial: num=1994, result=""

┌──────────┬──────────────────────────────┬──────────────┬──────────────┐
│ Value    │ Condition                    │ Operation    │ Result       │
├──────────┼──────────────────────────────┼──────────────┼──────────────┤
│ 1000     │ 1994 ≥ 1000 ✓                │ result="M"   │ num=994      │
│ 1000     │ 994 ≥ 1000 ✗                 │ Skip         │              │
│ 900      │ 994 ≥ 900 ✓                  │ result="MCM" │ num=94       │
│ 900      │ 94 ≥ 900 ✗                   │ Skip         │              │
│ 500      │ 94 ≥ 500 ✗                   │ Skip         │              │
│ 400      │ 94 ≥ 400 ✗                   │ Skip         │              │
│ 100      │ 94 ≥ 100 ✗                   │ Skip         │              │
│ 90       │ 94 ≥ 90 ✓                    │ result="MCMXC"│ num=4       │
│ 90       │ 4 ≥ 90 ✗                     │ Skip         │              │
│ 50       │ 4 ≥ 50 ✗                     │ Skip         │              │
│ 40       │ 4 ≥ 40 ✗                     │ Skip         │              │
│ 10       │ 4 ≥ 10 ✗                     │ Skip         │              │
│ 9        │ 4 ≥ 9 ✗                      │ Skip         │              │
│ 5        │ 4 ≥ 5 ✗                      │ Skip         │              │
│ 4        │ 4 ≥ 4 ✓                      │ result="MCMXCIV"│ num=0      │
│ 1        │ 0 ≥ 1 ✗                      │ Skip         │              │
└──────────┴──────────────────────────────┴──────────────┴──────────────┘

Final result: "MCMXCIV" ✓
```

- **Time Complexity:** `O(1)` - bounded by max iterations (num can be at most 3999)
- **Space Complexity:** `O(1)` - fixed arrays

Number to Text

Description: Convert a number to its English word representation using recursion.

Examples:

```
Input: n = 0
Output: "Zero"
Explanation: Special case for zero

Input: n = 123
Output: "One Hundred Twenty Three"
Explanation: Full English representation

Input: n = 1000
Output: "One Thousand"
Explanation: Thousands place
```

Constraints

- Input is a positive integer
- Convert digits to their English word equivalents
- Recursive implementation
- No negative numbers

Recursion Approach

Pseudocode:

```
FUNCTION ConvertNumberToText(number)
  IF number == 0 THEN RETURN "Zero" END IF
  IF number < 20 THEN RETURN unitsMap[number] END IF
  IF number < 100 THEN RETURN tensMap[number / 10] + (IF number % 10 > 0 THEN " " + ConvertNumberToText(number % 10) ELSE "") END IF
  IF number < 1000 THEN RETURN unitsMap[number / 100] + " Hundred" + (IF number % 100 > 0 THEN " " + ConvertNumberToText(number % 100) ELSE "") END IF
  IF number < 100000 THEN RETURN ConvertNumberToText(number / 1000) + " Thousand" + (IF number % 1000 > 0 THEN " " + ConvertNumberToText(number % 1000) ELSE "") END I
  RETURN ConvertNumberToText(number / 100000) + " Lac" + (IF number % 100000 > 0 THEN " " + ConvertNumberToText(number % 100000) ELSE "")
END FUNCTION
```

```
public class NumberToText
{
    private static string[] unitsMap = { "Zero", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine", "Ten", "Eleven", "Twelve", "Thirteen", "Fourt
    private static string[] tensMap = { "Zero", "Ten", "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety" };

    public static string ConvertNumberToText(int number)
    {
        if (number == 0)
            return unitsMap[0];

        if (number < 20)
            return unitsMap[number];

        if (number < 100)
            return tensMap[number / 10] + ((number % 10 > 0) ? " " + ConvertNumberToText(number % 10) : "");
```

```
        if (number < 1000)
            return unitsMap[number / 100] + " Hundred" + ((number % 100 > 0) ? " " + ConvertNumberToText(number % 100) : "");

        if (number < 100000)
            return ConvertNumberToText(number / 1000) + " Thousand" + ((number % 1000 > 0) ? " " + ConvertNumberToText(number % 1000) : "");

        return ConvertNumberToText(number / 100000) + " Lac" + ((number % 100000 > 0) ? " " + ConvertNumberToText(number % 100000) : "");
    }
}

ConvertNumberToText(123456)
│
├─ ConvertNumberToText(1) → "One"
│
└─ ConvertNumberToText(23456)
   │
   ├─ ConvertNumberToText(23)
   │  │
   │  ├─ tensMap[2] → "Twenty"
   │  │
   │  └─ ConvertNumberToText(3) → "Three"
   │     Result: "Twenty Three"
   │
   └─ ConvertNumberToText(456)
      │
      ├─ ConvertNumberToText(4) → "Four"
      │
      └─ ConvertNumberToText(56)
         │
         ├─ tensMap[5] → "Fifty"
         │
         └─ ConvertNumberToText(6) → "Six"
            Result: "Fifty Six"
         Result: "Four Hundred Fifty Six"
      Result: "Twenty Three Thousand Four Hundred Fifty Six"
   Result: "One Lac Twenty Three Thousand Four Hundred Fifty Six"
```

- **Time Complexity :** `O(log n)`
  - Each recursive call reduces the number by a factor (÷100000, ÷1000, ÷100, ÷10)
  - Number of digits = d = $\lfloor \log_{10}(n) \rfloor + 1$
  - Maximum recursion depth ≈ number of digit groups = O(log n)
- **Space Complexity :** `O(log n)`
  - Call stack depth = O(log n) (number of digit groups)
  - String concatenation creates new strings, but total length = O(log n)
  - Arrays unitsMap and tensMap are constant size O(1)

Single Number (All appear twice except one)

Description: Find the single number that appears once while all other numbers appear exactly twice in the array.

Examples:

```
Input: nums = [2,2,1]
Output: 1
Explanation: 2 appears twice, 1 appears once

Input: nums = [4,1,2,1,2]
Output: 4
Explanation: 4 appears once, 1 and 2 appear twice

Input: nums = [5]
Output: 5
Explanation: Only one element
```

Constraints

- Array contains positive integers
- All numbers appear exactly twice except one
- One number appears exactly once
- Cannot use extra space (O(1) space required)

Bit Manipulation Approach

Pseudocode:

```
FUNCTION SingleNumber(nums):
    // Initialize result to 0
    result = 0

    // XOR all numbers in the array
    FOR EACH num IN nums:
        result = result XOR num
    END FOR

    RETURN result
END FUNCTION

public static int SingleNumber(int[] nums)
{
    int result = 0;
    for (int num : nums)
    {
        result ^= num;
    }
    return result;
}

[4, 1, 2, 1, 2]
 │  │  │  │  │
 │──┴──┴──┴──┴──
 │  │  │  │  └── Pair cancels (1⊕1=0)
 │  │  └──────── Pair cancels (2⊕2=0)
 │  └─────────── Single remains: 4
 └──────────────
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Find Missing Number in Array

Description: Find the single missing number in an array containing n distinct numbers from range [0, n].

Examples:

```
Input: nums = [3,0,1]
Output: 2
Explanation: Array has 0,1,3 but missing 2 from range [0-3]

Input: nums = [0,1]
Output: 2
Explanation: Array has 0,1 but missing 2 from range [0-2]

Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: Array missing 8 from range [0-9]
```

Constraints

- Array contains n elements
- Numbers range from 0 to n (inclusive)
- Exactly one number is missing
- Array is unsorted
- O(1) space solution preferred

Array of size `n` containing numbers from `0` to `n`, one missing.

Bit Manipulation Approach

Pseudocode:

```
FUNCTION MissingNumber(nums):
    // Get length of array (n numbers from 0 to n, one missing)
    n = LENGTH(nums)

    // Initialize XOR result to 0
    xor = 0

    // XOR all elements in the array
    FOR i = 0 TO n-1:
        xor = xor XOR nums[i]
    END FOR

    // XOR all numbers from 0 to n (complete range)
    FOR i = 0 TO n:
        xor = xor XOR i
    END FOR

    // Remaining number is the missing one
    RETURN xor
END FUNCTION
```

```
public static int MissingNumber(int[] nums)
{
    int n = nums.Length;
    int xor = 0;
     // XOR all array elements
    for (int i = 0; i < nums.Length; i++)
    {
        xor ^= nums[i];
    }

    // XOR all numbers from 0 to n
    for (int i = 0; i <= nums.Length; i++)
    {
        xor ^= i;
    }

    return xor;  // Missing number
}
```

```
input: [3, 0, 1]
Array XOR:   3 ⊕ 0 ⊕ 1 = ?
Range XOR:   0 ⊕ 1 ⊕ 2 ⊕ 3 = ?

Combine: (3 ⊕ 0 ⊕ 1) ⊕ (0 ⊕ 1 ⊕ 2 ⊕ 3)

Cancel pairs:
   3's cancel: (3 from array) ⊕ (3 from range) = 0
   0's cancel: (0 from array) ⊕ (0 from range) = 0
   1's cancel: (1 from array) ⊕ (1 from range) = 0

What remains? 2 (only in range, not in array)
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Minimum Bit Flips to Convert Number

Description: Find minimum number of bit flips needed to convert integer a to integer b.

Examples:

```
Input: start = 8, goal = 3
Output: 2
Explanation: 8 = 1000, 3 = 0011 → flip 2 bits: 1000→0011

Input: start = 1, goal = 4
Output: 2
```

```
Explanation: 1 = 0001, 4 = 0100 → flip 2 bits

Input: start = 5, goal = 5
Output: 0
Explanation: Numbers are same, no flips needed
```

Constraints

- Input is a positive integer greater than zero
- Find minimum bit flips needed to convert a to b
- XOR operation gives differing bits
- Count number of 1s in XOR result

The number is a positive number and greater than zero.

Bit Manipulation Approach

Pseudocode:

```
FUNCTION MinBitFlips(start, goal)
    // Count differing bits
    xor ← start XOR goal
    count ← 0
    WHILE xor > 0 DO
        count ← count + (xor & 1)
        xor ← xor >> 1
    END WHILE
    RETURN count
END FUNCTION

public static int MinBitFlips(int start, int goal)
{
    // Count differing bits
    int xor = start ^ goal;

    int count = 0;
    while (xor > 0)
    {
        count += xor & 1;
        xor >>= 1;
    }

    return count;
}

start: 3 = 0011
goal:  4 = 0100
xor:   3 ^ 4 = 0011 ^ 0100 = 0111 (7)

Count 1s in 0111:
  0 1 1 1
  │ │ │ │
  │ │ │ └─ 1 (count=1)
  │ │ └── 1 (count=2)
  │ └──── 1 (count=3)
  └────── 0 (done)

Result: 3 bit flips needed
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

Reverse String

Description: Reverse a string in-place using character swapping from both ends.

Examples:

```
Input: s = "hello"
Output: "olleh"
Explanation: Reversed string

Input: s = "racecar"
Output: "racecar"
Explanation: Palindrome, same when reversed

Input: s = "a"
Output: "a"
Explanation: Single character
```

Constraints

- Input string is not null
- Reverse in-place or return new reversed string
- Only ASCII characters

The string is not null.

Two-Pointer Approach

Pseudocode:

```
FUNCTION ReverseString(s):
    // Convert string to character array
    chars = s.TO_CHAR_ARRAY()

    // Initialize pointers
    left = 0
    right = LENGTH(chars) - 1

    // Swap characters until pointers meet
    WHILE left < right:
        // Swap chars[left] and chars[right]
```

```
        temp = chars[left]
        chars[left] = chars[right]
        chars[right] = temp

        // Move pointers inward
        left = left + 1
        right = right - 1
    END WHILE

    // Convert character array back to string
    RETURN NEW_STRING(chars)
END FUNCTION
```

```
public static string ReverseString(string s)
{
    char[] chars = s.ToCharArray();
    int left = 0;
    int right = chars.Length - 1;
    while (left < right)
    {
        char temp = chars[left];
        chars[left] = chars[right];
        chars[right] = temp;
        left++;
        right--;
    }
    return new string(chars);
}
```

```
Initial:  h  e  l  l  o
           ↑           ↑
          left       right

Swap 1:  o  e  l  l  h
             ↑     ↑
            left  right

Swap 2:  o  l  l  e  h
                ↑
              left/right

Final:   o  l  l  e  h
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

Using String Builder

Pseudocode:

```
FUNCTION ReverseString(s):
    // Initialize empty builder
    result = NEW StringBuilder()

    // Append characters in reverse order
    FOR i FROM length(s)-1 TO 0 STEP -1:
        result.ADD_CHAR(s[i])

    RETURN result.BUILD_STRING()
END FUNCTION
```

```
public static string ReverseString(string s)
{
    var sb = new StringBuilder(s.Length);

    // Build from end to start
    for (int i = s.Length - 1; i >= 0; i--)
    {
        sb.Append(s[i]);
    }

    return sb.ToString();
}
```

```
String:  H   E   L   L   O
Indices: 0   1   2   3   4

Processing order: 4 → 3 → 2 → 1 → 0

sb building process:
Start:     ""
Step 1:    "O"        (append s[4])
Step 2:    "OL"       (append s[3])
Step 3:    "OLL"      (append s[2])
Step 4:    "OLLE"     (append s[1])
Step 5:    "OLLEH"    (append s[0])

Final: "OLLEH" ✅
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

Contains Duplicate

Description: Determine if an array contains any duplicate elements.

Examples:

```
Input: nums = [1,2,3,1]
Output: true
Explanation: '1' appears twice

Input: nums = [1,2,3,4]
Output: false
Explanation: All elements are unique

Input: nums = [99,99]
```

```
Output: true
Explanation: Duplicate element found
```

Constraints

- Array is not null
- Find if any duplicate element exists
- No specific range constraint

The array is not null. Please complete with `O(1)` space complexity.

Brute Force - Nested Loop

Pseudocode:

```
FUNCTION ContainsDuplicate(nums):
    n = LENGTH(nums)

    // Compare each element with every other element
    FOR i = 0 TO n-1:
        FOR j = i+1 TO n-1:
            IF nums[i] == nums[j]:
                RETURN true  // Duplicate found
        END FOR
    END FOR

    RETURN false  // No duplicates found
END FUNCTION

public static bool ContainsDuplicate(int[] nums)
{
    for (int i = 0; i < nums.Length; i++)
    {
        for (int j = i + 1; j < nums.Length; j++)
        {
            if (nums[i] == nums[j])
            {
                return true;
            }
        }
    }
    return false;
}

nums = [1, 2, 3, 1]
Compare each pair (i, j) where j > i:

    j=0 j=1 j=2 j=3
i=0  -   1=2 1=3 1=1 ✓
i=1       -   2=3 2=1
i=2            -   3=1
i=3                -

Comparisons made:
(0,1): 1 vs 2 ✗
(0,2): 1 vs 3 ✗
(0,3): 1 vs 1 ✓ → Found duplicate!
```

- **Time Complexity :** `O(n²)`
- **Space Complexity :** `O(1)`

Hash Set Approch

The brute force approach requires comparing each pair, resulting in quadratic time complexity. We can optimize this to linear time by using a **hash set** to track seen elements.

Pseudocode:

```
FUNCTION ContainsDuplicateWithHashSet(nums):
    seen = NEW HashSet()

    FOR num IN nums:
        IF num IN seen:
            RETURN true  // Duplicate found
        seen.ADD(num)
    END FOR

    RETURN false  // No duplicates found
END FUNCTION

public static bool ContainsDuplicateWithHashSet(int[] nums)
{
    HashSet<int> seen = new HashSet<int>();

    foreach (int num in nums)
    {
        // If the element is already in the set, we found a duplicate
        if (seen.Contains(num))
        {
            return true;
        }

        // Add the current element to the set
        seen.Add(num);
    }

    return false;
}

nums = [1, 2, 3, 1]

Step 1: Process nums[0] = 1
  seen = {1}

Step 2: Process nums[1] = 2
  2 in seen? NO → Add 2
  seen = {1, 2}
```

```
Step 3: Process nums[2] = 3
  3 in seen? NO → Add 3
  seen = {1, 2, 3}

Step 4: Process nums[3] = 1
  1 in seen? YES ✓ → Found duplicate!
  Return true

Early termination at index 3, no further comparisons needed!
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

Valid Palindrome

Description: Check if a string is a palindrome using two-pointer approach, handling case insensitivity.

Examples:

```
Input: s = "racecar"
Output: true
Explanation: Reads same forwards and backwards

Input: s = "hello"
Output: false
Explanation: "hello" != "olleh"

Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: True palindrome (ignoring spaces/punctuation)
```

Constraints

- Input string is not null
- Compare characters case-insensitively
- Only check alphanumeric characters

Two Pointer Approch

Pseudocode:

```
FUNCTION IsPalindrome(s):
    // Initialize pointers
    left = 0
    right = LENGTH(s) - 1

    // Compare characters moving inward
    WHILE left < right:
        IF s[left] ≠ s[right]:
            RETURN false  // Not a palindrome

        // Move pointers inward
        left = left + 1
        right = right - 1
    END WHILE

    RETURN true  // All characters matched
END FUNCTION

public static bool IsPalindrome(string s)
{
    int left = 0;
    int right = s.Length - 1;

    while (left < right)
    {
        if(s[left] != s[right])
        {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

String:  r a c e c a r
         ↑           ↑
        left       right  (Compare: r==r ✓)

         r a c e c a r
           ↑       ↑
          left   right   (Compare: a==a ✓)

         r a c e c a r
             ↑   ↑
            left right  (Compare: c==c ✓)

         r a c e c a r
               ↑
            left/right  (Done!)
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Check for Anagrams

Description: Determine if two strings are anagrams (contain same characters with same frequencies).

Examples:

```
Input: s = "listen", t = "silent"
Output: true
Explanation: Both strings contain: l,i,s,t,e,n
```

```
Input: s = "hello", t = "world"
Output: false
Explanation: Different characters: hello has 2 l's, world has 0 l's

Input: s = "abc", t = "cba"
Output: true
Explanation: Same characters, different order
```

Constraints

- Both strings are not null
- Strings contain only lowercase English letters
- Two strings are anagrams if they have same characters
- Order doesn't matter, frequency matters

An anagram of a string is another string that contains the same characters, only the order of characters can be different.

Brute Force

Pseudocode:

```
FUNCTION IsAnagram(s, t):
    IF s.LENGTH != t.LENGTH:
        RETURN false

    // Use dictionary for Unicode characters
    frequency = NEW DICTIONARY<char, int>

    // Count characters in s
    FOR EACH ch IN s:
        IF frequency CONTAINS ch:
            frequency[ch] = frequency[ch] + 1
        ELSE:
            frequency[ch] = 1

    // Verify characters in t
    FOR EACH ch IN t:
        IF NOT frequency CONTAINS ch OR frequency[ch] == 0:
            RETURN false

        frequency[ch] = frequency[ch] - 1

    RETURN true
END FUNCTION
```

```
public static bool IsAnagram(string s, string t)
{
    if (s.Length != t.Length)
    {
        return false;
    }

    int[] charCount = new int[256];
    foreach (char c in s)
    {
        charCount[c]++;
    }

    foreach (char c in t)
    {
        if(charCount[c] == 0)
        {
            return false;
        }

        charCount[c]--;
    }

    return true;
}
```

```
String s: a n a g r a m
Count:    a:3, n:1, g:1, r:1, m:1

String t: n a g a r a m
Check:    n✓ a✓ g✓ a✓ r✓ a✓ m✓ → All good!
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Duplicate Characters

Description: Find and return all characters that appear more than once in a string, preserving order.

Examples:

```
Input: s = "abcda"
Output: "a"
Explanation: 'a' appears twice, others once

Input: s = "abab"
Output: "ab"
Explanation: Both 'a' and 'b' appear twice

Input: s = "abcdef"
Output: ""
Explanation: No duplicates
```

Constraints

- Input string is not null
- Return characters that appear more than once
- Return in original order from string

The string is not null.

Brute Force

Pseudocode:

```
FUNCTION FindDuplicates(s):
    seen = NEW SET()
    duplicates = NEW SET()

    FOR EACH char c IN s:
        IF seen CONTAINS c:
            duplicates.ADD(c)
        ELSE:
            seen.ADD(c)

    // Convert duplicates set to string
    result = ""
    FOR EACH char c IN duplicates:
        result = result + c

    RETURN result
END FUNCTION
```

```
public static string DuplicateCharacters(string s)
{
    char[] charCount = new char[256];
    foreach (char c in s)
    {
        charCount[c]++;
    }

    StringBuilder result = new StringBuilder();
    foreach (char c in s)
    {
        if (charCount[c] > 1)
        {
            result.Append(c);
        }
    }

    return result.ToString();
}
```

```
String: p r o g r a m m i n g
Index:  1 2 3 4 5 6 7 8 9 10 11

Step 1: Count occurrences
  p:1, r:2, o:1, g:2, a:1, m:2, i:1, n:1

Step 2: Mark duplicates
  Duplicates: r(✓), g(✓), m(✓)

Step 3: Collect in original order
  p(✗) r(✓) o(✗) g(✓) r(✓) a(✗) m(✓) m(✓) i(✗) n(✗) g(✓)
  Result: r g r m m g
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Fizz Buzz

Description: Return string array where each number is replaced: 3→"Fizz", 5→"Buzz", 15→"FizzBuzz", else the number itself.

Examples:

```
Input: n = 3
Output: ["1","2","Fizz"]
Explanation: 3 is divisible by 3

Input: n = 5
Output: ["1","2","Fizz","4","Buzz"]
Explanation: 3→Fizz, 5→Buzz

Input: n = 15
Output: ["1","2","Fizz","4","Buzz","Fizz","7","8","Fizz","Buzz","11","Fizz","13","14","FizzBuzz"]
Explanation: 3,6,9,12→Fizz; 5,10→Buzz; 15→FizzBuzz
```

Constraints

- Input n is positive integer
- For each i from 1 to n:
    - If divisible by 3: "Fizz"
    - If divisible by 5: "Buzz"
    - If divisible by both 15: "FizzBuzz"
    - Otherwise: string representation of number

Given an integer `n`, return a string array answer (1-indexed) where:

- answer[i] == "FizzBuzz" if i is divisible by 3 and 5.
- answer[i] == "Fizz" if i is divisible by 3.
- answer[i] == "Buzz" if i is divisible by 5.
- answer[i] == i (as a string) if none of the above conditions are true.

Brute Force

Pseudocode:

```
FUNCTION FizzBuzz(n):
    // Initialize result list
    result = NEW LIST()

    // Iterate from 1 to n
```

```
    FOR i = 1 TO n:
        IF i MOD 3 == 0 AND i MOD 5 == 0:
            result.ADD("FizzBuzz")
        ELSE IF i MOD 3 == 0:
            result.ADD("Fizz")
        ELSE IF i MOD 5 == 0:
            result.ADD("Buzz")
        ELSE:
            result.ADD(TO_STRING(i))
        END IF
    END FOR

    RETURN result
END FUNCTION

public static IList<string> FizzBuzz(int n)
{
    var result = new List<string>();

    for (int i = 1; i <= n; i++)
    {
        if (i % 3 == 0 && i % 5 == 0)
        {
            result.Add("FizzBuzz");
        }
        else if (i % 3 == 0)
        {
            result.Add("Fizz");
        }
        else if (i % 5 == 0)
        {
            result.Add("Buzz");
        }
        else
        {
            result.Add(i.ToString());
        }
    }
    return result;
}

n = 15

["1", "2", "Fizz", "4", "Buzz", "Fizz", "7", "8", "Fizz", "Buzz", "11", "Fizz", "13", "14", "FizzBuzz"]
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

Length of Last Word

Description: Find the length of the last word in a string (ignoring trailing spaces).

Examples:

```
Input: s = "Hello World"
Output: 5
Explanation: "World" has 5 characters

Input: s = "   fly me   "
Output: 2
Explanation: "me" has 2 characters

Input: s = "luffy is still joyboy"
Output: 6
Explanation: "joyboy" has 6 characters
```

Constraints

- Input string is not null
- String may contain multiple spaces
- Last word contains only letters

Brute Force

Pseudocode:

```
FUNCTION LengthOfLastWord(s):
    // Initialize
    length = 0
    i = LENGTH(s) - 1

    // Skip trailing spaces
    WHILE i >= 0 AND s[i] == ' ':
        i = i - 1
    END WHILE

    // Count characters of last word
    WHILE i >= 0 AND s[i] ≠ ' ':
        length = length + 1
        i = i - 1
    END WHILE

    RETURN length
END FUNCTION

public static int LengthOfLastWord(string s)
{
    int length = 0;
    int i = s.Length - 1;

    // Skip trailing spaces
    while (i >= 0 && s[i] == ' ') i--;

    // Count last word from end
    while (i >= 0 && s[i] != ' ')
    {
        length++;
```

```
        i--;
    }

    return length;
}

String: "  H e l l o   W o r l d  "
Index:   0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
          ↑ ← ← ← ← ← ← ← ← ← ← ← ← ← ← Start from end

Step 1: Skip trailing spaces (← moves left)
        14: ' ' ← skip
        13: ' ' ← skip
        12: 'd' ✓ stop skipping

Step 2: Count characters until space (← moves left, ✓ counts)
        12: 'd' ✓ count=1
        11: 'l' ✓ count=2
        10: 'r' ✓ count=3
        9:  'o' ✓ count=4
        8:  'W' ✓ count=5
        7:  ' ' ✗ stop counting

Result: 5 characters in last word
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Multiply two strings

Description: Multiply two number strings without converting to integers. Handles very large numbers represented as strings.

Examples:

```
Input: num1 = "123", num2 = "456"
Output: "56088"
Explanation: 123 × 456 = 56088

Input: num1 = "0", num2 = "123"
Output: "0"
Explanation: 0 × any number = 0

Input: num1 = "999", num2 = "999"
Output: "998001"
Explanation: 999 × 999 = 998001
```

Constraints

- Input strings represent non-negative integers
- No leading zeros in output (except "0" itself)
- Strings may be very large (cannot convert to int)
- Return result as string

Brute Force

Pseudocode:

```
FUNCTION Multiply(num1, num2):
    // Handle zero multiplication
    IF num1 == "0" OR num2 == "0":
        RETURN "0"

    // Get lengths of numbers
    m = LENGTH(num1)
    n = LENGTH(num2)

    // Initialize result array (max length = m + n)
    result = NEW ARRAY[m + n] INITIALIZED_TO 0

    // Process from right to left (least significant digit first)
    FOR i = m-1 DOWNTO 0:
        FOR j = n-1 DOWNTO 0:
            // Convert characters to digits
            digit1 = num1[i] - '0'
            digit2 = num2[j] - '0'

            // Multiply digits
            mul = digit1 × digit2

            // Calculate positions in result array
            carryIndex = i + j
            valueIndex = i + j + 1

            // Add to existing value at position
            sum = mul + result[valueIndex]

            // Extract carry and digit
            carry = sum DIV 10     // Integer division
            digit = sum MOD 10     // Remainder

            // Update result array
            result[valueIndex] = digit
            result[carryIndex] = result[carryIndex] + carry

        END FOR
    END FOR

    // Convert result array to string, removing leading zeros
    output = NEW StringBuilder()

    FOR a = 0 TO result.LENGTH - 1:
        // Skip leading zeros
        IF output.LENGTH == 0 AND result[a] == 0:
            CONTINUE
```

```
        output.APPEND(result[a])
    END FOR

    // Handle case where result is zero
    IF output.LENGTH == 0:
        RETURN "0"
    ELSE:
        RETURN output.TO_STRING()
END FUNCTION
```

```
public static string Multiply(string num1, string num2)
{
    // Both strings are not null and contains only positive numbers.
    if(num1 == "0" || num2 == "0")
    {
        return "0";
    }

    int m = num1.Length;
    int n = num2.Length;
    // (m + n)
    int[] result = new int [m + n];

    // right to left
    for(int i = m - 1; i >= 0; i--)
    {
        for(int j = n - 1; j >= 0; j--)
        {
            // convert to integer
            int digit1 = num1[i] - '0';
            int digit2 = num2[j] - '0';

            int mul = digit1* digit2;

            int carryIndex = i + j;
            int valueIndex =  i + j + 1;
            int sum = mul + result[valueIndex];  // This adds to existing
            // (15/10 = 1)
            int carry = sum / 10;
            // (15%10 = 5)
            int digit = sum % 10;

            result[valueIndex] = digit; // Set to digit
            result[carryIndex] += carry; // Add carry (accumulate)

        }
    }

    var output = new StringBuilder();
     for(int a = 0; a < result.Length; a++)
     {
        if(output.Length == 0 && result[a] == 0)
        {
            // remove leading zeros
            continue;
        }
        output.Append(result[a]);
     }

     return output.Length > 0 ? output.ToString() : "0";
}
```

```
num1: "12" i=0;
num2: "13" j=0;

ITERATION MAP:
```

Iteration	Positions Used	Result Array Evolution
Initial		[0, 0, 0, 0]
i=1, j=1	carry[2], value[3]	[0, 0, 0, 6] (2×3=6) ↑
i=1, j=0	carry[1], value[2]	[0, 0, 2, 6] (2×1=2) ↑
i=0, j=1	carry[1], value[2] ↑	[0, 0, 5, 6] (1×3=3+2=5) ↑ (overwrites)
i=0, j=0	carry[0], value[1] ↑	[0, 1, 5, 6] (1×1=1) ↑

- **Time Complexity :** `O(m*n)`
- **Space Complexity :** `O(m+n)`

Concatenate two Strings

Description: Interleave characters from two strings alternately, appending remaining characters.

Examples:

```
Input: s1 = "abc", s2 = "pqr"
Output: "apbqcr"
Explanation: Alternating characters from both strings

Input: s1 = "abcd", s2 = "pq"
Output: "apbqcd"
Explanation: Append remaining characters from s1

Input: s1 = "ab", s2 = "pqrs"
Output: "apbqrs"
Explanation: Append remaining characters from s2
```

Constraints

- Both strings are not null or empty
- Interleave characters from both strings
- Include remaining characters from longer string
- Return concatenated result

```
str1 = "abc" st2="pqr" o/p - apbqcr
str1 = "abcd" st2="pq" o/p - apbqcd
str1 = "ab" st2="pqrs" o/p - apbqrs
```

Brute Force (Zipper Merge)

Pseudocode:

```
FUNCTION ConcateStrings(s1, s2):
    // Initialize pointers and result builder
    i = 0
    j = 0
    result = NEW StringBuilder(s1.LENGTH + s2.LENGTH)

    // Alternate characters while both strings have characters
    WHILE i < s1.LENGTH AND j < s2.LENGTH:
        result.APPEND(s1[i])
        result.APPEND(s2[j])
        i = i + 1
        j = j + 1
    END WHILE

    // Append remaining characters from s1
    WHILE i < s1.LENGTH:
        result.APPEND(s1[i])
        i = i + 1
    END WHILE

    // Append remaining characters from s2
    WHILE j < s2.LENGTH:
        result.APPEND(s2[j])
        j = j + 1
    END WHILE

    RETURN result.TO_STRING()
END FUNCTION
```

```
public string ConcateStrings(string s1, string s2)
{

    int i = 0;
    int j = 0;
    var result = new StringBuilder(s1.Length + s2.Length);

    while (i < s1.Length && j < s2.Length)
    {
        result.Append(s1[i]);
        result.Append(s2[j]);
        i++;
        j++;
    }

    // Add remaning characters of s1 string if any
    for (; i < s1.Length; i++)
    {
        result.Append(s1[i]);
    }

    // Add remaning characters of s2 string if any
    for (; j < s2.Length; j++)
    {
        result.Append(s2[j]);
    }

    return result.ToString();
}
```

```
s1 = "ABC", s2 = "XYZ"
Interleave: A X B Y C Z
Remaining: none
Result: "AXBYCZ"
```

- **Time Complexity :** `O(n + m)`
- **Space Complexity :** `O(n + m)`

Move Zeroes To End

Description: Modify array in-place to move all zeros to the end while maintaining relative order of non-zero elements.

Examples:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
Explanation: Zeros moved to end, order preserved

Input: nums = [0,0,1]
Output: [1,0,0]
Explanation: All zeros moved to end

Input: nums = [1,2,3]
Output: [1,2,3]
Explanation: No zeros, no change
```

Constraints

- Array contains positive numbers only
- Must modify array in-place
- Preserve relative order of non-zero elements

- O(1) extra space required

Brute Force - Two Pass

Pseudocode:

```
FUNCTION MoveZeroes(nums):
    // Pointer for last non-zero position
    lastNonZeroIndex = 0

    // First pass: Move all non-zero elements to front
    FOR i = 0 TO LENGTH(nums) - 1:
        IF nums[i] ≠ 0:
            nums[lastNonZeroIndex] = nums[i]
            lastNonZeroIndex = lastNonZeroIndex + 1
        END IF
    END FOR

    // Second pass: Fill remaining positions with zeros
    FOR i = lastNonZeroIndex TO LENGTH(nums) - 1:
        nums[i] = 0
    END FOR
END FUNCTION
```

```
public static void MoveZeroes(int[] nums)
{
    int lastNonZeroIndex = 0;
    for (int i = 0; i < nums.Length; i++)
    {
        if (nums[i] != 0)
        {
            nums[lastNonZeroIndex++] = nums[i];
        }
    }

    for (int i = lastNonZeroIndex; i < nums.Length; i++)
    {
        nums[i] = 0;
    }
}
```

```
Initial: [4, 2, 0, 1, 0, 3, 0]

Pass 1:
  i=0: 4 → pos 0, L=1: [4, 2, 0, 1, 0, 3, 0]
  i=1: 2 → pos 1, L=2: [4, 2, 0, 1, 0, 3, 0]
  i=2: 0 → skip
  i=3: 1 → pos 2, L=3: [4, 2, 1, 1, 0, 3, 0]
  i=4: 0 → skip
  i=5: 3 → pos 3, L=4: [4, 2, 1, 3, 0, 3, 0]
  i=6: 0 → skip

After pass 1: [4, 2, 1, 3, 0, 3, 0], L=4

Pass 2 (fill zeros from index 4):
  [4, 2, 1, 3, 0, 3, 0] → [4, 2, 1, 3, 0, 0, 0]

Final: [4, 2, 1, 3, 0, 0, 0]
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

Two-Pointer - Single Pass

While the current solution is already O(n) time and O(1) space, we can optimize it further by avoiding redundant assignments when there are no zeros. The **two-pointer** technique only swaps when necessary, reducing write operations.

Pseudocode:

```
FUNCTION MoveZeroesTwoPointer(nums):
    // leftPtr points to position for next non-zero
    leftPtr = 0

    // rightPtr scans the array
    FOR rightPtr = 0 TO LENGTH(nums) - 1:
        IF nums[rightPtr] ≠ 0:
            // Only swap if pointers are at different positions
            IF leftPtr ≠ rightPtr:
                // Swap non-zero to left, zero to right
                nums[leftPtr] = nums[rightPtr]
                nums[rightPtr] = 0

            // Move left pointer forward
            leftPtr = leftPtr + 1
        END IF
    END FOR
END FUNCTION
```

```
public static void MoveZeroesTwoPointer(int[] nums)
{
    int leftPtr = 0;  // Points to position where next non-zero should go

    // First pass: move all non-zeros to the left
    for (int rightPtr = 0; rightPtr < nums.Length; rightPtr++)
    {
        if (nums[rightPtr] != 0)
        {
            // Swap only if pointers are different (zeros exist)
            if (leftPtr != rightPtr)
            {
                nums[leftPtr] = nums[rightPtr];
                nums[rightPtr] = 0;
            }
            leftPtr++;
        }
    }
}
```

```
Input: [0, 1, 0, 3, 12]

Pointers move through array:
  leftPtr=0 (next position for non-zero)
  rightPtr=0 (scanning)

Step 1: rightPtr=0, nums[0]=0 → Skip (zero)
  State: [0, 1, 0, 3, 12]
           L     R

Step 2: rightPtr=1, nums[1]=1 (non-zero)
  leftPtr ≠ rightPtr → Swap
  nums[0]=1, nums[1]=0, leftPtr=1
  State: [1, 0, 0, 3, 12]
              L  R

Step 3: rightPtr=2, nums[2]=0 → Skip (zero)
  State: [1, 0, 0, 3, 12]
             L     R

Step 4: rightPtr=3, nums[3]=3 (non-zero)
  leftPtr ≠ rightPtr → Swap
  nums[1]=3, nums[3]=0, leftPtr=2
  State: [1, 3, 0, 0, 12]
                L  R

Step 5: rightPtr=4, nums[4]=12 (non-zero)
  leftPtr ≠ rightPtr → Swap
  nums[2]=12, nums[4]=0, leftPtr=3
  State: [1, 3, 12, 0, 0]
                   L  R

Final: [1, 3, 12, 0, 0] ✓
```

- **Time Complexity :** `O(n)`
  - Single pass through array (rightPtr goes from 0 to n)
  - Each element visited once
- **Space Complexity :** `O(1)`
  - Only two pointers used
  - In-place modification

Valid Parentheses

Constraints

- Input string is not null
- Contains three types: (), {}, []
- Every opening bracket has matching closing bracket
- Brackets must be closed in correct order
- Assume input contains only bracket characters

The string is not null.

```
s = "({[]})"
Result: true ✓
```

[Brute Force - Multiple If-Else]

```
public static bool IsValid(string s)
 {

     if (s.Length % 2 != 0)
     {
         return false;
     }

     Stack<char> braces = new Stack<char>();
     foreach (char ch in s)
     {

         if (braces.Count == 0)
         {
             braces.Push(ch);
             continue;
         }

         if (ch == ')' && braces.Peek() == '(')
         {
             braces.Pop();
         }
         else if (ch == '}' && braces.Peek() == '{')
         {
             braces.Pop();
         }
         else if (ch == ']' && braces.Peek() == '[')
         {
             braces.Pop();
         }
         else
         {
             braces.Push(ch);
         }

     }

     return braces.Count == 0;
 }


String:  {  [  (  )  ]  }
         1  2  3  4  5  6

Step 1: Push { → Stack: {
Step 2: Push [ → Stack: { [
Step 3: Push ( → Stack: { [ (
Step 4: ) matches ( → Pop ( → Stack: { [
```

```
Step 5: ] matches [ → Pop [ → Stack: {
Step 6: } matches { → Pop { → Stack: EMPTY

All matched → Valid ✓
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

[Bracket Mapping - Dictionary]

The current solution requires multiple if-else statements for bracket matching. We can optimize code clarity and slightly improve performance using a **dictionary** for bracket pair mapping.

```
public static bool IsValidWithMapping(string s)
{
    // Early exit for odd-length strings (can't be balanced)
    if (s.Length % 2 != 0)
    {
        return false;
    }

    // Define bracket pair mapping
    Dictionary<char, char> bracketMap = new Dictionary<char, char>
    {
        { ')', '(' },
        { '}', '{' },
        { ']', '[' }
    };

    Stack<char> stack = new Stack<char>();

    foreach (char ch in s)
    {
        // If current char is a closing bracket
        if (bracketMap.ContainsKey(ch))
        {
            // Check if stack is empty or top doesn't match
            if (stack.Count == 0 || stack.Pop() != bracketMap[ch])
            {
                return false;
            }
        }
        else
        {
            // Current char is an opening bracket
            stack.Push(ch);
        }
    }

    // Valid if stack is empty (all brackets matched)
    return stack.Count == 0;
}

String: ({[]})
Bracket map: { ')' → '(', '}' → '{', ']' → '[' }

Position 0: ch='(' → Opening bracket → Push (
  Stack: (

Position 1: ch='{' → Opening bracket → Push {
  Stack: ( {

Position 2: ch='[' → Opening bracket → Push [
  Stack: ( { [

Position 3: ch=']' → Closing bracket
  Check: bracketMap[']'] = '[', stack.Pop() = '[' ✓ Match!
  Stack: ( {

Position 4: ch='}' → Closing bracket
  Check: bracketMap['}'] = '{', stack.Pop() = '{' ✓ Match!
  Stack: (

Position 5: ch=')' → Closing bracket
  Check: bracketMap[')'] = '(', stack.Pop() = '(' ✓ Match!
  Stack: EMPTY

Return: stack.Count == 0 → true ✓

All brackets matched in order!
```

Advantages of Mapping Approach:

```
Original (Multiple if-else):
  if (ch == ')' && braces.Peek() == '(') { braces.Pop(); }
  else if (ch == '}' && braces.Peek() == '{') { braces.Pop(); }
  else if (ch == ']' && braces.Peek() == '[') { braces.Pop(); }
  else { braces.Push(ch); }

Problems:
  - 3 separate comparisons per character
  - Redundant bracket pushing logic
  - Not scalable to more bracket types

Optimized (Dictionary Mapping):
  if (bracketMap.ContainsKey(ch)) {
    if (stack.Pop() != bracketMap[ch]) return false;
  } else {
    stack.Push(ch);
  }

Benefits:
  - Cleaner, more readable code
  - Easily extensible for more bracket types
  - Dictionary lookup is O(1) average case
  - Single code path for all closing brackets
```

Comparison:

Aspect	Original	Mapping
Closing bracket detection	3 separate if-else	Dictionary contains check
Matching logic	Repeated in each condition	Unified with dictionary lookup
Extensibility	Add more if-else for new types	Just add to dictionary
Code lines	10+	6-8
Logic clarity	Verbose	Clear intent

- **Time Complexity :** `O(n)`
  - Single pass through string
  - Stack operations (push/pop): O(1)
  - Dictionary operations (contains/lookup): O(1) average
- **Space Complexity :** `O(n)`
  - Stack size: O(n) in worst case (all opening brackets)
  - Dictionary size: O(1) constant (always 3 bracket pairs)

50. Evaluate Reverse Polish Notation

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

The string is not null.

```
Example 1:
Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9
Example 2:
Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: (4 + (13 / 5)) = 6
Example 3:
Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Output: 22
```

```
public static int EvalRPN(string[] tokens)
{
    var stack = new Stack<int>();
    foreach (var token in tokens)
    {
        // if token is a number, push it onto the stack
        if (int.TryParse(token, out int number))
        {
            stack.Push(number);
        }
        else
        {
            // token is an operator, pop two numbers from the stack
            int b = stack.Pop();
            int a = stack.Pop();
            int result = token switch
            {
                "+" => a + b,
                "-" => a - b,
                "*" => a * b,
                "/" => a / b,
                _ => throw new InvalidOperationException("Invalid operator")
            };
            // push the result back onto the stack
            stack.Push(result);
        }
    }
    // return the final result
    return stack.Pop();

}
```

tokens = ["4", "13", "5", "/", "+"]

1. Push 4 → Stack: [4]
2. Push 13 → Stack: [4, 13]
3. Push 5 → Stack: [4, 13, 5]
4. "/" → Pop 5, Pop 13 → 13 / 5 = 2 → Push 2 → Stack: [4, 2]
5. "+" → Pop 2, Pop 4 → 4 + 2 = 6 → Push 6 → Stack: [6]

Return: 6 ✓

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

62. Decode String

Constraints

- Input string contains valid encoded format: `k[encoded_string]`
- k is a positive integer
- Brackets may be nested
- String contains only lowercase letters and numbers

Given an encoded string, decode it. The encoding rule is: `k[encoded_string]`, where k is a number and represents repeating the `encoded_string` k times.

```
Examples:
- "3[a]2[bc]" → "aaabcbc"
- "3[a2[c]]" → "accaccacc"
- "2[abc]3[cd]eaea.at" → "abcabccdcdcdeaea.at"
```

62.1 [Recursion with Stack]

```
public static string DecodeString_Recursion(string s)
{
```

```
    Stack<(StringBuilder, int)> stack = new();
    StringBuilder current = new();
    int num = 0;

    foreach (char c in s)
    {
        if (char.IsDigit(c))
        {
            num = num * 10 + (c - '0');
        }
        else if (c == '[')
        {
            // Save current state and start new
            stack.Push((current, num));
            current = new StringBuilder();
            num = 0;
        }
        else if (c == ']')
        {
            // Pop and repeat
            var (prevStr, count) = stack.Pop();
            string temp = current.ToString();
            current = prevStr;
            for (int i = 0; i < count; i++)
            {
                current.Append(temp);
            }
        }
        else
        {
            current.Append(c);
        }
    }

    return current.ToString();
}
```

```
Input: "3[a2[c]]"

c='3': num=3
c='[': Push (new StringBuilder, 3), current=empty, num=0
c='a': current="a"
c='2': num=2
c='[': Push (current="a", 2), current=empty, num=0
c='c': current="c"
c=']': Pop (prev="a", count=2)
       temp="c", current="a" + "c"×2 = "acc"
c=']': Pop (prev=empty, count=3)
       temp="acc", current = "acc"×3 = "accaccacc"

Result: "accaccacc" ✓
```

- **Time Complexity:** `O(max_k^max_depth × n)` where k is max multiplier
- **Space Complexity:** `O(m)` where m is decoded string length

62.2 [Iterative Stack Approach]

```
public static string DecodeString_Iterative(string s)
{
    Stack<string> strStack = new();
    Stack<int> numStack = new();
    string current = "";
    int num = 0;

    foreach (char c in s)
    {
        if (char.IsDigit(c))
        {
            num = num * 10 + (c - '0');
        }
        else if (c == '[')
        {
            // Push to stacks
            numStack.Push(num);
            strStack.Push(current);
            current = "";
            num = 0;
        }
        else if (c == ']')
        {
            // Pop and build
            int count = numStack.Pop();
            string prev = strStack.Pop();
            string temp = "";
            for (int i = 0; i < count; i++)
            {
                temp += current;
            }
            current = prev + temp;
        }
        else
        {
            current += c;
        }
    }

    return current;
}
```

- **Time Complexity:** `O(max_k^max_depth × n)`
- **Space Complexity:** `O(m)`

66. Two Sum

Constraints

- Array contains positive integers
- Exactly one solution exists (two different indices)

- Cannot use same element twice
- Return array of [index1, index2] where index1 < index2

Given an array of integers `nums` and an integer `target`, return the indices of the two numbers that add up to the target. You may assume each input has exactly one solution, and you cannot use the same element twice.

Constraints:

- You must return the indices (not values)
- Cannot use same element twice
- Return in any order

```
Examples:
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1] (because nums[0] + nums[1] = 2 + 7 = 9)

Input: nums = [3, 3], target = 6
Output: [0, 1] (because nums[0] + nums[1] = 3 + 3 = 6)
```

51.1 [Brute Force - Nested Loop]

Check all pairs of numbers to find the two that sum to target.

```
public static int[] TwoSumBruteForce(int[] nums, int target)
{
    // Check every pair
    for (int i = 0; i < nums.Length; i++)
    {
        for (int j = i + 1; j < nums.Length; j++)
        {
            if (nums[i] + nums[j] == target)
            {
                return new int[] { i, j };
            }
        }
    }

    return new int[] { -1, -1 }; // No solution found
}
```

```
nums = [2, 7, 11, 15], target = 9

i=0, j=1: 2+7=9 ✓ → Return [0, 1]

Without early exit (showing all comparisons):
i=0: j=1 (2+7=9✓), j=2 (2+11=13✗), j=3 (2+15=17✗)
i=1: j=2 (7+11=18✗), j=3 (7+15=22✗)
i=2: j=3 (11+15=26✗)
```

- **Time Complexity :** `O(n²)`
  - Nested loops checking all pairs
- **Space Complexity :** `O(1)`
  - Only using constant space

51.2 [Hash Map - Two Pass]

Use a hash map to store seen numbers and their indices. For each number, check if `target - number` exists in the map.

```
public static int[] TwoSumHashMap(int[] nums, int target)
{
    // Dictionary to store value → index mapping
    var numMap = new Dictionary<int, int>();

    // Store all numbers and their indices
    for (int i = 0; i < nums.Length; i++)
    {
        numMap[nums[i]] = i;
    }

    // Check each number to find its complement
    for (int i = 0; i < nums.Length; i++)
    {
        int complement = target - nums[i];

        // Check if complement exists and is not the same element
        if (numMap.ContainsKey(complement) && numMap[complement] != i)
        {
            return new int[] { i, numMap[complement] };
        }
    }

    return new int[] { -1, -1 };
}
```

```
nums = [2, 7, 11, 15], target = 9

Step 1: Build HashMap
numMap = { 2→0, 7→1, 11→2, 15→3 }

Step 2: Find complement for each number
i=0: nums[0]=2, complement=9-2=7
     7 exists in map at index 1 ✓
     Return [0, 1]
```

- **Time Complexity :** `O(n)`
  - First pass: O(n) to build hash map
  - Second pass: O(n) to find complement
  - Total: O(n) + O(n) = O(n)
- **Space Complexity :** `O(n)`
  - Hash map stores up to n elements

Alternative: One-Pass Hash Map (Even Better)

```
public static int[] TwoSumOnePass(int[] nums, int target)
{
    var numMap = new Dictionary<int, int>();

    for (int i = 0; i < nums.Length; i++)
    {
        int complement = target - nums[i];

        // Check if complement was already seen
        if (numMap.ContainsKey(complement))
        {
            return new int[] { numMap[complement], i };
        }

        // Store current number for future checks
        numMap[nums[i]] = i;
    }

    return new int[] { -1, -1 };
}
```

48. Merge two sorted arrays

Given two sorted arrays arr1[] of size n and arr2[] of size m. Merge these two arrays. After the merge, the first n smallest elements of the combined sorted array should be stored in arr1[], and the remaining m largest elements should be stored in arr2[]. After the merging process, both arr1[] and arr2[] must remain sorted in non-decreasing order.

```
Examples:
Input: arr1[] = [1, 3, 4, 5], arr2[] = [2, 4, 6, 8]
Output: arr1[] = [1, 2, 3, 4], arr2[] = [4 5, 6, 8]
Explanation: Combined sorted array = [1, 2, 3, 4, 4, 5, 6, 8], array arr1[] contains smallest 4 elements: 1, 2, 3, 4, and array arr2[] contains the remaining 4 elemen

Input: arr1[] = [5, 8, 9], arr2[] = [4, 7, 8]
Output: arr1[] = [4, 5, 7], arr2[] = [8, 8, 9]
Explanation: Combined sorted array = [4, 5, 7, 8, 8, 9], array arr1[] contains smallest 3 elements: 4, 5, 7, and array arr2[] contains the remaining 3 elements: 8, 8,
```

```
public int[] MergeSortedArrays(int[] num1, int[] num2)
 {
     int m = num1.Length;
     int n = num2.Length;
     int[] mergedArray = new int[m + n];
     int i = 0, j = 0, k = 0;
     while (i < m && j < n)
     {
         if (num1[i] <= num2[j])
         {
             mergedArray[k++] = num1[i++];
         }
         else
         {
             mergedArray[k++] = num2[j++];
         }
     }
     // Copy remaining elements of num1, if any
     while (i < m)
     {
         mergedArray[k++] = num1[i++];
     }
     // Copy remaining elements of num2, if any
     while (j < n)
     {
         mergedArray[k++] = num2[j++];
     }
     return mergedArray;
 }
```

```
num1:  [1]   3   5   7
         ↑ →   →   →   → end
num2:  [2]   4   6   8   9
         ↑ →   →   →   →   → end

Comparison sequence:
1 vs 2 → 1 ✓
3 vs 2 → 2 ✓
3 vs 4 → 3 ✓
5 vs 4 → 4 ✓
5 vs 6 → 5 ✓
7 vs 6 → 6 ✓
7 vs 8 → 7 ✓
Remaining: 8, 9 ✓
```

- **Time Complexity :** `O(n + m)`
- **Space Complexity :** `O(n + m)`

59. Next Greater Element

Constraints

- Array contains positive integers
- Find next greater element to the right
- Return -1 if no greater element exists
- Use stack for O(n) solution

Given an array, for each element find the next greater element to its right. If not found, return -1.

```
Example: [1, 3, 4, 2]
- 1: next greater = 3
- 3: next greater = 4
- 4: next greater = -1
- 2: next greater = -1

Result: [3, 4, -1, -1]
```

59.1 [Brute Force - Linear Search]

```
public static int[] NextGreaterElement_BruteForce(int[] nums)
{
    int[] result = new int[nums.Length];

    for (int i = 0; i < nums.Length; i++)
    {
        result[i] = -1;
        for (int j = i + 1; j < nums.Length; j++)
        {
            if (nums[j] > nums[i])
            {
                result[i] = nums[j];
                break;
            }
        }
    }

    return result;
}
```

- **Time Complexity:** `O(n²)` - nested loops
- **Space Complexity:** `O(n)` - result array

59.2 [Monotonic Stack - O(n)]

```
public static int[] NextGreaterElement_MonotonicStack(int[] nums)
{
    int[] result = new int[nums.Length];
    Array.Fill(result, -1);
    Stack<int> stack = new();  // Store indices

    // Traverse from right to left
    for (int i = nums.Length - 1; i >= 0; i--)
    {
        // Pop all smaller elements
        while (stack.Count > 0 && nums[stack.Peek()] <= nums[i])
        {
            stack.Pop();
        }

        // Top of stack is next greater (if exists)
        if (stack.Count > 0)
        {
            result[i] = nums[stack.Peek()];
        }

        // Push current index
        stack.Push(i);
    }

    return result;
}
```

```
nums: [1, 3, 4, 2]

i=3 (val=2): Stack empty → result[3]=-1, Push 3
  Stack: [3]

i=2 (val=4): nums[3]=2 <= 4? YES, Pop 3 → Empty
  Stack empty → result[2]=-1, Push 2
  Stack: [2]

i=1 (val=3): nums[2]=4 > 3? YES, KEEP
  result[1]=4, Push 1
  Stack: [2, 1]

i=0 (val=1): nums[1]=3 > 1? YES, KEEP
  result[0]=3, Push 0
  Stack: [2, 1, 0]

Result: [3, 4, -1, -1] ✓
```

- **Time Complexity:** `O(n)` - each element pushed/popped once
- **Space Complexity:** `O(n)` - stack size

60. Daily Temperatures

Constraints

- Input array contains daily temperatures
- Find days until warmer temperature
- Return 0 if no warmer day exists
- O(n) time using stack preferred

Given an array of temperatures, return for each day how many days you have to wait until a warmer temperature.

```
Example: [73, 74, 75, 71, 69, 72, 76, 73]
- Day 0 (73°): Wait 1 day → 74°
- Day 1 (74°): Wait 1 day → 75°
- Day 2 (75°): Wait 4 days → 76°
- Day 3 (71°): Wait 1 day → 72°
- Day 4 (69°): Wait 1 day → 72°
- Day 5 (72°): Wait 1 day → 76°
- Day 6 (76°): No warmer → 0
- Day 7 (73°): No warmer → 0

Result: [1, 1, 4, 2, 1, 1, 0, 0]
```

60.1 [Brute Force - Linear Search]

```
public static int[] DailyTemperatures_BruteForce(int[] temperatures)
{
    int[] result = new int[temperatures.Length];

    for (int i = 0; i < temperatures.Length; i++)
    {
```

```
        for (int j = i + 1; j < temperatures.Length; j++)
        {
            if (temperatures[j] > temperatures[i])
            {
                result[i] = j - i;
                break;
            }
        }
    }

    return result;
}
```

- **Time Complexity:** `O(n²)`
- **Space Complexity:** `O(n)`

60.2 [Monotonic Stack - O(n)]

```
public static int[] DailyTemperatures_MonotonicStack(int[] temperatures)
{
    int[] result = new int[temperatures.Length];
    Stack<int> stack = new();  // Store indices of decreasing temperatures

    for (int i = 0; i < temperatures.Length; i++)
    {
        // While current temp is warmer than stack top
        while (stack.Count > 0 && temperatures[i] > temperatures[stack.Peek()])
        {
            int prevIndex = stack.Pop();
            result[prevIndex] = i - prevIndex;
        }

        stack.Push(i);
    }

    return result;
}
```

```
temperatures: [73, 74, 75, 71, 69, 72, 76, 73]

i=0 (73°): Stack empty → Push 0
  Stack: [0]

i=1 (74°): 74 > 73? YES
  Pop 0, result[0] = 1 - 0 = 1 ✓
  Push 1
  Stack: [1]

i=2 (75°): 75 > 74? YES
  Pop 1, result[1] = 2 - 1 = 1 ✓
  Push 2
  Stack: [2]

i=3 (71°): 71 > 75? NO → Push 3
  Stack: [2, 3]

i=4 (69°): 69 > 71? NO → Push 4
  Stack: [2, 3, 4]

i=5 (72°): 72 > 69? YES → Pop 4, result[4] = 1 ✓
          72 > 71? YES → Pop 3, result[3] = 2 ✓
          72 > 75? NO → Push 5
  Stack: [2, 5]

i=6 (76°): 76 > 72? YES → Pop 5, result[5] = 1 ✓
          76 > 75? YES → Pop 2, result[2] = 4 ✓
          Push 6
  Stack: [6]

i=7 (73°): 73 > 76? NO → Push 7
  Stack: [6, 7]

Result: [1, 1, 4, 2, 1, 1, 0, 0] ✓
```

- **Time Complexity:** `O(n)` - each element processed once
- **Space Complexity:** `O(n)`

Isomorphic Strings

Given two strings s and t, determine if they are isomorphic. Two strings are isomorphic if the characters in s can be replaced to get t.

```
Examples:
- s = "egg", t = "add" → true (e→a, g→d)
- s = "badc", t = "baba" → false (a→b, but also a→a conflict)
```

[HashMap - Character Mapping]

```
public static bool IsIsomorphic_HashMap(string s, string t)
{
    if (s.Length != t.Length) return false;

    Dictionary<char, char> map = new();

    for (int i = 0; i < s.Length; i++)
    {
        char sChar = s[i];
        char tChar = t[i];

        if (map.ContainsKey(sChar))
        {
            if (map[sChar] != tChar)
            {
                return false; // Conflict: s[i] maps to different t char
            }
        }
        else
```

```
        {
            map[sChar] = tChar;
        }
    }

    return true;
}
```

```
s = "egg", t = "add"

i=0: e→a (map: {e→a})
i=1: g→d (map: {e→a, g→d})
i=2: g→d (already mapped, g→d ✓)

Result: true ✓

---

s = "badc", t = "baba"

i=0: b→b (map: {b→b})
i=1: a→a (map: {b→b, a→a})
i=2: d→b (map: {b→b, a→a, d→b})
i=3: c→a (map: {b→b, a→a, d→b, c→a})

No conflicts! But wait...
a maps to 'a' in position 1 and 'a' in position 3 ✓
But d maps to 'b', and b maps to 'b' → 'b' appears in both!

This algorithm only checks one direction (s→t), not reverse!
```

- **Time Complexity:** `O(n)` - single pass
- **Space Complexity:** `O(k)` - where k is size of alphabet (≤256)

[Bidirectional Mapping]

```
public static bool IsIsomorphic_Bidirectional(string s, string t)
{
    if (s.Length != t.Length) return false;

    Dictionary<char, char> mapStoT = new();
    Dictionary<char, char> mapTtoS = new();

    for (int i = 0; i < s.Length; i++)
    {
        char sChar = s[i];
        char tChar = t[i];

        // Check s → t mapping
        if (mapStoT.ContainsKey(sChar))
        {
            if (mapStoT[sChar] != tChar) return false;
        }
        else
        {
            mapStoT[sChar] = tChar;
        }

        // Check t → s mapping (ensures one-to-one)
        if (mapTtoS.ContainsKey(tChar))
        {
            if (mapTtoS[tChar] != sChar) return false;
        }
        else
        {
            mapTtoS[tChar] = sChar;
        }
    }

    return true;
}
```

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(k)` - two hash maps

First Unique Character in a String

Given a string s, find the first non-repeating character in it and return its index. If the string does not contain a unique character, return -1.

```
Examples:
- s = "leetcode" → 0 (l)
- s = "loveleetcode" → 2 (v)
- s = "aabb" → -1
```

44.1 [Brute Force - Nested Loop]

```
public static int FirstUniqChar_BruteForce(string s)
{
    // For each character, check if it appears elsewhere
    for (int i = 0; i < s.Length; i++)
    {
        bool isUnique = true;
        for (int j = 0; j < s.Length; j++)
        {
            if (i != j && s[i] == s[j])
            {
                isUnique = false;
                break;
            }
        }

        if (isUnique)
        {
            return i;
        }
    }
```

```
    return -1;
}
```

```
s = "leetcode"

i=0 (l): Check against all → appears once → return 0 ✓

Result: 0
```

- **Time Complexity:** `O(n²)` - nested loops
- **Space Complexity:** `O(1)` - no extra space

44.2 [HashMap - O(n)]

```
public static int FirstUniqChar_HashMap(string s)
{
    // Count frequency of each character
    Dictionary<char, int> freq = new();
    foreach (char c in s)
    {
        if (freq.ContainsKey(c))
            freq[c]++;
        else
            freq[c] = 1;
    }

    // Find first character with frequency 1
    for (int i = 0; i < s.Length; i++)
    {
        if (freq[s[i]] == 1)
        {
            return i;
        }
    }

    return -1;
}
```

```
s = "leetcode"

Pass 1 - Count:
l:1, e:3, t:1, c:1, o:1, d:1

Pass 2 - Find First with count=1:
i=0: l has count 1 → return 0 ✓

Result: 0
```

- **Time Complexity:** `O(n)` - two passes
- **Space Complexity:** `O(k)` - frequency map of alphabet size

36. Generate All Subsets

Description: Generate all possible subsets (power set) of a given array using bitwise iteration.

Examples:

```
Input: nums = [1,2]
Output: [[], [1], [2], [1,2]]
Explanation: All subsets of {1,2}

Input: nums = [0]
Output: [[], [0]]
Explanation: Power set has 2^1 = 2 subsets

Input: nums = [1,2,3]
Output: [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
Explanation: 2^3 = 8 subsets total
```

Constraints

- Input array may contain duplicate elements
- Generate all possible subsets (power set)
- Subsets should be unique if duplicates exist
- Return List of Lists

```
public static List<List<int>> GenerateSubsets(int[] nums)
{
    List<List<int>> subsets = new List<List<int>>();
    int n = nums.Length;

    // 2^n possible subsets
    for (int mask = 0; mask < (1 << n); mask++)
    {
        List<int> subset = new List<int>();

        for (int i = 0; i < n; i++)
        {
            // Check if i-th bit is set in mask
            if ((mask & (1 << i)) != 0)
            {
                subset.Add(nums[i]);
            }
        }

        subsets.Add(subset);
    }

    return subsets;
}

// Example: nums=[1,2,3]
// mask=0(000): []
// mask=1(001): [1]
// mask=2(010): [2]
// mask=3(011): [1,2]
// mask=4(100): [3]
```

```
// mask=5(101): [1,3]
// mask=6(110): [2,3]
// mask=7(111): [1,2,3]
```

- **Time Complexity :** `O(n × 2ⁿ)`
- **Space Complexity :** `O(n × 2ⁿ)`

Longest Substring Without Repeating Characters

Constraints

- Input string is not null or empty
- Find substring with no repeating characters
- String contains lowercase English letters
- Return length of longest substring

Given a string s, find the length of the longest substring without repeating characters.

```
Examples:
- s = "abcabcbb" → 3 ("abc")
- s = "bbbbb" → 1 ("b")
- s = "pwwkew" → 3 ("wke")
```

[Brute Force - All Substrings]

```
public static int LengthOfLongestSubstring_BruteForce(string s)
{
    int maxLength = 0;

    // Try all substrings
    for (int i = 0; i < s.Length; i++)
    {
        HashSet<char> seen = new();
        for (int j = i; j < s.Length; j++)
        {
            if (seen.Contains(s[j]))
            {
                break; // Repeating character found
            }

            seen.Add(s[j]);
            maxLength = Math.Max(maxLength, j - i + 1);
        }
    }

    return maxLength;
}
```

```
s = "abcabcbb"

i=0: a,b,c (max=3), hit 'a' → break
i=1: b,c,a (max=3), hit 'b' → break
i=2: c,a,b (max=3), hit 'c' → break
i=3: a,b,c (max=3), hit 'a' → break
...

Result: 3
```

- **Time Complexity:** `O(n²)` - nested loops
- **Space Complexity:** `O(k)` - HashSet

[Sliding Window - O(n)]

```
public static int LengthOfLongestSubstring_SlidingWindow(string s)
{
    Dictionary<char, int> charIndex = new();
    int maxLength = 0;
    int left = 0;

    for (int right = 0; right < s.Length; right++)
    {
        char c = s[right];

        // If character already in current window, move left pointer
        if (charIndex.ContainsKey(c) && charIndex[c] >= left)
        {
            left = charIndex[c] + 1;
        }

        charIndex[c] = right;
        maxLength = Math.Max(maxLength, right - left + 1);
    }

    return maxLength;
}
```

```
s = "abcabcbb"

[a]          left=0, right=0, max=1
[ab]         left=0, right=1, max=2
[abc]        left=0, right=2, max=3
[bca]        left=1, right=3, max=3 (a at 0, move left to 1)
[cab]        left=2, right=4, max=3 (b at 1, move left to 2)
[abc]        left=3, right=5, max=3 (c at 2, move left to 3)
[bc]         left=4, right=6, max=3 (b at 4, move left to 5)
[b]          left=5, right=7, max=3 (b at 5, move left to 6)

Result: 3
```

- **Time Complexity:** `O(n)` - single pass with two pointers
- **Space Complexity:** `O(k)` - HashMap of characters

53. Longest Common Prefix

Constraints

- Input string array is not null or empty
- Strings are not null individually
- Find longest common prefix of all strings
- Return empty string if no common prefix

The string is not null.

```
public static string LongestCommonPrefix(string[] strs)
{
    if (strs.Length == 0)
    {
        return "";
    }

    string smallStr = strs[0];

    for(int i = 0; i < smallStr.Length; i++)
    {
        for (int j = 1; j < strs.Length; j++)
        {
            if (i >= strs[j].Length || strs[j][i] != smallStr[i])
            {
                return smallStr.Substring(0, i);
            }
        }
    }

    return smallStr;
}
```

```
Strings:  flower
          flow
          flight

Position 0:  f   f   f  → All 'f' ✓
Position 1:  l   l   l  → All 'l' ✓
Position 2:  o   o   i  → Mismatch! 'o' ≠ 'i' ✗

Common prefix: "fl"
```

- **Time Complexity :** `O(n*m)`
- **Space Complexity :** `O(1)`

21. Print all subsequence

Description: Generate all possible subsequences of a string using backtracking.

Examples:

```
Input: "ABC"
Output: ["", "A", "B", "C", "AB", "AC", "BC", "ABC"]
Explanation: All 2^n subsequences

Input: "AB"
Output: ["", "A", "B", "AB"]
Explanation: 4 subsequences

Input: "A"
Output: ["", "A"]
Explanation: 2 subsequences
```

Constraints

- Input string is not null or empty
- Generate all possible subsequences
- Output order follows natural recursion tree

Pseudocode:

```
FUNCTION PrintAllSubsequence(input, output)
  IF input.LENGTH == 0 THEN
    PRINT output
    RETURN
  END IF
  first ← input[0]
  rest ← input[1:]
  PrintAllSubsequence(rest, output + first)
  PrintAllSubsequence(rest, output)
END FUNCTION
```

```
public void PrintAllSubsequence(string input, string output)
{
    // Base case
    if (input.Length == 0)
    {
        Console.WriteLine(output);
        return;
    }

    // Inductive Hypothesis
    char firstChar = input[0];
    string restOfString = input.Substring(1);
    // Inductive Step
    // Include the first character
    PrintAllSubsequence(restOfString, output + firstChar);
    // Exclude the first character
    PrintAllSubsequence(restOfString, output);
}
```

```
                    Print("abc", "")
                         /        \
                        /          \
            Include 'a'            Exclude 'a'
```

```
            Print("bc", "a")        Print("bc", "")
                /    \                  /    \
               /      \                /      \
        Include 'b'  Exclude 'b'  Include 'b'  Exclude 'b'
   Print("c","ab") Print("c","a") Print("c","b") Print("c","")
        /    \        /    \        /    \        /    \
       /      \      /      \      /      \      /      \
   Inc 'c'  Exc 'c' Inc 'c' Exc 'c' Inc 'c' Exc 'c' Inc 'c' Exc 'c'
   "abc"    "ab"    "ac"    "a"     "bc"    "b"     "c"     ""

output:
"abc"
"ab"
"ac"
"a"
"bc"
"b"
"c"
""
```

- **Time Complexity :** `O(n × 2ⁿ)`
  - Total recursive calls: `2ⁿ⁺¹ - 1`
  - Each call: `O(n)` for Substring(1) operation
- **Space Complexity :** `O(d²)`
  - Call stack depth: `d` (when going down one branch)
  - String operations create new strings: `O(d²)` total space

51. Remove All Adjacent Duplicates In String

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them. We repeatedly make duplicate removals on s until we no longer can. Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

```
Example 1:
Input: s = "abbaca"
Output: "ca"
Explanation:
For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move.  The result of this move is that the strin
```

```
public static string RemoveDuplicates(string s)
{
    // Use a StringBuilder as a stack-like structure
    var sb = new StringBuilder(s.Length);

    foreach (char ch in s)
    {
        int lastIndex = sb.Length - 1;

        if (lastIndex >= 0 && sb[lastIndex] == ch)
        {
            sb.Length--;  // pop last character
        }
        else
        {
            sb.Append(ch); // push
        }
    }

    return sb.ToString();
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

58. Min Stack

Constraints

- Design stack with push, pop, top, and getMin operations
- All operations must be O(1) time complexity
- Stack is not null
- Values are integers

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time O(1).

```
Operations:
- push(x): Push element x onto stack
- pop(): Remove top element
- top(): Get top element
- getMin(): Get minimum element in O(1)
```

22. Print All Permutation of String

Description: Generate all permutations of a string using backtracking.

Examples:

```
Input: "ABC"
Output: ["ABC", "ACB", "BAC", "BCA", "CAB", "CBA"]
Explanation: All n! permutations

Input: "AB"
Output: ["AB", "BA"]
Explanation: 2 permutations

Input: "A"
Output: ["A"]
Explanation: 1 permutation
```

Constraints

- Input string is not null or empty

- Characters can repeat in output permutations
- Generate all unique permutations

Given a string s, the task is to return all permutations of a given string in lexicographically sorted order. Note: A permutation is the rearrangement of all the elements of a string. Duplicate arrangement can exist.

```
Examples:
Input: s = "ABC"
Output: "ABC", "ACB", "BAC", "BCA", "CAB", "CBA"
Input: s = "XY"
Output: "XY", "YX"
Input: s = "AAA"
Output: "AAA", "AAA", "AAA", "AAA", "AAA", "AAA"
```

Pseudocode:

```
FUNCTION PrintAllPermutationOfString(str, index)
  IF index >= LENGTH(str) - 1 OR LENGTH(str) == 0 THEN
    PRINT str
    RETURN
  END IF
  FOR i FROM index TO LENGTH(str) - 1 DO
    SWAP str[index] WITH str[i]
    PrintAllPermutationOfString(str, index + 1)
    SWAP str[index] WITH str[i]  // Backtrack
  END FOR
END FUNCTION
```

```
public static void PrintAllPermutationOfString(char[] str, int index)
{
    // Base case
    if (index >= str.Length - 1 || str.Length == 0)
    {
        Console.WriteLine(new string(str));
        return;
    }

    // Inductive Hypothesis and Inductive Step
    for(int i = index; i < str.Length; i++)
    {
        // Swap the current index with the loop index
        char temp = str[index];
        str[index] = str[i];
        str[i] = temp;
        // Recurse for the next index
        PrintAllPermutationOfString(str, index + 1);
        // Backtrack: Swap back to the original configuration
        temp = str[index];
        str[index] = str[i];
        str[i] = temp;
    }
}
```

```
Initial: ABC (index=0)
    │
    ├─ i=0: ABC → ABC (index=1)
    │         │
    │         ├─ i=1: ABC → ABC (index=2) → Print ABC
    │         │
    │         └─ i=2: ABC → ACB (index=2) → Print ACB
    │
    ├─ i=1: ABC → BAC (index=1)
    │         │
    │         ├─ i=1: BAC → BAC (index=2) → Print BAC
    │         │
    │         └─ i=2: BAC → BCA (index=2) → Print BCA
    │
    └─ i=2: ABC → CBA (index=1)
              │
              ├─ i=1: CBA → CBA (index=2) → Print CBA
              │
              └─ i=2: CBA → CAB (index=2) → Print CAB
```

- **Time Complexity :** `O(n × n!)`
  - Total permutations: `n!`
  - Each permutation: `O(n)` to print/construct string
  - Each permutation also requires `O(n)` swaps along the path
- **Space Complexity :** `O(n)`
  - Call stack depth: `n` (when going down one branch)
  - Character array: `O(n)` modified in-place

Reverse Words in a String

Constraints

- Input string is not null or empty
- Reverse order of words (maintain word order internally)
- Handle multiple spaces between words
- Trim leading/trailing spaces

Given an input string s, reverse the order of the words. A word is defined as a sequence of non-space characters.

```
Examples:
- s = "the sky is blue" → "blue is sky the"
- s = "  hello world  " → "world hello"
```

[Split and Reverse]

```
public static string ReverseWords_Split(string s)
{
    // Split on spaces and filter empty strings
    string[] words = s.Split(' ', System.StringSplitOptions.RemoveEmptyEntries);

    // Reverse array
```

```
    System.Array.Reverse(words);

    // Join back
    return string.Join(" ", words);
}

s = "the sky is blue"

Split: ["the", "sky", "is", "blue"]
Reverse: ["blue", "is", "sky", "the"]
Join: "blue is sky the"

Result: "blue is sky the" ✓
```

- **Time Complexity:** `O(n)` - split + reverse + join
- **Space Complexity:** `O(n)` - word array

[Two-Pointer]

```
public static string ReverseWords_TwoPointer(string s)
{
    char[] chars = s.ToCharArray();
    int n = chars.Length;

    // Step 1: Reverse entire string
    Reverse(chars, 0, n - 1);

    // Step 2: Reverse each word
    int start = 0;
    for (int i = 0; i <= n; i++)
    {
        if (i == n || chars[i] == ' ')
        {
            Reverse(chars, start, i - 1);
            start = i + 1;
        }
    }

    // Step 3: Remove extra spaces
    return CleanSpaces(chars, n);
}

private static void Reverse(char[] chars, int start, int end)
{
    while (start < end)
    {
        char temp = chars[start];
        chars[start] = chars[end];
        chars[end] = temp;
        start++;
        end--;
    }
}

private static string CleanSpaces(char[] chars, int n)
{
    int j = 0;
    for (int i = 0; i < n; i++)
    {
        if (chars[i] != ' ')
        {
            // Add space before word (if not first word)
            if (j != 0) chars[j++] = ' ';
            while (i < n && chars[i] != ' ')
            {
                chars[j++] = chars[i++];
            }
        }
    }
    return new string(chars, 0, j);
}
```

- **Time Complexity:** `O(n)` - multiple single passes
- **Space Complexity:** `O(1)` - in-place (after converting to char array)

Word Break

Constraints

- Input string s is not null or empty
- wordDict contains valid dictionary words
- Each word in dict is unique
- Must use all characters in s
- Word can be reused multiple times

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of dictionary words.

```
Examples:
- s = "leetcode", dict = ["leet","code"] → true
- s = "applepenapple", dict = ["apple","pen"] → true
- s = "catsandog", dict = ["cat","cats","and","sand","dog"] → false
```

[Recursion with Memoization]

```
public static bool WordBreak_Memoization(string s, IList<string> wordDict)
{
    var memo = new Dictionary<int, bool>();
    var dictSet = new HashSet<string>(wordDict);

    return CanBreak(s, 0, dictSet, memo);
}

private static bool CanBreak(string s, int start, HashSet<string> dict, Dictionary<int, bool> memo)
{
    // Base case
```

```
    if (start == s.Length)
    {
        return true;
    }

    // Check memo
    if (memo.ContainsKey(start))
    {
        return memo[start];
    }

    // Try all possible words starting at 'start'
    for (int end = start + 1; end <= s.Length; end++)
    {
        string word = s.Substring(start, end - start);

        if (dict.Contains(word) && CanBreak(s, end, dict, memo))
        {
            memo[start] = true;
            return true;
        }
    }

    memo[start] = false;
    return false;
}

s = "leetcode", dict = {leet, code}

CanBreak(0):
  ├─ word="l" → not in dict
  ├─ word="le" → not in dict
  ├─ word="lee" → not in dict
  ├─ word="leet" → in dict! → CanBreak(4)
  │   ├─ word="c" → not in dict
  │   ├─ word="co" → not in dict
  │   ├─ word="cod" → not in dict
  │   └─ word="code" → in dict! → CanBreak(8)
  │       └─ start==length → return true ✓
  └─ return true

Result: true ✓
```

- **Time Complexity:** `O(n²)` with memoization (each position computed once)
- **Space Complexity:** `O(n)` - memo dictionary + recursion stack

[Dynamic Programming - Tabulation]

```
public static bool WordBreak_DP(string s, IList<string> wordDict)
{
    var dictSet = new HashSet<string>(wordDict);
    var dp = new bool[s.Length + 1];
    dp[0] = true; // Empty string is always breakable

    for (int i = 1; i <= s.Length; i++)
    {
        for (int j = 0; j < i; j++)
        {
            // If s[0:j] is breakable and s[j:i] is in dict
            if (dp[j] && dictSet.Contains(s.Substring(j, i - j)))
            {
                dp[i] = true;
                break;
            }
        }
    }

    return dp[s.Length];
}

s = "leetcode", dict = {leet, code}

dp[0] = true (empty)

i=1: s[0:1]="l" → not in dict → dp[1]=false
i=2: s[0:2]="le" → not in dict → dp[2]=false
i=3: s[0:3]="lee" → not in dict → dp[3]=false
i=4: s[0:4]="leet" → in dict, dp[0]=true → dp[4]=true ✓
i=5: s[4:5]="c" → dp[4]=true but not in dict → dp[5]=false
...
i=8: s[4:8]="code" → dp[4]=true, "code" in dict → dp[8]=true ✓

Result: dp[8] = true ✓
```

i	s[0:i]	dp[i]	Reason
0	""	T	Base case
1	"l"	F	Not in dict
2	"le"	F	Not in dict
3	"lee"	F	Not in dict
4	"leet"	T	"leet" in dict
5	"leetc"	F	"c" not in dict
6	"leetco"	F	"co" not in dict
7	"leetcod"	F	"cod" not in dict
8	"leetcode"	T	"code" in dict, dp[4]=T

- **Time Complexity:** `O(n² × m)` - n^2 substrings, m for substring comparison
- **Space Complexity:** `O(n)` - DP array

Flatten

Description: Flatten a nested list structure recursively into a single-level list.

Examples:

```
Input: [[1,1],2,[1,1]]
Output: [1,1,2,1,1]
Explanation: Flatten all nested levels

Input: [1,[4,[6]]]
Output: [1,4,6]
Explanation: Multiple levels of nesting

Input: [1,2,3]
Output: [1,2,3]
Explanation: Already flat
```

Constraints

- Input is a nested list structure
- Flatten all levels recursively
- Preserve order of elements

Brute Force - Recursive

Pseudocode:

```
FUNCTION FlattenList(list)
  result ← empty list
  FOR each item IN list DO
    IF item IS list THEN
      result.ADD_ALL(FlattenList(item))
    ELSE
      result.ADD(item)
    END IF
  END FOR
  RETURN result
END FUNCTION
```

```
public static List<object> FlattenList(IEnumerable<object> nestedList)
{
    List<object> result = new List<object>();

    foreach (var item in nestedList)
    {
        if (item is Array && !(item is string))
        {
            result.AddRange(FlattenList((IEnumerable<object>)item));
        }
        else
        {
            result.Add(item);
        }
    }

    return result;
}
```

```
FlattenList([1, [2,3], [4,[5,6]], 7]) (Level 0)
│
├─ item=1 → Add 1 → result=[1]
│
├─ item=[2,3] (Array)
│  │
│  └─ FlattenList([2,3]) (Level 1)
│     │
│     ├─ item=2 → Add 2 → result=[2]
│     │
│     └─ item=3 → Add 3 → result=[2,3]
│        Return [2,3]
│  AddRange([2,3]) → result=[1,2,3]
│
├─ item=[4,[5,6]] (Array)
│  │
│  └─ FlattenList([4,[5,6]]) (Level 1)
│     │
│     ├─ item=4 → Add 4 → result=[4]
│     │
│     └─ item=[5,6] (Array)
│        │
│        └─ FlattenList([5,6]) (Level 2)
│           │
│           ├─ item=5 → Add 5 → result=[5]
│           │
│           └─ item=6 → Add 6 → result=[5,6]
│              Return [5,6]
│        AddRange([5,6]) → result=[4,5,6]
│     Return [4,5,6]
│  AddRange([4,5,6]) → result=[1,2,3,4,5,6]
│
└─ item=7 → Add 7 → result=[1,2,3,4,5,6,7]
   Return [1,2,3,4,5,6,7]
```

- **Time Complexity :** `O(n)`
  - `n` = total number of elements across all levels
- **Space Complexity :** `O(d)`
  - Call stack depth = O(d) (number of digit groups)
  - Additional `O(n)` space for result list

House Robber

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array nums representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

```
Example 1:
Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.
```

```
Example 2:
Input: nums = [2,7,9,3,1]
Output: 12
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.
```

Brute Force - Recursion

Pseudocode:

```
FUNCTION Rob(nums, n)
  IF n < 0 THEN
    RETURN 0
  END IF
  include ← nums[n] + Rob(nums, n - 2)
  exclude ← Rob(nums, n - 1)
  RETURN MAX(include, exclude)
END FUNCTION
```

```
public static int Rob(int[] nums, int n)
{
    if (n < 0)
    {
        return 0;
    }
    int includeCurrent = nums[n] + Rob(nums, n - 2);
    int excludeCurrent = Rob(nums, n - 1);
    return Math.Max(includeCurrent, excludeCurrent);
}
```

```
                      Rob(4)
                     /      \
                    /        \
          1+Rob(2)            Rob(3)
           /   \              /    \
          /     \            /      \
   9+Rob(0)   Rob(1)    3+Rob(1)  Rob(2)
     /   \    /   \       |        /   \
    /     \  /     \      |       /     \
2+Rob(-2) 0 7+0   Rob(0) 7+0  9+Rob(0) Rob(1)
   |       |   |    |     |      |        |
   2       0   7    2     7      11       7
```

- **Time Complexity :** $O(2^n)$
  - Each call makes 2 recursive calls (n-1 and n-2)
  - Forms binary tree with $\approx 2^n$ nodes
- **Space Complexity :** $O(d)$
  - Call stack depth = O(d) (number of digit groups)

Memoization Approch with Recursion

Pseudocode:

```
FUNCTION RobMemo(nums, n, memo = null):
    // Initialize memo dictionary if not provided
    IF memo == null:
        memo = NEW Dictionary<int, int>()

    // Base case: no houses to rob
    IF n < 0:
        RETURN 0

    // Return cached result if available
    IF memo CONTAINS KEY n:
        RETURN memo[n]

    // Two choices:
    // 1. Rob current house + best of n-2
    includeCurrent = nums[n] + RobMemo(nums, n - 2, memo)

    // 2. Skip current house, take best of n-1
    excludeCurrent = RobMemo(nums, n - 1, memo)

    // Take maximum of both choices
    result = MAX(includeCurrent, excludeCurrent)

    // Store result in memo for future use
    memo[n] = result

    RETURN result
END FUNCTION
```

```
public static int RobMemo(int[] nums, int n, Dictionary<int, int> memo = null)
{
    if (memo == null) memo = new Dictionary<int, int>();

    if (n < 0) return 0;

    if (memo.ContainsKey(n)) return memo[n];

    int include = nums[n] + RobMemo(nums, n - 2, memo);
    int exclude = RobMemo(nums, n - 1, memo);
    int result = Math.Max(include, exclude);

    memo[n] = result;
    return result;
}
```

```
                      RobMemo(4)
                      /        \
                     /          \
          1+RobMemo(2)         RobMemo(3) FROM MEMO!
           /       \            (already computed)
          /         \
9+RobMemo(0)   RobMemo(1) FROM MEMO!
   /     \      (already computed)
```

```
  /       \
2+Rob(-2) Rob(-1) FROM MEMO!
```

- **Time Complexity :** `O(n)`
  - Each Rob number computed once
- **Space Complexity :** `O(n)`
  - For memo dictionary + call stack

25. Staircase

Description: Count the number of ways to climb n stairs if you can take either 1 or 2 stairs at a time.

There are n stairs, and a person standing at the bottom wants to climb stairs to reach the top. The person can climb either 1 stair or 2 stairs at a time, the task is to count the number of ways that a person can reach at the top.

Examples:

```
Input: n = 1
Output: 1
Explanation: There is only one way to climb 1 stair.
Input: n = 2
Output: 2
Explanation: There are two ways to reach 2th stair: {1, 1} and {2}.
Input: n = 4
Output: 5
Explanation: There are five ways to reach 4th stair: {1, 1, 1, 1}, {1, 1, 2}, {2, 1, 1}, {1, 2, 1} and {2, 2}.
```

25.1 [Brute Force - Recursive]

Pseudocode:

```
FUNCTION ClimbStairs(n)
  IF n == 0 OR n == 1 THEN
    RETURN 1
  END IF
  IF n == 2 THEN
    RETURN 2
  END IF
  RETURN ClimbStairs(n - 1) + ClimbStairs(n - 2)
END FUNCTION
```

```
public int ClimbStairs(int n)
{
    // Base case
    if (n == 0 || n == 1)
    {
        return 1;
    }

    if (n == 2)
    {
        return 2;
    }

    //Inductive Hypothesis
    int waysFromNMinus1 = ClimbStairs(n - 1);
    int waysFromNMinus2 = ClimbStairs(n - 2);
    //Inductive Step
    return waysFromNMinus1 + waysFromNMinus2;
}
```

```
ClimbStairs(6)
├─ ClimbStairs(5)  // Calculates ClimbStairs(4), ClimbStairs(3), etc.
│  ├─ ClimbStairs(4)  // Calculates ClimbStairs(3), ClimbStairs(2)
│  └─ ClimbStairs(3)  // Calculates ClimbStairs(2), ClimbStairs(1)
└─ ClimbStairs(4)  // **DUPLICATE!** Calculates same as above
   ├─ ClimbStairs(3)  // **DUPLICATE!**
   └─ ClimbStairs(2)  // **DUPLICATE!**
```

- **Time Complexity :** `O(2ⁿ)`
  - Each call makes 2 recursive calls (n-1 and n-2)
  - Forms a binary tree of depth ≈ n
  - Total nodes ≈ 2^n (exponential growth)
- **Space Complexity :** `O(n)`
  - Call stack depth: `n` (when going down one branch)

25.2 [Memoization]

The brute force approach has exponential time complexity due to redundant calculations. We can optimize this by storing already computed results using **memoization**.

```
public int ClimbStairsWithMemo(int n, Dictionary<int, int> memo = null)
{
    if (memo == null) memo = new Dictionary<int, int>();

    // Base case
    if (n == 0 || n == 1)
    {
        return 1;
    }

    if (n == 2)
    {
        return 2;
    }

    // Check if result is already computed
    if (memo.ContainsKey(n))
    {
        return memo[n];
    }

    // Compute result and store in memo
```

```
    int result = ClimbStairsWithMemo(n - 1, memo) + ClimbStairsWithMemo(n - 2, memo);
    memo[n] = result;
    return result;
}
```

```
ClimbStairs(6) with Memoization

                         ClimbStairs(6) → memo[6]=?
                         /              \
                        /                \
            ClimbStairs(5) → memo[5]=?  ClimbStairs(4) → memo[4] FROM MEMO! ✓
            /              \            (already computed when processing left branch)
           /                \
ClimbStairs(4) → memo[4]=?  ClimbStairs(3) → memo[3] FROM MEMO! ✓
   /           \
  /             \
Compute...  ClimbStairs(2) → memo[2]=2 (base case)

Memoization prevents redundant calculations:
- ClimbStairs(4): Computed once, then retrieved from memo
- ClimbStairs(3): Computed once, then retrieved from memo
- ClimbStairs(2): Computed once, then retrieved from memo
- ClimbStairs(1): Computed once, then retrieved from memo
```

Key Improvements:

- **Before (Brute Force):** ClimbStairs(5) was called **multiple times** with exponential growth
- **After (Memoization):** Each value computed **only once**, results cached for reuse

```
Comparison: n=5

Brute Force Call Tree:
               ClimbStairs(5)
              /              \
          Clim(4)          Clim(3) ← Recalculated!
         /       \          /      \
      Clim(3)  Clim(2)  Clim(2)  Clim(1) ← Multiple calculations
     /    \      (base)  (base)   (base)
  Clim(2) Clim(1) ...

Total calls for n=5: 15 function calls

Memoization Call Tree:
               ClimbStairs(5)
              /              \
          Clim(4)          Clim(3) ← MEMO!
         /       \          /      \
      Clim(3)  Clim(2)   (cached) Clim(1)
     /    \      (base)           (base)
  Clim(2) Clim(1)
   (base)  (base)

Total calls for n=5: 5 function calls
```

- **Time Complexity :** `O(n)`
  - Each unique value (0 to n) computed exactly once
  - Memo lookup: `O(1)` per call
  - Total: `n` unique computations = `O(n)`
- **Space Complexity :** `O(n)`
  - Memo dictionary stores `n` entries
  - Call stack depth: `O(n)` maximum
  - Total: `O(n)`

---

26. Tower of Hanoi

Description: Solve the Tower of Hanoi problem: Move n disks from rod A to rod C using rod B as auxiliary, with rules that only one disk can be moved at a time and a larger disk cannot be placed on a smaller one.

Examples:

```
Input: n = 1
Output: 1
Explanation: Move disk from A to C directly (1 move)

Input: n = 2
Output: 3
Explanation: A→B, A→C, B→C (3 moves)

Input: n = 3
Output: 7
Explanation: 7 moves to transfer all disks
```

Constraints

- Input n is positive integer (number of disks)
- All disks start on rod A, must move to rod C
- Can only move one disk at a time
- Larger disk cannot be placed on smaller disk

Recursively calculates the minimum moves to solve Tower of Hanoi with n disks using the recurrence relation: `T(n) = 2×T(n-1) + 1`.

Pseudocode:

```
FUNCTION TowerOfHanoi(n)
  IF n == 0 THEN
    RETURN 0
  END IF
  smallResult ← TowerOfHanoi(n - 1)
  RETURN 2 * smallResult + 1
END FUNCTION

public static int TowerOfHanoi(int n)
{
    // Base case
```

```
    if (n == 0)
    {
        return 0;
    }
    //Inductive Hypothesis
    int smallResult = TowerOfHanoi(n - 1);
    //Inductive Step
    return 2 * smallResult + 1;
}

Call Stack (growing down):
TowerOfHanoi(3) waiting, will compute 2×?+1
  TowerOfHanoi(2) waiting, will compute 2×?+1
    TowerOfHanoi(1) waiting, will compute 2×?+1
      TowerOfHanoi(0) → returns 0
    returns 2×0+1 = 1
  returns 2×1+1 = 3
returns 2×3+1 = 7
```

- **Time Complexity :** `O(n)`
  - `n` recursive calls
- **Space Complexity :** `O(n)`
  - Call stack depth: `n` (when going down one branch)

27. Print Tower of Hanoi

Description: Print the moves required to solve the Tower of Hanoi problem: Move n disks from rod A to rod C using rod B, displaying each move with from-rod and to-rod.

Constraints

- Input n is positive integer (number of disks)
- Three rods: A (source), B (auxiliary), C (destination)
- Print all moves required to solve the puzzle
- Each move is from one rod to another

Tower of Hanoi is a mathematical puzzle where we have three rods (A, B, and C) and N disks. Initially, all the disks are stacked in decreasing value of diameter i.e., the smallest disk is placed on the top and they are on rod A. The objective of the puzzle is to move the entire stack to another rod (here considered C), obeying the following simple rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
- No disk may be placed on top of a smaller disk.

Examples:

```
Input: 2
Output: Disk 1 moved from A to B
Disk 2 moved from A to C
Disk 1 moved from B to C
Input: 3
Output: Disk 1 moved from A to C
Disk 2 moved from A to B
Disk 1 moved from C to B
Disk 3 moved from A to C
Disk 1 moved from B to A
Disk 2 moved from B to C
Disk 1 moved from A to C
```

Pseudocode:

```
FUNCTION PrintTowerOfHanoiMoves(n, source, destination, auxiliary)
  IF n == 0 THEN
    RETURN
  END IF
  PrintTowerOfHanoiMoves(n - 1, source, auxiliary, destination)
  PRINT "Move disk" n "from" source "to" destination
  PrintTowerOfHanoiMoves(n - 1, auxiliary, destination, source)
END FUNCTION

public void PrintTowerOfHanoiMoves(int n, char source, char destination, char auxiliary)
{
    // Base case
    if (n == 0)
    {
        return;
    }
    //Inductive Hypothesis and Inductive Step
    // Move n-1 disks from source to auxiliary
    PrintTowerOfHanoiMoves(n - 1, source, auxiliary, destination);
    // Move the nth disk from source to destination
    Console.WriteLine($"Move disk {n} from {source} to {destination}");
    // Move n-1 disks from auxiliary to destination
    PrintTowerOfHanoiMoves(n - 1, auxiliary, destination, source);
}

PrintTowerOfHanoiMoves(3, A, C, B)
│
├─ PrintTowerOfHanoiMoves(2, A, B, C)
│  │
│  ├─ PrintTowerOfHanoiMoves(1, A, C, B)
│  │  └─ Print: "Move disk 1 from A to C" (1)
│  │
│  ├─ Print: "Move disk 2 from A to B" (2)
│  │
│  └─ PrintTowerOfHanoiMoves(1, C, B, A)
│     └─ Print: "Move disk 1 from C to B" (3)
│
├─ Print: "Move disk 3 from A to C" (4)
│
└─ PrintTowerOfHanoiMoves(2, B, C, A)
   │
   ├─ PrintTowerOfHanoiMoves(1, B, A, C)
   │  └─ Print: "Move disk 1 from B to A" (5)
   │
   ├─ Print: "Move disk 2 from B to C" (6)
   │
```

```
└─ PrintTowerOfHanoiMoves(1, A, C, B)
   └─ Print: "Move disk 1 from A to C" (7)
```

- **Time Complexity :** `O(2ⁿ)`
  - Recurrence: `T(n) = 2×T(n-1) + 1`
- **Space Complexity :** `O(n)`
  - Call stack depth: `n` (when going down one branch)

28. Knight's Tour Problem

Constraints

- Input is board size n (n × n chessboard)
- Knight starts at position (0, 0)
- Knight must visit all squares exactly once
- Knight moves in L-shape: 2 squares one direction, 1 square perpendicular
- Find if complete tour exists

The Knight's Tour is a sequence of moves of a knight on a chessboard such that the knight visits every square exactly once. Given an `n × n` chessboard and a starting position, find if a complete knight's tour exists. A knight moves in an "L" shape: 2 squares in one direction and 1 square perpendicular (or vice versa).

Constraints:

- The knight must visit each square exactly once
- The knight can start from any position
- A complete tour exists only on certain board sizes

```
Examples:
8x8 board - Knight's Tour exists (Famous Puzzle)
4x4 board - Knight's Tour may/may not exist depending on starting position
3x3 board - No Knight's Tour exists

Sample 4x4 solution (starting at 0,0):
 0  1  2  3
 7  4  9  2
 6 11 10  1
 5 12 13  8
```

28.1 [Backtracking - Brute Force]

Try all possible knight moves from each position using backtracking. If a dead-end is reached, backtrack and try different moves.

```
public class KnightTourBruteForce
{
    private int n;
    private int[,] board;
    private int[] moveX = { 2, 1, -1, -2, -2, -1, 1, 2 };
    private int[] moveY = { 1, 2, 2, 1, -1, -2, -2, -1 };

    public bool FindKnightTour(int n, int startX, int startY)
    {
        this.n = n;
        this.board = new int[n, n];

        // Initialize board with -1 (unvisited)
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                board[i, j] = -1;
            }
        }

        // Mark starting position
        board[startX, startY] = 0;

        // Try to find tour starting from (startX, startY)
        if (Backtrack(startX, startY, 1))
        {
            PrintBoard();
            return true;
        }

        return false;
    }

    private bool Backtrack(int x, int y, int moveCount)
    {
        // If all squares are visited, tour is complete
        if (moveCount == n * n)
        {
            return true;
        }

        // Try all 8 possible knight moves
        for (int i = 0; i < 8; i++)
        {
            int nextX = x + moveX[i];
            int nextY = y + moveY[i];

            // Check if next position is valid and unvisited
            if (IsValid(nextX, nextY) && board[nextX, nextY] == -1)
            {
                // Mark current square
                board[nextX, nextY] = moveCount;

                // Recursively try to complete the tour
                if (Backtrack(nextX, nextY, moveCount + 1))
                {
                    return true;
                }

                // Backtrack: unmark the square
                board[nextX, nextY] = -1;
            }
```

```
        }

        return false;
    }

    private bool IsValid(int x, int y)
    {
        return x >= 0 && x < n && y >= 0 && y < n;
    }

    private void PrintBoard()
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                Console.Write(board[i, j].ToString().PadLeft(3));
            }
            Console.WriteLine();
        }
    }
}
```

```
Knight's Tour on 5x5 board (Brute Force Backtracking):

Starting at (0,0):
Move 1: (0,0) → Try move to (1,2) ✓
Move 2: (1,2) → Try moves... eventually reaches dead-end
        Backtrack! Unmark (1,2)
        Try next move...

This process explores all possibilities:
- Time: Extremely slow because explores ALL possible sequences
- Worst case: All permutations of positions
```

Knight Move Pattern:

```
     (up-left)  (up-right)
    ↗       ↖
   (-2,1) (2,1)
(-1,2)           (1,2)
  ↗               ↖
Knight            (x,y)
  ↖               ↗
(-1,-2)         (1,-2)
   ↙               ↖
    (-2,-1) (2,-1)
    ↙       ↖
   (down-left) (down-right)
```

- **Time Complexity :** `O(8^(n²))`
  - Worst case: Try all 8 moves for each of n^2 squares
  - Most moves lead to dead-ends
- **Space Complexity :** `O(n²)`
  - Board matrix: n^2
  - Recursion depth: up to n^2

28.2 [Backtracking - Warnsdorff's Heuristic]

Instead of trying all moves randomly, use **Warnsdorff's Rule**: Always move the knight to the square from which the knight will have the fewest onward moves. This dramatically reduces backtracking by avoiding dead-ends early.

```
public class KnightTourWarnsdorff
{
    private int n;
    private int[,] board;
    private int[] moveX = { 2, 1, -1, -2, -2, -1, 1, 2 };
    private int[] moveY = { 1, 2, 2, 1, -1, -2, -2, -1 };

    public bool FindKnightTourOptimized(int n, int startX, int startY)
    {
        this.n = n;
        this.board = new int[n, n];

        // Initialize board
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                board[i, j] = -1;
            }
        }

        board[startX, startY] = 0;

        if (BacktrackWarnsdorff(startX, startY, 1))
        {
            PrintBoard();
            return true;
        }

        return false;
    }

    private bool BacktrackWarnsdorff(int x, int y, int moveCount)
    {
        // If all squares visited, tour complete
        if (moveCount == n * n)
        {
            return true;
        }

        // Get all valid next moves
        var nextMoves = GetNextMovesByWarnsdorff(x, y);

        // Try moves sorted by Warnsdorff's heuristic
        foreach (var (nextX, nextY) in nextMoves)
        {
```

```
            board[nextX, nextY] = moveCount;

            if (BacktrackWarnsdorff(nextX, nextY, moveCount + 1))
            {
                return true;
            }

            board[nextX, nextY] = -1;
        }

        return false;
    }

    private List<(int, int)> GetNextMovesByWarnsdorff(int x, int y)
    {
        var validMoves = new List<(int, int, int)>(); // (x, y, degree)

        // Find all valid moves and count their accessibility
        for (int i = 0; i < 8; i++)
        {
            int nextX = x + moveX[i];
            int nextY = y + moveY[i];

            if (IsValid(nextX, nextY) && board[nextX, nextY] == -1)
            {
                // Count how many moves are available from this position
                int degree = CountAccessibility(nextX, nextY);
                validMoves.Add((nextX, nextY, degree));
            }
        }

        // Sort by degree (ascending) - prefer squares with fewer onward moves
        validMoves.Sort((a, b) => a.Item3.CompareTo(b.Item3));

        // Return moves sorted by Warnsdorff's heuristic
        return validMoves.Select(m => (m.Item1, m.Item2)).ToList();
    }

    private int CountAccessibility(int x, int y)
    {
        int count = 0;

        // Count valid unvisited squares reachable from (x, y)
        for (int i = 0; i < 8; i++)
        {
            int nextX = x + moveX[i];
            int nextY = y + moveY[i];

            if (IsValid(nextX, nextY) && board[nextX, nextY] == -1)
            {
                count++;
            }
        }

        return count;
    }

    private bool IsValid(int x, int y)
    {
        return x >= 0 && x < n && y >= 0 && y < n;
    }

    private void PrintBoard()
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                Console.Write(board[i, j].ToString().PadLeft(3));
            }
            Console.WriteLine();
        }
    }
}
```

```
Warnsdorff's Heuristic Example (5x5 board):

At each step, choose move to square with fewest onward moves:

Position (0,0), moveCount=0
├─ Can move to: (1,2), (2,1)
├─ Count moves from each:
│  ├─ From (1,2): 5 onward moves
│  └─ From (2,1): 4 onward moves ← Choose this (fewer moves)
│
At (2,1), moveCount=1
├─ Can move to: (0,0)✓, (0,2), (1,3), (3,3), (4,0), (4,2)
├─ Count accessibility for each unvisited
│  ├─ (0,2): 3 onward moves
│  ├─ (1,3): 4 onward moves ← Choose (0,2) (fewest)
│  └─ ...

This greedy choice avoids creating unreachable corners!
```

Warnsdorff's Rule:

- Always move to the square with the **fewest onward moves**
- This prevents the knight from being trapped in unreachable areas
- Dramatically reduces backtracking by making smart early decisions
- **Time Complexity :** `O(n²)` (Average case with Warnsdorff's heuristic)
    - Most board sizes: Single linear pass finds solution
    - Very few backtracks needed due to smart heuristic
    - Without heuristic: `O(8^(n²))` worst case
- **Space Complexity :** `O(n²)`

- Board matrix: n^2
- Recursion depth: up to n^2 (but rarely backtracks deep)

Comparison Table:

Approach	Time	Space	Strategy
Brute Force	$O(8^{(n^2)})$	$O(n^2)$	Try all moves blindly
Warnsdorff's	$O(n^2)$	$O(n^2)$	Greedy heuristic - avoid dead-ends

Why Warnsdorff's Works: Warnsdorff's heuristic is so effective that it finds a solution (when one exists) in almost linear time! The key insight is that by prioritizing moves to squares with fewer onward options, we prevent the knight from painting itself into corners where no more moves are possible.

63. Find First and Last Position of an Element in Sorted Array

Description: Find first and last positions of target in sorted array, return [-1,-1] if not found.

Examples:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Explanation: First 8 at index 3, last 8 at index 4

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
Explanation: Target 6 not found

Input: nums = [8], target = 8
Output: [0,0]
Explanation: Single element
```

Constraints

- Input array is sorted in ascending order
- Find first and last position of target
- Return [-1, -1] if target not found
- O(log n) time complexity required

```
public static List<int> FindFirstAndLastPositionOfAnElement(int[] sortedArray, int target)
{
    var result = new List<int> { -1, -1 };
    int left = 0;
    int right = sortedArray.Length - 1;
    // Find the first occurrence
    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (sortedArray[mid] >= target)
        {
            right = mid - 1;
        }
        else
        {
            left = mid + 1;
        }
    }
    // Check if the target is not found
    if (left >= sortedArray.Length || sortedArray[left] != target)
    {
        return result;
    }
    result[0] = left; // First occurrence
    // Find the last occurrence
    right = sortedArray.Length - 1; // Reset right pointer
    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (sortedArray[mid] <= target)
        {
            left = mid + 1;
        }
        else
        {
            right = mid - 1;
        }
    }
    result[1] = right; // Last occurrence
    return result;
}
```

```
Array: [5, 7, 7, 8, 8, 10]
Index:  0  1  2  3  4  5
Target: 8

First occurrence search:
  Binary search for "first 8"
  Finds index 3
  Check: arr[3] = 8 ✓

Last occurrence search:
  Binary search for "last 8"
  Starts from index 3
  Finds index 4
  Check: arr[4] = 8 ✓

Result: [3, 4]
```

- **Time Complexity :** `O(log n)`
  - First binary search: O(log n)
  - Second binary search: O(log n)
  - Total: O(2 log n) = O(log n)
- **Space Complexity :** `O(1)`

64. Search in Rotated Sorted Array

There is an integer array nums sorted in ascending order (with distinct values). Prior to being passed to your function, nums is possibly left rotated at an unknown index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2]. Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums. You must write an algorithm with O(log n) runtime complexity.

```
Example 1:
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
Example 2:
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
Example 3:
Input: nums = [1], target = 0
Output: -1
```

```
public static int Search(int[] nums, int target)
{
    // rotacted array binary search
    int left = 0;
    int right = nums.Length - 1;
    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target)
        {
            return mid;
        }
        // Left half is sorted
        if (nums[left] <= nums[mid])
        {
            if (target >= nums[left] && target < nums[mid])
            {
                right = mid - 1;
            }
            else
            {
                left = mid + 1;
            }
        }
        else
        { // Right half is sorted
            if (target > nums[mid] && target <= nums[right])
            {
                left = mid + 1;
            }
            else
            {
                right = mid - 1;
            }
        }
    }
    return -1;
}
```

```
Array: [4, 5, 6, 7, 0, 1, 2]
Index:  0  1  2  3  4  5  6
        ---------  ---------
        Sorted     Sorted
        left half  right half
        (4-7)      (0-2)

Target: 0

Step 1: mid=3 (value 7)
  Left half [4,5,6,7] is sorted
  Target 0 not in [4,7] range → search right half

Step 2: New range [4,5,6] → indices 4-6
  mid=5 (value 1)
  Left half [0,1] (indices 4-5) is sorted
  Target 0 in [0,1] range → search left half

Step 3: New range [4,4] → index 4
  Found target 0 at index 4
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

65. Find Minimum Element in Sorted Rotated Array

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array nums = [0,1,2,4,5,6,7] might become: • [4,5,6,7,0,1,2] if it was rotated 4 times. • [0,1,2,4,5,6,7] if it was rotated 7 times. Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2], ..., a[n-2]]. Given the sorted rotated array nums of unique elements, return the minimum element of this array. You must write an algorithm that runs in O(log n) time.

```
Example 1:
Input: nums = [3,4,5,1,2]
Output: 1
Explanation: The original array was [1,2,3,4,5] rotated 3 times.
Example 2:
Input: nums = [4,5,6,7,0,1,2]
Output: 0
Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
Example 3:
Input: nums = [11,13,15,17]
Output: 11
Explanation: The original array was [11,13,15,17] and it was rotated 4 times.
```

```
public static int FindMin(int[] nums)
{
    int start = 0;
    int end = nums.Length - 1;

    while (start < end)
    {
        int mid = start + (end - start) / 2;
```

```
        // Minimum is in the right half
        if (nums[mid] > nums[end])
        {
            start = mid + 1;
        }
        // Minimum is in the left half (including mid)
        else
        {
            end = mid;
        }
    }

    return nums[start];
}
```

```
Array: [4, 5, 6, 7, 0, 1, 2]
        ↑                 ↑
       start=0           end=6
       mid=3 → nums[3]=7 > nums[6]=2 → min in right → start=4

New range: [4, 5, 6, 7, 0, 1, 2]
                      ↑     ↑
                    start=4 end=6
                    mid=5 → nums[5]=1 ≤ nums[6]=2 → min in left → end=5

New range: [4, 5, 6, 7, 0, 1, 2]
                      ↑  ↑
                    start=4 end=5
                    mid=4 → nums[4]=0 ≤ nums[5]=1 → min in left → end=4

Found: nums[4] = 0
```

- **Time Complexity :** `O(log n)`
- **Space Complexity :** `O(1)`

61. Largest Rectangle in Histogram

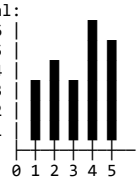
Constraints

- Heights array is not empty
- All values are non-negative integers
- Find largest rectangle area in histogram
- O(n) solution using stack preferred

Given an array of heights representing a histogram, find the largest rectangle area.

```
Example: [2, 1, 5, 6, 2, 3]

Visual:
```




```
Largest rectangle: between indices 2-3, height 5 → area = 5×2 = 10
```

61.1 [Brute Force - All Pairs]

```
public static int LargestRectangleArea_BruteForce(int[] heights)
{
    int maxArea = 0;

    for (int i = 0; i < heights.Length; i++)
    {
        int minHeight = heights[i];

        for (int j = i; j < heights.Length; j++)
        {
            minHeight = Math.Min(minHeight, heights[j]);
            int area = minHeight * (j - i + 1);
            maxArea = Math.Max(maxArea, area);
        }
    }

    return maxArea;
}
```

- **Time Complexity:** `O(n²)`
- **Space Complexity:** `O(1)`

61.2 [Monotonic Stack - O(n)]

```
public static int LargestRectangleArea_MonotonicStack(int[] heights)
{
    Stack<int> stack = new();
    int maxArea = 0;

    for (int i = 0; i < heights.Length; i++)
    {
        // Pop and calculate area for heights taller than current
        while (stack.Count > 0 && heights[stack.Peek()] > heights[i])
        {
            int h = heights[stack.Pop()];
            int w = stack.Count > 0 ? i - stack.Peek() - 1 : i;
            int area = h * w;
            maxArea = Math.Max(maxArea, area);
        }

        stack.Push(i);
    }
```

```
    // Process remaining heights
    while (stack.Count > 0)
    {
        int h = heights[stack.Pop()];
        int w = stack.Count > 0 ? heights.Length - stack.Peek() - 1 : heights.Length;
        int area = h * w;
        maxArea = Math.Max(maxArea, area);
    }

    return maxArea;
}
```

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(n)`

67. Search in 2D Sorted Matrix

Constraints

- Matrix is m × n size
- Rows are sorted in ascending order (left to right)
- Columns are sorted in ascending order (top to bottom)
- Target value may not exist in matrix
- O(m + n) or O(log m + log n) time preferred

Given an `m × n` matrix where rows and columns are sorted in ascending order, search for a target value. Return true if found, false otherwise.

Constraints:

- Rows sorted left to right
- Columns sorted top to bottom
- Matrix is not necessarily sorted overall

```
Example:
Matrix:
[1,  4,  7, 11, 15]
[2,  5,  8, 12, 19]
[3,  6,  9, 16, 22]
[10, 13, 14, 17, 24]
[18, 21, 23, 26, 30]

Search for 13: Found at [3, 1]
Search for 20: Not found
```

52.1 [Brute Force - Linear Search]

Check every element in the matrix sequentially.

```
public static bool SearchMatrixBruteForce(int[][] matrix, int target)
{
    // Check every element
    foreach (int[] row in matrix)
    {
        foreach (int num in row)
        {
            if (num == target)
                return true;
        }
    }

    return false;
}
```

- **Time Complexity :** `O(m × n)`
    - Check all m rows and n columns
- **Space Complexity :** `O(1)`

52.2 [Binary Search - Staircase Search]

Start from top-right corner (or bottom-left). If current element is less than target, move down; if greater, move left. This eliminates one row or column per step.

```
public static bool SearchMatrixStaircase(int[][] matrix, int target)
{
    int m = matrix.Length;        // rows
    int n = matrix[0].Length;      // columns

    // Start from top-right corner
    int row = 0;
    int col = n - 1;

    while (row < m && col >= 0)
    {
        if (matrix[row][col] == target)
        {
            return true;
        }

        if (matrix[row][col] < target)
        {
            // Current element too small, move down
            row++;
        }
        else
        {
            // Current element too large, move left
            col--;
        }
    }

    return false;
}

Matrix:
[1,  4,  7, 11, 15]    row 0
```

```
[2,  5,  8, 12, 19]    row 1
[3,  6,  9, 16, 22]    row 2
[10, 13, 14, 17, 24]   row 3
[18, 21, 23, 26, 30]   row 4
     ↑                   ↑
   col 1               col 4 (start)

Search for 13:
Start: [0, 4] = 15 > 13 → col--
       [0, 3] = 11 < 13 → row++
       [1, 3] = 12 < 13 → row++
       [2, 3] = 16 > 13 → col--
       [2, 2] = 9 < 13 → row++
       [3, 2] = 14 > 13 → col--
       [3, 1] = 13 = 13 → Found! ✓
```

- **Time Complexity :** `O(m + n)`
  - At most m down moves + n left moves
- **Space Complexity :** `O(1)`
  - Only using constant space

68. Spiral Matrix Traversal

Constraints

- Matrix is m × n size
- Traverse clockwise from outside to inside
- Visit all elements exactly once
- Return result as single array

Given an `m × n` matrix, return all elements of the matrix in spiral order (clockwise from outside to inside).

```
Example:
[1,  2,  3]
[4,  5,  6]
[7,  8,  9]

Spiral: [1, 2, 3, 6, 9, 8, 7, 4, 5]
```

53.1 [Brute Force - Layer by Layer]

Process the matrix layer by layer, moving right → down → left → up, then shrink boundaries.

```
public static List<int> SpiralMatrixLayerByLayer(int[][] matrix)
{
    var result = new List<int>();

    int top = 0, bottom = matrix.Length - 1;
    int left = 0, right = matrix[0].Length - 1;

    while (top <= bottom && left <= right)
    {
        // Move right along top row
        for (int col = left; col <= right; col++)
        {
            result.Add(matrix[top][col]);
        }
        top++;

        // Move down along right column
        for (int row = top; row <= bottom; row++)
        {
            result.Add(matrix[row][right]);
        }
        right--;

        // Move left along bottom row (if exists)
        if (top <= bottom)
        {
            for (int col = right; col >= left; col--)
            {
                result.Add(matrix[bottom][col]);
            }
            bottom--;
        }

        // Move up along left column (if exists)
        if (left <= right)
        {
            for (int row = bottom; row >= top; row--)
            {
                result.Add(matrix[row][left]);
            }
            left++;
        }
    }

    return result;
}
```

```
Matrix:
[1,  2,  3]
[4,  5,  6]
[7,  8,  9]

Layer 1:
→ Right: 1, 2, 3
↓ Down: 6, 9
← Left: 8, 7
↑ Up: 4

Layer 2:
  5 (center)

Result: [1, 2, 3, 6, 9, 8, 7, 4, 5]
```

- **Time Complexity :** `O(m × n)`
    - Visit each element exactly once
- **Space Complexity :** `O(1)` (excluding output list)

53.2 [Optimized - Boundary Tracking]

Same approach but cleaner implementation with early termination checks.

```
public static List<int> SpiralMatrixOptimized(int[][] matrix)
{
    var result = new List<int>();

    int top = 0, bottom = matrix.Length - 1;
    int left = 0, right = matrix[0].Length - 1;

    while (top <= bottom && left <= right)
    {
        // Traverse right
        for (int col = left; col <= right; col++)
        {
            result.Add(matrix[top][col]);
        }
        top++;

        // Traverse down
        for (int row = top; row <= bottom; row++)
        {
            result.Add(matrix[row][right]);
        }
        right--;

        // Traverse left (only if row exists)
        if (top <= bottom)
        {
            for (int col = right; col >= left; col--)
            {
                result.Add(matrix[bottom][col]);
            }
            bottom--;
        }

        // Traverse up (only if column exists)
        if (left <= right)
        {
            for (int row = bottom; row >= top; row--)
            {
                result.Add(matrix[row][left]);
            }
            left++;
        }
    }

    return result;
}
```

- **Time Complexity :** `O(m × n)`
- **Space Complexity :** `O(1)`

54. Reverse a Linked List

54.1 [Brute Force - Iterative]

Pseudocode:

```
FUNCTION ReverseIterative(head)
  prev ← NULL
  current ← head
  WHILE current != NULL DO
    nextTemp ← current.next    // Save next node
    current.next ← prev        // Reverse the link
    prev ← current             // Move prev forward
    current ← nextTemp         // Move current forward
  END WHILE
  RETURN prev  // New head
END FUNCTION
```

Code Implementation:

```
public class ListNode {
    public int val;
    public ListNode next;
}

public static ListNode ReverseIterative(ListNode head)
{
    ListNode prev = null;
    ListNode current = head;

    while (current != null)
    {
        ListNode nextTemp = current.next;
        current.next = prev;
        prev = current;
        current = nextTemp;
    }

    return prev;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

54.2 [Recursive]

Pseudocode:

```
FUNCTION ReverseRecursive(head)
  IF head == NULL OR head.next == NULL THEN
    RETURN head
  END IF

  newHead ← ReverseRecursive(head.next)
  head.next.next ← head     // Reverse the link
  head.next ← NULL          // Remove forward link
  RETURN newHead
END FUNCTION
```

Code Implementation:

```
public static ListNode ReverseRecursive(ListNode head)
{
    if (head == null || head.next == null)
        return head;

    ListNode newHead = ReverseRecursive(head.next);

    head.next.next = head;
    head.next = null;

    return newHead;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)` (Call stack)

---

55. Detect Cycle in Linked List

55.1 [Brute Force - Hash Set]

Pseudocode:

```
FUNCTION DetectCycleHashSet(head)
  visited ← EMPTY_SET
  current ← head

  WHILE current != NULL DO
    IF current IN visited THEN
      RETURN TRUE  // Cycle found
    END IF
    visited.ADD(current)
    current ← current.next
  END WHILE

  RETURN FALSE  // No cycle
END FUNCTION
```

Code Implementation:

```
public static bool DetectCycleHashSet(ListNode head)
{
    var visited = new HashSet<ListNode>();
    ListNode current = head;

    while (current != null)
    {
        if (visited.Contains(current))
            return true;

        visited.Add(current);
        current = current.next;
    }

    return false;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

55.2 [Two-Pointer (Floyd's Cycle Detection)]

Pseudocode:

```
FUNCTION DetectCycleFloyd(head)
  slow ← head
  fast ← head

  WHILE fast != NULL AND fast.next != NULL DO
    slow ← slow.next         // Move 1 step
    fast ← fast.next.next    // Move 2 steps

    IF slow == fast THEN      // Cycle detected
      RETURN TRUE
    END IF
  END WHILE

  RETURN FALSE
END FUNCTION
```

Code Implementation:

```
public static bool DetectCycleFloyd(ListNode head)
{
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast)
```

```
            return true;
    }

    return false;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

---

56. Merge Two Sorted Linked Lists

Constraints

- Both lists are sorted in ascending order
- Lists may be of different lengths
- Return merged sorted list
- Use O(1) space (pointer manipulation only)

Pseudocode:

```
FUNCTION MergeSorted(list1, list2)
  dummyNode ← NEW ListNode(0)
  current ← dummyNode

  WHILE list1 != NULL AND list2 != NULL DO
    IF list1.val <= list2.val THEN
      current.next ← list1
      list1 ← list1.next
    ELSE
      current.next ← list2
      list2 ← list2.next
    END IF
    current ← current.next
  END WHILE

  current.next ← list1 != NULL ? list1 : list2
  RETURN dummyNode.next
END FUNCTION
```

Code Implementation:

```
public static ListNode MergeSorted(ListNode list1, ListNode list2)
{
    ListNode dummy = new ListNode(0);
    ListNode current = dummy;

    while (list1 != null && list2 != null)
    {
        if (list1.val <= list2.val)
        {
            current.next = list1;
            list1 = list1.next;
        }
        else
        {
            current.next = list2;
            list2 = list2.next;
        }
        current = current.next;
    }

    current.next = list1 ?? list2;
    return dummy.next;
}
```

- **Time Complexity :** `O(n + m)`
- **Space Complexity :** `O(1)`

---

57. Find Middle of Linked List

Constraints

- Linked list node is not null
- Return middle node (if even length, return second middle)
- Use slow-fast pointer technique
- O(n) time, O(1) space

Pseudocode:

```
FUNCTION FindMiddle(head)
  slow ← head
  fast ← head

  WHILE fast != NULL AND fast.next != NULL DO
    slow ← slow.next       // Move 1 step
    fast ← fast.next.next  // Move 2 steps
  END WHILE

  RETURN slow  // slow is at middle
END FUNCTION
```

Code Implementation:

```
public static ListNode FindMiddle(ListNode head)
{
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
```

```
    }

    return slow;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

---

58. Remove Nth Node From End of List

Constraints

- List contains at least one node
- Remove nth node from end of list
- Use two-pointer (dummy node) approach
- Return head of modified list

Pseudocode:

```
FUNCTION RemoveNthFromEnd(head, n)
  dummy ← NEW ListNode(0)
  dummy.next ← head

  first ← dummy
  second ← dummy

  FOR i = 0 TO n DO
    first ← first.next
  END FOR

  WHILE first.next != NULL DO
    first ← first.next
    second ← second.next
  END WHILE

  second.next ← second.next.next
  RETURN dummy.next
END FUNCTION
```

Code Implementation:

```
public static ListNode RemoveNthFromEnd(ListNode head, int n)
{
    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode first = dummy;
    ListNode second = dummy;

    for (int i = 0; i <= n; i++)
    {
        first = first.next;
    }

    while (first != null)
    {
        first = first.next;
        second = second.next;
    }

    second.next = second.next.next;
    return dummy.next;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

---

59. Linked List Palindrome Check

Constraints

- Linked list node is not null
- Check if list forms a palindrome
- Use fast-slow pointer to find middle
- Reverse second half and compare

Pseudocode:

```
FUNCTION IsPalindrome(head)
  slow ← head
  fast ← head

  WHILE fast != NULL AND fast.next != NULL DO
    slow ← slow.next
    fast ← fast.next.next
  END WHILE

  secondHalf ← ReverseList(slow)

  first ← head
  second ← secondHalf

  WHILE second != NULL DO
    IF first.val != second.val THEN
      RETURN FALSE
    END IF
    first ← first.next
    second ← second.next
  END WHILE

  RETURN TRUE
END FUNCTION
```

Code Implementation:

```
public static bool IsPalindrome(ListNode head)
{
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }

    ListNode secondHalf = ReverseIterative(slow);

    ListNode first = head;
    ListNode second = secondHalf;

    while (second != null)
    {
        if (first.val != second.val)
            return false;
        first = first.next;
        second = second.next;
    }

    return true;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)` or `O(n)`

60. Tree Traversals (Inorder, Preorder, Postorder)

60.1 [Brute Force - Recursive]

Pseudocode (Inorder: Left → Node → Right):

```
FUNCTION InorderTraversal(node)
  IF node == NULL THEN
    RETURN
  END IF

  InorderTraversal(node.left)      // Left
  PRINT node.val                   // Node
  InorderTraversal(node.right)     // Right
END FUNCTION
```

Code Implementation:

```
public class TreeNode {
    public int val;
    public TreeNode left;
    public TreeNode right;
}

public static void InorderRecursive(TreeNode node)
{
    if (node == null) return;

    InorderRecursive(node.left);
    Console.WriteLine(node.val);
    InorderRecursive(node.right);
}

public static void PreorderRecursive(TreeNode node)
{
    if (node == null) return;

    Console.WriteLine(node.val);     // Node first
    PreorderRecursive(node.left);
    PreorderRecursive(node.right);
}

public static void PostorderRecursive(TreeNode node)
{
    if (node == null) return;

    PostorderRecursive(node.left);
    PostorderRecursive(node.right);
    Console.WriteLine(node.val);     // Node last
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)` (h = height, call stack)

60.2 [Iterative - Stack]

Pseudocode (Inorder - Iterative):

```
FUNCTION InorderIterative(root)
  stack ← EMPTY_STACK
  current ← root

  WHILE current != NULL OR stack NOT EMPTY DO
    // Go to leftmost node
    WHILE current != NULL DO
      stack.PUSH(current)
      current ← current.left
    END WHILE

    // Current is NULL, pop from stack
    current ← stack.POP()
    PRINT current.val

    // Visit right subtree
    current ← current.right
  END WHILE
END FUNCTION
```

Code Implementation:

```
public static void InorderIterative(TreeNode root)
{
    var stack = new Stack<TreeNode>();
    TreeNode current = root;

    while (current != null || stack.Count > 0)
    {
        // Go to leftmost node
        while (current != null)
        {
            stack.Push(current);
            current = current.left;
        }

        // Pop and process
        current = stack.Pop();
        Console.WriteLine(current.val);

        // Visit right subtree
        current = current.right;
    }
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

---

61. Level Order Traversal (BFS)

61.1 [Brute Force - Recursive]

Pseudocode:

```
FUNCTION LevelOrderRecursive(node, level, result)
  IF node == NULL THEN
    RETURN
  END IF

  IF level == result.SIZE THEN
    result.ADD(NEW List)
  END IF

  result[level].ADD(node.val)
  LevelOrderRecursive(node.left, level + 1, result)
  LevelOrderRecursive(node.right, level + 1, result)
END FUNCTION
```

Code Implementation:

```
public static void LevelOrderRecursive(TreeNode node, int level, List<List<int>> result)
{
    if (node == null) return;

    if (level == result.Count)
        result.Add(new List<int>());

    result[level].Add(node.val);
    LevelOrderRecursive(node.left, level + 1, result);
    LevelOrderRecursive(node.right, level + 1, result);
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

61.2 [Iterative - Queue]

Pseudocode:

```
FUNCTION LevelOrderIterative(root)
  IF root == NULL THEN
    RETURN EMPTY_LIST
  END IF

  result ← EMPTY_LIST
  queue ← EMPTY_QUEUE
  queue.ENQUEUE(root)

  WHILE queue NOT EMPTY DO
    levelSize ← queue.SIZE
    currentLevel ← EMPTY_LIST

    FOR i = 0 TO levelSize - 1 DO
      node ← queue.DEQUEUE()
      currentLevel.ADD(node.val)

      IF node.left != NULL THEN
        queue.ENQUEUE(node.left)
      END IF

      IF node.right != NULL THEN
        queue.ENQUEUE(node.right)
      END IF
    END FOR

    result.ADD(currentLevel)
  END WHILE

  RETURN result
END FUNCTION
```

Code Implementation:

```
public static List<List<int>> LevelOrderIterative(TreeNode root)
{
```

```
    var result = new List<List<int>>();
    if (root == null) return result;

    var queue = new Queue<TreeNode>();
    queue.Enqueue(root);

    while (queue.Count > 0)
    {
        int levelSize = queue.Count;
        var currentLevel = new List<int>();

        for (int i = 0; i < levelSize; i++)
        {
            TreeNode node = queue.Dequeue();
            currentLevel.Add(node.val);

            if (node.left != null) queue.Enqueue(node.left);
            if (node.right != null) queue.Enqueue(node.right);
        }

        result.Add(currentLevel);
    }

    return result;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(w)` (w = max width)

---

62. Binary Search Tree Validation

62.1 [Brute Force - Min/Max Range]

Pseudocode:

```
FUNCTION IsValidBST(node, minVal, maxVal)
  IF node == NULL THEN
    RETURN TRUE
  END IF

  IF node.val <= minVal OR node.val >= maxVal THEN
    RETURN FALSE
  END IF

  RETURN IsValidBST(node.left, minVal, node.val) AND
         IsValidBST(node.right, node.val, maxVal)
END FUNCTION
```

Code Implementation:

```
public static bool IsValidBST(TreeNode node, long minVal = long.MinValue, long maxVal = long.MaxValue)
{
    if (node == null) return true;

    if (node.val <= minVal || node.val >= maxVal)
        return false;

    return IsValidBST(node.left, minVal, node.val) &&
           IsValidBST(node.right, node.val, maxVal);
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

62.2 [Optimized - Recursive Range Check]

Pseudocode:

```
FUNCTION IsValidBSTOptimized(node, minVal, maxVal)
  IF node == NULL THEN
    RETURN TRUE
  END IF

  IF (minVal != NULL AND node.val <= minVal) OR
     (maxVal != NULL AND node.val >= maxVal) THEN
    RETURN FALSE
  END IF

  RETURN IsValidBSTOptimized(node.left, minVal, node.val) AND
         IsValidBSTOptimized(node.right, node.val, maxVal)
END FUNCTION
```

Code Implementation:

```
public static bool IsValidBSTOptimized(TreeNode node, long? minVal = null, long? maxVal = null)
{
    if (node == null) return true;

    if ((minVal != null && node.val <= minVal) ||
        (maxVal != null && node.val >= maxVal))
        return false;

    return IsValidBSTOptimized(node.left, minVal, node.val) &&
           IsValidBSTOptimized(node.right, node.val, maxVal);
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

---

63. Lowest Common Ancestor (LCA)

63.1 [Brute Force - Store Path]

Pseudocode:

```
FUNCTION FindLCA_StorePath(root, p, q)
  pathP ← FindPath(root, p, EMPTY_LIST)
  pathQ ← FindPath(root, q, EMPTY_LIST)

  lca ← NULL
  FOR i = 0 TO MIN(pathP.SIZE, pathQ.SIZE) - 1 DO
    IF pathP[i] == pathQ[i] THEN
      lca ← pathP[i]
    ELSE
      BREAK
    END IF
  END FOR

  RETURN lca
END FUNCTION

FUNCTION FindPath(node, target, path)
  IF node == NULL THEN
    RETURN FALSE
  END IF

  path.ADD(node)

  IF node.val == target.val THEN
    RETURN TRUE
  END IF

  IF FindPath(node.left, target, path) OR
     FindPath(node.right, target, path) THEN
    RETURN TRUE
  END IF

  path.REMOVE_LAST()
  RETURN FALSE
END FUNCTION
```

Code Implementation:

```
public static TreeNode FindLCA_StorePath(TreeNode root, TreeNode p, TreeNode q)
{
    var pathP = new List<TreeNode>();
    var pathQ = new List<TreeNode>();

    FindPath(root, p, pathP);
    FindPath(root, q, pathQ);

    TreeNode lca = null;
    int minLen = Math.Min(pathP.Count, pathQ.Count);

    for (int i = 0; i < minLen; i++)
    {
        if (pathP[i].val == pathQ[i].val)
            lca = pathP[i];
        else
            break;
    }

    return lca;
}

private static bool FindPath(TreeNode node, TreeNode target, List<TreeNode> path)
{
    if (node == null) return false;

    path.Add(node);

    if (node.val == target.val) return true;

    if (FindPath(node.left, target, path) || FindPath(node.right, target, path))
        return true;

    path.RemoveAt(path.Count - 1);
    return false;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

63.2 [Optimized - Single Pass]

Pseudocode:

```
FUNCTION FindLCA_SinglePass(node, p, q)
  IF node == NULL THEN
    RETURN NULL
  END IF

  IF node.val == p.val OR node.val == q.val THEN
    RETURN node
  END IF

  leftLCA ← FindLCA_SinglePass(node.left, p, q)
  rightLCA ← FindLCA_SinglePass(node.right, p, q)

  IF leftLCA != NULL AND rightLCA != NULL THEN
    RETURN node
  END IF

  RETURN leftLCA != NULL ? leftLCA : rightLCA
END FUNCTION
```

Code Implementation:

```
public static TreeNode FindLCA_SinglePass(TreeNode node, TreeNode p, TreeNode q)
{
    if (node == null) return null;
```

```
    if (node.val == p.val || node.val == q.val)
        return node;

    TreeNode leftLCA = FindLCA_SinglePass(node.left, p, q);
    TreeNode rightLCA = FindLCA_SinglePass(node.right, p, q);

    if (leftLCA != null && rightLCA != null)
        return node;

    return leftLCA ?? rightLCA;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

---

64. Maximum Path Sum in Binary Tree

Constraints

- Tree root may be null
- Path may include any nodes (not necessarily root to leaf)
- Return maximum sum of any path
- Nodes can have negative values

Pseudocode:

```
FUNCTION MaxPathSum(node, maxSum_ref)
  IF node == NULL THEN
    RETURN 0
  END IF

  leftMax ← MAX(0, MaxPathSum(node.left, maxSum_ref))
  rightMax ← MAX(0, MaxPathSum(node.right, maxSum_ref))

  pathSum ← node.val + leftMax + rightMax
  maxSum_ref.value ← MAX(maxSum_ref.value, pathSum)

  RETURN node.val + MAX(leftMax, rightMax)
END FUNCTION
```

Code Implementation:

```
public static int MaxPathSum(TreeNode root)
{
    int[] maxSum = { int.MinValue };
    MaxPathSumHelper(root, maxSum);
    return maxSum[0];
}

private static int MaxPathSumHelper(TreeNode node, int[] maxSum)
{
    if (node == null) return 0;

    int leftMax = Math.Max(0, MaxPathSumHelper(node.left, maxSum));
    int rightMax = Math.Max(0, MaxPathSumHelper(node.right, maxSum));

    int pathSum = node.val + leftMax + rightMax;
    maxSum[0] = Math.Max(maxSum[0], pathSum);

    return node.val + Math.Max(leftMax, rightMax);
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

---

65. Serialize and Deserialize Tree

Constraints

- Tree can be null (leaf position)
- Serialize to string format
- Deserialize back to identical tree
- Use pre-order or BFS traversal

Pseudocode (DFS approach):

```
FUNCTION Serialize(node, result)
  IF node == NULL THEN
    result.ADD("null")
    RETURN
  END IF

  result.ADD(node.val)
  Serialize(node.left, result)
  Serialize(node.right, result)
END FUNCTION

FUNCTION Deserialize(data, index_ref)
  val ← data[index_ref.value]
  index_ref.value ← index_ref.value + 1

  IF val == "null" THEN
    RETURN NULL
  END IF

  node ← NEW TreeNode(INT(val))
  node.left ← Deserialize(data, index_ref)
  node.right ← Deserialize(data, index_ref)
  RETURN node
END FUNCTION
```

Code Implementation:

```
public class Codec
{
    public string Serialize(TreeNode root)
    {
        var result = new List<string>();
        SerializeHelper(root, result);
        return string.Join(",", result);
    }

    private void SerializeHelper(TreeNode node, List<string> result)
    {
        if (node == null)
        {
            result.Add("null");
            return;
        }

        result.Add(node.val.ToString());
        SerializeHelper(node.left, result);
        SerializeHelper(node.right, result);
    }

    public TreeNode Deserialize(string data)
    {
        var values = data.Split(',').ToList();
        return DeserializeHelper(values, new int[] { 0 });
    }

    private TreeNode DeserializeHelper(List<string> values, int[] index)
    {
        if (index[0] >= values.Count || values[index[0]] == "null")
        {
            index[0]++;
            return null;
        }

        TreeNode node = new TreeNode(int.Parse(values[index[0]]));
        index[0]++;

        node.left = DeserializeHelper(values, index);
        node.right = DeserializeHelper(values, index);

        return node;
    }
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(n)`

---

66. Balanced Binary Tree Check

Constraints

- Tree root may be null
- Balanced: height difference of left/right <= 1
- Check balance recursively for all nodes
- Return true if balanced, false otherwise

Pseudocode:

```
FUNCTION IsBalanced(node)
  RETURN GetHeight(node) != -1
END FUNCTION

FUNCTION GetHeight(node)
  IF node == NULL THEN
    RETURN 0
  END IF

  leftHeight ← GetHeight(node.left)
  IF leftHeight == -1 THEN
    RETURN -1  // Left subtree not balanced
  END IF

  rightHeight ← GetHeight(node.right)
  IF rightHeight == -1 THEN
    RETURN -1  // Right subtree not balanced
  END IF

  IF ABS(leftHeight - rightHeight) > 1 THEN
    RETURN -1  // Current node not balanced
  END IF

  RETURN MAX(leftHeight, rightHeight) + 1
END FUNCTION
```

Code Implementation:

```
public static bool IsBalanced(TreeNode root)
{
    return GetHeight(root) != -1;
}

private static int GetHeight(TreeNode node)
{
    if (node == null) return 0;

    int leftHeight = GetHeight(node.left);
    if (leftHeight == -1) return -1;

    int rightHeight = GetHeight(node.right);
    if (rightHeight == -1) return -1;

    if (Math.Abs(leftHeight - rightHeight) > 1)
        return -1;
```

```
    return Math.Max(leftHeight, rightHeight) + 1;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(h)`

---

67. Graph Traversals (DFS, BFS)

67.1 [Brute Force - DFS Recursive]

Pseudocode:

```
FUNCTION DFSRecursive(node, visited)
  visited.ADD(node)
  PRINT node

  FOR each neighbor IN node.neighbors DO
    IF neighbor NOT IN visited THEN
      DFSRecursive(neighbor, visited)
    END IF
  END FOR
END FUNCTION
```

Code Implementation:

```
public static void DFSRecursive(int node, List<List<int>> graph, HashSet<int> visited)
{
    visited.Add(node);
    Console.WriteLine(node);

    foreach (int neighbor in graph[node])
    {
        if (!visited.Contains(neighbor))
            DFSRecursive(neighbor, graph, visited);
    }
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)` (Visited set + Call stack)

67.2 [DFS Iterative - Stack]

Pseudocode:

```
FUNCTION DFSIterative(start, graph)
  visited ← EMPTY_SET
  stack ← EMPTY_STACK
  stack.PUSH(start)

  WHILE stack NOT EMPTY DO
    node ← stack.POP()
    IF node NOT IN visited THEN
      visited.ADD(node)
      PRINT node

      FOR each neighbor IN node.neighbors DO
        IF neighbor NOT IN visited THEN
          stack.PUSH(neighbor)
        END IF
      END FOR
    END IF
  END WHILE
END FUNCTION
```

Code Implementation:

```
public static void DFSIterative(int start, List<List<int>> graph)
{
    var visited = new HashSet<int>();
    var stack = new Stack<int>();
    stack.Push(start);

    while (stack.Count > 0)
    {
        int node = stack.Pop();
        if (!visited.Contains(node))
        {
            visited.Add(node);
            Console.WriteLine(node);

            foreach (int neighbor in graph[node])
            {
                if (!visited.Contains(neighbor))
                    stack.Push(neighbor);
            }
        }
    }
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)`

67.3 [BFS - Queue]

Pseudocode:

```
FUNCTION BFS(start, graph)
  visited ← EMPTY_SET
  queue ← EMPTY_QUEUE
  queue.ENQUEUE(start)
  visited.ADD(start)

  WHILE queue NOT EMPTY DO
    node ← queue.DEQUEUE()
```

```
    PRINT node

    FOR each neighbor IN node.neighbors DO
      IF neighbor NOT IN visited THEN
        visited.ADD(neighbor)
        queue.ENQUEUE(neighbor)
      END IF
    END FOR
  END WHILE
END FUNCTION
```

Code Implementation:

```
public static void BFS(int start, List<List<int>> graph)
{
    var visited = new HashSet<int>();
    var queue = new Queue<int>();
    queue.Enqueue(start);
    visited.Add(start);

    while (queue.Count > 0)
    {
        int node = queue.Dequeue();
        Console.WriteLine(node);

        foreach (int neighbor in graph[node])
        {
            if (!visited.Contains(neighbor))
            {
                visited.Add(neighbor);
                queue.Enqueue(neighbor);
            }
        }
    }
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)`

---

68. Detect Cycle in Graph

68.1 [Brute Force - DFS]

Pseudocode:

```
FUNCTION HasCycleDFS(node, visited, recStack, graph)
  visited.ADD(node)
  recStack.ADD(node)

  FOR each neighbor IN graph[node] DO
    IF neighbor NOT IN visited THEN
      IF HasCycleDFS(neighbor, visited, recStack, graph) THEN
        RETURN TRUE
      END IF
    ELSE IF neighbor IN recStack THEN
      RETURN TRUE  // Back edge = cycle
    END IF
  END FOR

  recStack.REMOVE(node)
  RETURN FALSE
END FUNCTION
```

Code Implementation:

```
public static bool HasCycleDFS(int node, HashSet<int> visited, HashSet<int> recStack, List<List<int>> graph)
{
    visited.Add(node);
    recStack.Add(node);

    foreach (int neighbor in graph[node])
    {
        if (!visited.Contains(neighbor))
        {
            if (HasCycleDFS(neighbor, visited, recStack, graph))
                return true;
        }
        else if (recStack.Contains(neighbor))
            return true;  // Back edge
    }

    recStack.Remove(node);
    return false;
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)`

68.2 [Union-Find (Disjoint Set)]

Pseudocode:

```
FUNCTION HasCycleUnionFind(edges, n)
  parent ← [0, 1, 2, ..., n-1]

  FOR each edge (u, v) IN edges DO
    rootU ← Find(u, parent)
    rootV ← Find(v, parent)

    IF rootU == rootV THEN
      RETURN TRUE  // Cycle detected
    END IF

    Union(rootU, rootV, parent)
  END FOR
```

```
  RETURN FALSE
END FUNCTION

FUNCTION Find(x, parent)
  IF parent[x] != x THEN
    parent[x] ← Find(parent[x], parent)  // Path compression
  END IF
  RETURN parent[x]
END FUNCTION

FUNCTION Union(x, y, parent)
  parent[y] ← x
END FUNCTION
```

Code Implementation:

```
public static bool HasCycleUnionFind(int[][] edges, int n)
{
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;

    foreach (int[] edge in edges)
    {
        int rootU = Find(edge[0], parent);
        int rootV = Find(edge[1], parent);

        if (rootU == rootV) return true;

        parent[rootV] = rootU;
    }

    return false;
}

private static int Find(int x, int[] parent)
{
    if (parent[x] != x)
        parent[x] = Find(parent[x], parent);
    return parent[x];
}
```

- **Time Complexity :** `O((V + E) * α(V))` (α = inverse Ackermann)
- **Space Complexity :** `O(V)`

---

69. Topological Sort

69.1 [Brute Force - DFS]

Pseudocode:

```
FUNCTION TopologicalSortDFS(graph)
  visited ← EMPTY_SET
  stack ← EMPTY_STACK

  FOR each node = 0 TO graph.length - 1 DO
    IF node NOT IN visited THEN
      TopologicalDFSUtil(node, visited, stack, graph)
    END IF
  END FOR

  result ← EMPTY_LIST
  WHILE stack NOT EMPTY DO
    result.ADD(stack.POP())
  END WHILE

  RETURN result
END FUNCTION

FUNCTION TopologicalDFSUtil(node, visited, stack, graph)
  visited.ADD(node)

  FOR each neighbor IN graph[node] DO
    IF neighbor NOT IN visited THEN
      TopologicalDFSUtil(neighbor, visited, stack, graph)
    END IF
  END FOR

  stack.PUSH(node)
END FUNCTION
```

Code Implementation:

```
public static List<int> TopologicalSortDFS(List<List<int>> graph)
{
    var visited = new HashSet<int>();
    var stack = new Stack<int>();

    for (int i = 0; i < graph.Count; i++)
    {
        if (!visited.Contains(i))
            TopologicalDFSUtil(i, visited, stack, graph);
    }

    var result = new List<int>();
    while (stack.Count > 0)
        result.Add(stack.Pop());

    return result;
}

private static void TopologicalDFSUtil(int node, HashSet<int> visited, Stack<int> stack, List<List<int>> graph)
{
    visited.Add(node);

    foreach (int neighbor in graph[node])
    {
        if (!visited.Contains(neighbor))
```

```
            TopologicalDFSUtil(neighbor, visited, stack, graph);
    }

    stack.Push(node);
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)`

69.2 [Kahn's Algorithm - BFS]

Pseudocode:

```
FUNCTION TopologicalSortKahn(graph, n)
  inDegree ← [0, 0, ..., 0] (size n)

  FOR each node = 0 TO n - 1 DO
    FOR each neighbor IN graph[node] DO
      inDegree[neighbor]++
    END FOR
  END FOR

  queue ← EMPTY_QUEUE
  FOR each node = 0 TO n - 1 DO
    IF inDegree[node] == 0 THEN
      queue.ENQUEUE(node)
    END IF
  END FOR

  result ← EMPTY_LIST
  WHILE queue NOT EMPTY DO
    node ← queue.DEQUEUE()
    result.ADD(node)

    FOR each neighbor IN graph[node] DO
      inDegree[neighbor]--
      IF inDegree[neighbor] == 0 THEN
        queue.ENQUEUE(neighbor)
      END IF
    END FOR
  END WHILE

  RETURN result
END FUNCTION
```

Code Implementation:

```
public static List<int> TopologicalSortKahn(List<List<int>> graph, int n)
{
    int[] inDegree = new int[n];

    for (int i = 0; i < n; i++)
    {
        foreach (int neighbor in graph[i])
            inDegree[neighbor]++;
    }

    var queue = new Queue<int>();
    for (int i = 0; i < n; i++)
    {
        if (inDegree[i] == 0)
            queue.Enqueue(i);
    }

    var result = new List<int>();
    while (queue.Count > 0)
    {
        int node = queue.Dequeue();
        result.Add(node);

        foreach (int neighbor in graph[node])
        {
            inDegree[neighbor]--;
            if (inDegree[neighbor] == 0)
                queue.Enqueue(neighbor);
        }
    }

    return result;
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)`

---

70. Shortest Path (Dijkstra)

70.1 [Brute Force - Bellman-Ford]

Pseudocode:

```
FUNCTION BellmanFord(graph, src, n)
  dist ← [INF, INF, ..., INF] (size n)
  dist[src] ← 0

  // Relax edges n-1 times
  FOR i = 0 TO n - 2 DO
    FOR each edge (u, v, w) IN graph DO
      IF dist[u] + w < dist[v] THEN
        dist[v] ← dist[u] + w
      END IF
    END FOR
  END FOR

  // Check for negative cycles
  FOR each edge (u, v, w) IN graph DO
    IF dist[u] + w < dist[v] THEN
```

```
      RETURN "Negative cycle exists"
    END IF
  END FOR

  RETURN dist
END FUNCTION
```

Code Implementation:

```
public static int[] BellmanFord(int n, int[][] edges, int src)
{
    int[] dist = new int[n];
    for (int i = 0; i < n; i++) dist[i] = int.MaxValue;
    dist[src] = 0;

    for (int i = 0; i < n - 1; i++)
    {
        foreach (int[] edge in edges)
        {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != int.MaxValue && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }

    return dist;
}
```

- **Time Complexity :** `O(V * E)`
- **Space Complexity :** `O(V)`

70.2 [Optimized - Dijkstra with Priority Queue]

Pseudocode:

```
FUNCTION Dijkstra(graph, src, n)
  dist ← [INF, INF, ..., INF] (size n)
  dist[src] ← 0
  pq ← PRIORITY_QUEUE()
  pq.ADD((0, src))

  WHILE pq NOT EMPTY DO
    (d, u) ← pq.EXTRACT_MIN()

    IF d > dist[u] THEN
      CONTINUE  // Already processed
    END IF

    FOR each (v, w) IN graph[u] DO
      IF dist[u] + w < dist[v] THEN
        dist[v] ← dist[u] + w
        pq.ADD((dist[v], v))
      END IF
    END FOR
  END WHILE

  RETURN dist
END FUNCTION
```

Code Implementation:

```
public static int[] Dijkstra(List<List<(int, int)>> graph, int src, int n)
{
    int[] dist = new int[n];
    for (int i = 0; i < n; i++) dist[i] = int.MaxValue;
    dist[src] = 0;

    var pq = new PriorityQueue<(int, int), int>();
    pq.Enqueue((src, 0), 0);

    while (pq.Count > 0)
    {
        var (d, u) = pq.Dequeue();

        if (d > dist[u]) continue;

        foreach (var (v, w) in graph[u])
        {
            if (dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
                pq.Enqueue((v, dist[v]), dist[v]);
            }
        }
    }

    return dist;
}
```

- **Time Complexity :** `O((V + E) log V)`
- **Space Complexity :** `O(V)`

---

71. Minimum Spanning Tree

71.1 [Kruskal's Algorithm]

Pseudocode:

```
FUNCTION Kruskal(edges, n)
  // Sort edges by weight
  Sort(edges by weight)

  parent ← [0, 1, 2, ..., n-1]
  mst ← EMPTY_LIST
  totalWeight ← 0
```

```
  FOR each edge (u, v, w) IN edges DO
    rootU ← Find(u, parent)
    rootV ← Find(v, parent)

    IF rootU != rootV THEN
      mst.ADD((u, v, w))
      totalWeight ← totalWeight + w
      Union(rootU, rootV, parent)
    END IF
  END FOR

  RETURN mst
END FUNCTION
```

Code Implementation:

```
public static List<(int, int, int)> Kruskal(List<(int, int, int)> edges, int n)
{
    edges.Sort((a, b) => a.Item3.CompareTo(b.Item3));

    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;

    var mst = new List<(int, int, int)>();

    foreach (var (u, v, w) in edges)
    {
        int rootU = Find(u, parent);
        int rootV = Find(v, parent);

        if (rootU != rootV)
        {
            mst.Add((u, v, w));
            parent[rootV] = rootU;
        }
    }

    return mst;
}
```

- **Time Complexity :** `O(E log E)`
- **Space Complexity :** `O(V)`

71.2 [Prim's Algorithm]

Pseudocode:

```
FUNCTION Prim(graph, src, n)
  inMST ← [FALSE, FALSE, ..., FALSE] (size n)
  key ← [INF, INF, ..., INF] (size n)
  key[src] ← 0

  mst ← EMPTY_LIST

  FOR count = 0 TO n - 1 DO
    // Find minimum key vertex not in MST
    u ← FindMinKey(key, inMST)
    inMST[u] ← TRUE

    // Update keys of adjacent vertices
    FOR each neighbor IN graph[u] DO
      weight ← EdgeWeight(u, neighbor)
      IF NOT inMST[neighbor] AND weight < key[neighbor] THEN
        key[neighbor] ← weight
        mst.ADD((u, neighbor, weight))
      END IF
    END FOR
  END FOR

  RETURN mst
END FUNCTION
```

Code Implementation:

```
public static List<(int, int)> Prim(List<List<(int, int)>> graph, int n)
{
    bool[] inMST = new bool[n];
    int[] key = new int[n];
    for (int i = 0; i < n; i++) key[i] = int.MaxValue;
    key[0] = 0;

    var mst = new List<(int, int)>();

    for (int count = 0; count < n; count++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!inMST[i] && (u == -1 || key[i] < key[u]))
                u = i;
        }

        inMST[u] = true;

        foreach (var (v, w) in graph[u])
        {
            if (!inMST[v] && w < key[v])
            {
                key[v] = w;
                mst.Add((u, v));
            }
        }
    }

    return mst;
}
```

- **Time Complexity :** `O(V²)` or `O((V + E) log V)` with priority queue
- **Space Complexity :** `O(V)`

72. Word Ladder / Connected Components

Constraints

- Both strings and list are not null
- Words have same length
- Transformation changes one letter at a time
- All intermediate words must be in word list
- BFS approach for shortest path

Pseudocode (Connected Components):

```
FUNCTION CountConnectedComponents(graph, n)
  visited ← EMPTY_SET
  count ← 0

  FOR node = 0 TO n - 1 DO
    IF node NOT IN visited THEN
      DFS(node, visited, graph)
      count++
    END IF
  END FOR

  RETURN count
END FUNCTION

FUNCTION DFS(node, visited, graph)
  visited.ADD(node)

  FOR each neighbor IN graph[node] DO
    IF neighbor NOT IN visited THEN
      DFS(neighbor, visited, graph)
    END IF
  END FOR
END FUNCTION
```

Code Implementation:

```
public static int CountConnectedComponents(int n, int[][] edges)
{
    var graph = new List<List<int>>[n];
    for (int i = 0; i < n; i++)
        graph[i] = new List<int>();

    foreach (int[] edge in edges)
    {
        graph[edge[0]].Add(edge[1]);
        graph[edge[1]].Add(edge[0]);
    }

    var visited = new HashSet<int>();
    int count = 0;

    for (int i = 0; i < n; i++)
    {
        if (!visited.Contains(i))
        {
            DFS(i, visited, graph);
            count++;
        }
    }

    return count;
}

private static void DFS(int node, HashSet<int> visited, List<int>[] graph)
{
    visited.Add(node);

    foreach (int neighbor in graph[node])
    {
        if (!visited.Contains(neighbor))
            DFS(neighbor, visited, graph);
    }
}
```

- **Time Complexity :** `O(V + E)`
- **Space Complexity :** `O(V)`

---

73. Rotting Oranges (2D Matrix BFS)

Constraints

- Grid contains three values: 0 (empty), 1 (fresh), 2 (rotten)
- Rotten orange spreads to adjacent fresh oranges each minute
- Find minutes until all oranges rot or return -1 if impossible
- Adjacent means up/down/left/right (not diagonal)

In a given grid, each cell can have one of three values:

- 0: empty cell
- 1: fresh orange
- 2: rotten orange

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes until no fresh oranges remain (or -1 if impossible).

Constraints:

- Oranges rot in 4 directions: up, down, left, right
- All rotten oranges rot simultaneously each minute

- Cannot rot diagonally

```
Example:
[2, 1, 1]
[1, 1, 0]
[0, 1, 1]

Minute 0: Rotten at [0,0]
Minute 1: [0,1], [1,0] become rotten
Minute 2: [1,1] becomes rotten
Minute 3: [1,2], [2,1] become rotten
Minute 4: [2,2] becomes rotten

Answer: 4 minutes

public static int OrangesRotting(int[][] grid)
{
    int rows = grid.Length;
    int cols = grid[0].Length;

    var queue = new Queue<(int, int, int)>(); // (row, col, time)
    int freshCount = 0;

    // Find all rotten oranges and count fresh ones
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (grid[i][j] == 2)
            {
                queue.Enqueue((i, j, 0));
            }
            else if (grid[i][j] == 1)
            {
                freshCount++;
            }
        }
    }

    // If no fresh oranges, return 0
    if (freshCount == 0) return 0;

    // BFS to rot adjacent oranges
    int[][] directions = { new[] { -1, 0 }, new[] { 1, 0 },
                           new[] { 0, -1 }, new[] { 0, 1 } };
    int maxTime = 0;

    while (queue.Count > 0)
    {
        var (row, col, time) = queue.Dequeue();
        maxTime = Math.Max(maxTime, time);

        // Check all 4 directions
        foreach (var dir in directions)
        {
            int newRow = row + dir[0];
            int newCol = col + dir[1];

            // If within bounds and fresh, rot it
            if (newRow >= 0 && newRow < rows &&
                newCol >= 0 && newCol < cols &&
                grid[newRow][newCol] == 1)
            {
                grid[newRow][newCol] = 2;
                queue.Enqueue((newRow, newCol, time + 1));
                freshCount--;
            }
        }
    }

    // If fresh oranges remain, return -1
    return freshCount == 0 ? maxTime : -1;
}

Grid:
[2, 1, 1]
[1, 1, 0]
[0, 1, 1]

BFS Timeline:
T=0: Queue = [(0,0)]
     Rot neighbors: (0,1), (1,0)

T=1: Queue = [(0,1), (1,0)]
     From (0,1): Rot (0,2), (1,1)
     From (1,0): Rot (1,1) [already rotting]

T=2: Queue = [(0,2), (1,1)]
     From (0,2): No fresh neighbors
     From (1,1): Rot (1,2), (2,1)

T=3: Queue = [(1,2), (2,1)]
     From (1,2): No fresh neighbors
     From (2,1): Rot (2,2)

T=4: Queue = [(2,2)]
     From (2,2): No fresh neighbors

freshCount = 0 → Return 4
```

- **Time Complexity :** `O(m × n)`
  - Each cell visited once during BFS
- **Space Complexity :** `O(m × n)`
  - Queue can contain up to all cells

---

82. Longest Common Subsequence (LCS)

82.1 [Brute Force - Recursive]

Pseudocode:

```
FUNCTION LCS_Recursive(s1, s2, i, j)
  IF i == 0 OR j == 0 THEN
    RETURN 0
  END IF

  IF s1[i-1] == s2[j-1] THEN
    RETURN 1 + LCS_Recursive(s1, s2, i-1, j-1)
  ELSE
    RETURN MAX(
      LCS_Recursive(s1, s2, i-1, j),
      LCS_Recursive(s1, s2, i, j-1)
    )
  END IF
END FUNCTION
```

Code Implementation:

```
public static int LCS_Recursive(string s1, string s2, int i, int j)
{
    if (i == 0 || j == 0) return 0;

    if (s1[i - 1] == s2[j - 1])
        return 1 + LCS_Recursive(s1, s2, i - 1, j - 1);
    else
        return Math.Max(
            LCS_Recursive(s1, s2, i - 1, j),
            LCS_Recursive(s1, s2, i, j - 1)
        );
}
```

- **Time Complexity :** $O(2^{n+m})$
- **Space Complexity :** $O(n + m)$ (Call stack)

82.2 [Memoization]

Pseudocode:

```
FUNCTION LCS_Memo(s1, s2, i, j, memo)
  IF i == 0 OR j == 0 THEN
    RETURN 0
  END IF

  IF memo[i][j] FOUND THEN
    RETURN memo[i][j]
  END IF

  IF s1[i-1] == s2[j-1] THEN
    result ← 1 + LCS_Memo(s1, s2, i-1, j-1, memo)
  ELSE
    result ← MAX(
      LCS_Memo(s1, s2, i-1, j, memo),
      LCS_Memo(s1, s2, i, j-1, memo)
    )
  END IF

  memo[i][j] ← result
  RETURN result
END FUNCTION
```

Code Implementation:

```
public static int LCS_Memo(string s1, string s2, int i, int j, int[,] memo)
{
    if (i == 0 || j == 0) return 0;

    if (memo[i, j] != -1) return memo[i, j];

    int result;
    if (s1[i - 1] == s2[j - 1])
        result = 1 + LCS_Memo(s1, s2, i - 1, j - 1, memo);
    else
        result = Math.Max(
            LCS_Memo(s1, s2, i - 1, j, memo),
            LCS_Memo(s1, s2, i, j - 1, memo)
        );

    memo[i, j] = result;
    return result;
}
```

- **Time Complexity :** $O(n * m)$
- **Space Complexity :** $O(n * m)$ (Memo table + Call stack)

82.3 [Tabulation - DP Table]

Pseudocode:

```
FUNCTION LCS_Tabulation(s1, s2)
  n ← s1.length
  m ← s2.length
  dp ← [n+1][m+1] initialized to 0

  FOR i = 1 TO n DO
    FOR j = 1 TO m DO
      IF s1[i-1] == s2[j-1] THEN
        dp[i][j] ← dp[i-1][j-1] + 1
      ELSE
        dp[i][j] ← MAX(dp[i-1][j], dp[i][j-1])
      END IF
    END FOR
  END FOR

  RETURN dp[n][m]
END FUNCTION
```

Code Implementation:

```
public static int LCS_Tabulation(string s1, string s2)
{
    int n = s1.Length;
    int m = s2.Length;
    int[,] dp = new int[n + 1, m + 1];

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            if (s1[i - 1] == s2[j - 1])
                dp[i, j] = dp[i - 1, j - 1] + 1;
            else
                dp[i, j] = Math.Max(dp[i - 1, j], dp[i, j - 1]);
        }
    }

    return dp[n, m];
}
```

---

83. Longest Increasing Subsequence (LIS)

83.1 [Brute Force - $O(2^n)$]

Placeholder for implementation

- **Time Complexity :** `O(2ⁿ)`
- **Space Complexity :** `O(n)` (Call stack)

83.2 [DP - $O(n^2)$]

Placeholder for implementation

- **Time Complexity :** `O(n²)`
- **Space Complexity :** `O(n)`

83.3 [Binary Search Optimized - O(n log n)]

Placeholder for implementation

- **Time Complexity :** `O(n log n)`
- **Space Complexity :** `O(n)`

---

90. Best Time to Buy and Sell Stock

Constraints

- Array contains daily stock prices
- Buy on one day, sell on a later day
- Maximize profit (or return 0 if no profit possible)
- O(n) time, O(1) space required

You are given an array `prices` where `prices[i]` is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Constraints:

- You may not engage in multiple transactions (buy-sell-buy-sell, etc.)
- You cannot sell before you buy
- You must select both buy and sell days

```
Examples:
Input: prices = [7, 1, 5, 3, 6, 4]
Output: 5
Explanation: Buy on day 2 (price = 1), Sell on day 5 (price = 6). Profit = 6 - 1 = 5

Input: prices = [7, 6, 4, 3, 1]
Output: 0
Explanation: No profit possible since prices are strictly decreasing
```

90.1 [Brute Force - All Pairs]

```
public static int MaxProfitBruteForce(int[] prices)
{
    int maxProfit = 0;

    // Try all possible pairs of (buy day, sell day)
    for (int i = 0; i < prices.Length; i++)
    {
        for (int j = i + 1; j < prices.Length; j++)
        {
            int profit = prices[j] - prices[i];
            maxProfit = Math.Max(maxProfit, profit);
        }
    }

    return maxProfit;
}
```

90.2 [Dynamic Programming - Max Profit with State]

```
public static int MaxProfitDP(int[] prices)
{
    if (prices.Length <= 1) return 0;
```

```
    int maxProfit = 0;
    int minPrice = prices[0];  // Minimum price seen so far

    // For each price, calculate profit if we sell at this price
    for (int i = 1; i < prices.Length; i++)
    {
        // Profit if we sell at current price (bought at minPrice)
        int profit = prices[i] - minPrice;

        // Update max profit
        maxProfit = Math.Max(maxProfit, profit);

        // Update minimum price if current price is lower
        minPrice = Math.Min(minPrice, prices[i]);
    }

    return maxProfit;
}
```

90.3 [Greedy - Single Pass Optimal]

```
public static int MaxProfitGreedy(int[] prices)
{
    if (prices.Length <= 1) return 0;

    int minPrice = int.MaxValue;
    int maxProfit = 0;

    foreach (int price in prices)
    {
        // Calculate potential profit if we sell at this price
        maxProfit = Math.Max(maxProfit, price - minPrice);

        // Update minimum price for future transactions
        minPrice = Math.Min(minPrice, price);
    }

    return maxProfit;
}
```

---

91. Trapping Rain Water (Max Water Container)

Constraints

- Array contains elevation heights (non-negative)
- Calculate water trapped after rain
- Water level: MIN(max_left, max_right) - height[i]
- Use two-pointer or pre-computed arrays

Given an elevation map represented by an array of heights, compute how much water can be trapped after raining. Water trapped is determined by the minimum of the maximum heights to the left and right, minus the current height.

```
Example:
Heights: [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

Water trapped: 6 units
```

91.1 [Brute Force - Height Pairs]

For each position, find max height to the left and right, then calculate water trapped.

```
public static int TrappingRainWaterBruteForce(int[] height)
{
    int water = 0;

    for (int i = 0; i < height.Length; i++)
    {
        int leftMax = 0;
        for (int j = 0; j <= i; j++) leftMax = Math.Max(leftMax, height[j]);

        int rightMax = 0;
        for (int j = i; j < height.Length; j++) rightMax = Math.Max(rightMax, height[j]);

        int waterLevel = Math.Min(leftMax, rightMax) - height[i];
        water += Math.Max(0, waterLevel);
    }

    return water;
}
```

- **Time Complexity :** $O(n^3)$
- **Space Complexity :** $O(1)$

91.2 [Two-Pointer Approach]

Use two pointers moving from outside-in. Track left and right maximum as we go.

```
public static int TrappingRainWaterTwoPointer(int[] height)
{
    int water = 0;
    int left = 0, right = height.Length - 1;
    int leftMax = 0, rightMax = 0;

    while (left < right)
    {
        if (height[left] < height[right])
        {
            if (height[left] >= leftMax) leftMax = height[left];
            else water += leftMax - height[left];
            left++;
        }
        else
        {
            if (height[right] >= rightMax) rightMax = height[right];
```

```
            else water += rightMax - height[right];
            right--;
        }
    }

    return water;
}
```

- **Time Complexity :** `O(n)`
- **Space Complexity :** `O(1)`

91.3 [Dynamic Programming]

Pre-compute maximum heights to left and right for all positions.

```
public static int TrappingRainWaterDP(int[] height)
{
    if (height.Length == 0) return 0;

    int n = height.Length;
    int[] leftMax = new int[n];
    int[] rightMax = new int[n];

    leftMax[0] = height[0];
    for (int i = 1; i < n; i++) leftMax[i] = Math.Max(leftMax[i - 1], height[i]);

    rightMax[n - 1] = height[n - 1];
    for (int i = n - 2; i >= 0; i--) rightMax[i] = Math.Max(rightMax[i + 1], height[i]);

    int water = 0;
    for (int i = 0; i < n; i++) water += Math.Min(leftMax[i], rightMax[i]) - height[i];

    return water;
}
```

---

92. Activity Selection Problem

Constraints

- Each activity has start and finish time
- Select maximum non-overlapping activities
- Sort by finish time ascending
- Greedy approach: O(n log n) time

Select the maximum number of activities that don't overlap, where each activity has a start and finish time.

92.1 [Brute Force - All Combinations]

Pseudocode:

```
FUNCTION SelectActivitiesBruteForce(activities)
  maxCount ← 0
  maxActivities ← EMPTY_LIST

  FUNCTION GenerateCombinations(index, currentSelection, lastFinish)
    IF index == activities.length THEN
      IF currentSelection.size > maxCount THEN
        maxCount ← currentSelection.size
        maxActivities ← COPY(currentSelection)
      END IF
      RETURN
    END IF

    activity ← activities[index]
    // Include current activity if it doesn't overlap
    IF activity.start >= lastFinish THEN
      currentSelection.ADD(activity)
      GenerateCombinations(index + 1, currentSelection, activity.finish)
      currentSelection.REMOVE_LAST()
    END IF

    // Exclude current activity
    GenerateCombinations(index + 1, currentSelection, lastFinish)
  END FUNCTION

  GenerateCombinations(0, EMPTY_LIST, 0)
  RETURN maxActivities
END FUNCTION
```

Code Implementation:

```
public class Activity
{
    public int start, finish;
    public Activity(int s, int f) { start = s; finish = f; }
}

public static List<Activity> SelectActivitiesBruteForce(Activity[] activities)
{
    List<Activity> maxActivities = new List<Activity>();
    GenerateCombinations(activities, 0, new List<Activity>(), 0, maxActivities);
    return maxActivities;
}

private static void GenerateCombinations(Activity[] activities, int index,
    List<Activity> currentSelection, int lastFinish, List<Activity> maxActivities)
{
    if (index == activities.Length)
    {
        if (currentSelection.Count > maxActivities.Count)
            maxActivities = new List<Activity>(currentSelection);
        return;
    }

    if (activities[index].start >= lastFinish)
```

```
    {
        currentSelection.Add(activities[index]);
        GenerateCombinations(activities, index + 1, currentSelection,
            activities[index].finish, maxActivities);
        currentSelection.RemoveAt(currentSelection.Count - 1);
    }

    GenerateCombinations(activities, index + 1, currentSelection, lastFinish, maxActivities);
}
```

92.2 [Greedy - Earliest Finish Time]

Pseudocode:

```
FUNCTION SelectActivitiesGreedy(activities)
  // Sort by finish time
  Sort(activities by finish time)

  selected ← EMPTY_LIST
  selected.ADD(activities[0])
  lastFinish ← activities[0].finish

  FOR i = 1 TO activities.length - 1 DO
    IF activities[i].start >= lastFinish THEN
      selected.ADD(activities[i])
      lastFinish ← activities[i].finish
    END IF
  END FOR

  RETURN selected
END FUNCTION
```

Code Implementation:

```
public static List<Activity> SelectActivitiesGreedy(Activity[] activities)
{
    Array.Sort(activities, (a, b) => a.finish.CompareTo(b.finish));

    var selected = new List<Activity> { activities[0] };
    int lastFinish = activities[0].finish;

    for (int i = 1; i < activities.Length; i++)
    {
        if (activities[i].start >= lastFinish)
        {
            selected.Add(activities[i]);
            lastFinish = activities[i].finish;
        }
    }

    return selected;
}
```

---

93. Huffman Coding

Constraints

- Input: frequency of each character
- Build binary tree with minimum total encoding length
- Assign variable-length codes (frequent = shorter)
- Use min-heap for tree construction

Create a binary tree for data compression by assigning variable-length codes based on character frequencies.

93.1 [Brute Force - All Frequencies]

Pseudocode:

```
FUNCTION HuffmanBruteForce(frequencies)
  // Generate all possible binary trees and select best
  FUNCTION GenerateTrees(charList)
    IF charList.size == 1 THEN
      RETURN [CreateLeafNode(charList[0])]
    END IF

    allTrees ← EMPTY_LIST
    FOR i = 0 TO charList.size - 1 DO
      FOR j = i+1 TO charList.size - 1 DO
        subList ← Combine(charList[i], charList[j])
        subtrees ← GenerateTrees(subList)
        allTrees.ADD_ALL(subtrees)
      END FOR
    END FOR

    RETURN allTrees
  END FUNCTION

  allTrees ← GenerateTrees(frequencies)
  // Select tree with minimum encoding length
  RETURN SelectBestTree(allTrees)
END FUNCTION
```

Code Implementation:

```
public class HuffmanNode
{
    public char ch;
    public int freq;
    public HuffmanNode left, right;

    public HuffmanNode(char c, int f) { ch = c; freq = f; }
    public HuffmanNode(HuffmanNode l, HuffmanNode r)
    {
        left = l;
        right = r;
```

```
        freq = l.freq + r.freq;
    }
}

public static HuffmanNode HuffmanBruteForce(Dictionary<char, int> frequencies)
{
    var nodes = frequencies.Select(x => new HuffmanNode(x.Key, x.Value)).ToList();

    while (nodes.Count > 1)
    {
        nodes.Sort((a, b) => a.freq.CompareTo(b.freq));
        HuffmanNode left = nodes[0];
        HuffmanNode right = nodes[1];
        nodes.RemoveRange(0, 2);

        HuffmanNode parent = new HuffmanNode(left, right);
        nodes.Add(parent);
    }

    return nodes[0];
}
```

- **Time Complexity :** $O(2^n)$
- **Space Complexity :** $O(n)$

93.2 [Greedy - Min Heap]

Pseudocode:

```
FUNCTION HuffmanCodingGreedy(frequencies)
  minHeap ← EMPTY_MIN_HEAP

  // Insert all characters with their frequencies
  FOR each (char, freq) IN frequencies DO
    minHeap.INSERT(HuffmanNode(char, freq))
  END FOR

  // Build tree bottom-up
  WHILE minHeap.size > 1 DO
    left ← minHeap.EXTRACT_MIN()
    right ← minHeap.EXTRACT_MIN()

    parent ← NEW HuffmanNode()
    parent.left ← left
    parent.right ← right
    parent.freq ← left.freq + right.freq

    minHeap.INSERT(parent)
  END WHILE

  root ← minHeap.EXTRACT_MIN()

  // Generate codes
  codes ← EMPTY_DICTIONARY
  GenerateCodes(root, "", codes)

  RETURN codes
END FUNCTION
```

Code Implementation:

```
public static Dictionary<char, string> HuffmanCodingGreedy(Dictionary<char, int> frequencies)
{
    var minHeap = new PriorityQueue<HuffmanNode, int>();

    foreach (var kvp in frequencies)
        minHeap.Enqueue(new HuffmanNode(kvp.Key, kvp.Value), kvp.Value);

    while (minHeap.Count > 1)
    {
        HuffmanNode left = minHeap.Dequeue();
        HuffmanNode right = minHeap.Dequeue();

        HuffmanNode parent = new HuffmanNode(left, right);
        minHeap.Enqueue(parent, parent.freq);
    }

    HuffmanNode root = minHeap.Dequeue();

    var codes = new Dictionary<char, string>();
    GenerateCodesHelper(root, "", codes);

    return codes;
}

private static void GenerateCodesHelper(HuffmanNode node, string code, Dictionary<char, string> codes)
{
    if (node == null) return;

    if (node.left == null && node.right == null)
        codes[node.ch] = code;
    else
    {
        GenerateCodesHelper(node.left, code + "0", codes);
        GenerateCodesHelper(node.right, code + "1", codes);
    }
}
```

---

94. Fractional Knapsack Problem

Constraints

- Each item has weight and value
- Capacity constraint: total weight <= W
- Can take fractions of items (unlike 0/1 knapsack)
- Maximize total value

- Greedy: sort by value/weight ratio

Select items (or fractions of items) to maximize value while staying within weight capacity.

94.1 [Brute Force - All Permutations]

Pseudocode:

```
FUNCTION FractionalKnapsackBruteForce(items, capacity)
  maxValue ← 0

  FUNCTION TryAllCombinations(index, currentWeight, currentValue)
    IF index == items.length OR currentWeight >= capacity THEN
      maxValue ← MAX(maxValue, currentValue)
      RETURN
    END IF

    item ← items[index]

    // Try including whole item
    IF currentWeight + item.weight <= capacity THEN
      TryAllCombinations(index + 1, currentWeight + item.weight,
                         currentValue + item.value)
    END IF

    // Try including fraction of item
    remainingCapacity ← capacity - currentWeight
    IF remainingCapacity > 0 THEN
      fraction ← MIN(item.weight, remainingCapacity) / item.weight
      TryAllCombinations(index + 1, currentWeight + fraction * item.weight,
                         currentValue + fraction * item.value)
    END IF

    // Exclude item
    TryAllCombinations(index + 1, currentWeight, currentValue)
  END FUNCTION

  TryAllCombinations(0, 0, 0)
  RETURN maxValue
END FUNCTION
```

Code Implementation:

```
public class Item
{
    public double weight, value;
    public Item(double w, double v) { weight = w; value = v; }
}

public static double FractionalKnapsackBruteForce(Item[] items, double capacity)
{
    double maxValue = 0;

    void TryAllCombinations(int index, double currentWeight, double currentValue)
    {
        if (index == items.Length || currentWeight >= capacity)
        {
            maxValue = Math.Max(maxValue, currentValue);
            return;
        }

        // Include whole item
        if (currentWeight + items[index].weight <= capacity)
            TryAllCombinations(index + 1, currentWeight + items[index].weight,
                currentValue + items[index].value);

        // Include fraction
        double remainingCapacity = capacity - currentWeight;
        if (remainingCapacity > 0)
        {
            double fraction = Math.Min(items[index].weight, remainingCapacity) / items[index].weight;
            TryAllCombinations(index + 1, currentWeight + fraction * items[index].weight,
                currentValue + fraction * items[index].value);
        }

        // Exclude item
        TryAllCombinations(index + 1, currentWeight, currentValue);
    }

    TryAllCombinations(0, 0, 0);
    return maxValue;
}
```

- **Time Complexity :** `O(n!)`
- **Space Complexity :** `O(n)`

94.2 [Greedy - Value/Weight Ratio]

Pseudocode:

```
FUNCTION FractionalKnapsackGreedy(items, capacity)
  // Sort by value/weight ratio in descending order
  Sort(items by ratio = value/weight DESC)

  totalValue ← 0
  remainingCapacity ← capacity

  FOR each item IN items DO
    IF remainingCapacity == 0 THEN
      BREAK
    END IF

    IF item.weight <= remainingCapacity THEN
      // Take whole item
      totalValue ← totalValue + item.value
      remainingCapacity ← remainingCapacity - item.weight
    ELSE
      // Take fraction of item
```

```
      fraction ← remainingCapacity / item.weight
      totalValue ← totalValue + fraction * item.value
      remainingCapacity ← 0
    END IF
  END FOR

  RETURN totalValue
END FUNCTION
```

Code Implementation:

```
public static double FractionalKnapsackGreedy(Item[] items, double capacity)
{
    Array.Sort(items, (a, b) =>
        (b.value / b.weight).CompareTo(a.value / a.weight));

    double totalValue = 0;
    double remainingCapacity = capacity;

    foreach (Item item in items)
    {
        if (remainingCapacity == 0) break;

        if (item.weight <= remainingCapacity)
        {
            totalValue += item.value;
            remainingCapacity -= item.weight;
        }
        else
        {
            double fraction = remainingCapacity / item.weight;
            totalValue += fraction * item.value;
            remainingCapacity = 0;
        }
    }

    return totalValue;
}
```

---

95. Jump Game / Reach End of Array

Constraints

- Each element = maximum jump length from that position
- Determine if last index is reachable
- Start from index 0
- Greedy: track maximum reachable index

Determine if you can reach the last index of an array, where each element indicates the maximum jump length.

95.1 [Brute Force - BFS/DFS]

Pseudocode:

```
FUNCTION CanJumpBFS(nums)
  IF nums.length <= 1 THEN
    RETURN TRUE
  END IF

  visited ← SET()
  queue ← QUEUE()
  queue.ENQUEUE(0)
  visited.ADD(0)

  WHILE queue NOT EMPTY DO
    index ← queue.DEQUEUE()
    maxJump ← nums[index]

    FOR jump = 1 TO maxJump DO
      nextIndex ← index + jump

      IF nextIndex == nums.length - 1 THEN
        RETURN TRUE
      END IF

      IF nextIndex < nums.length AND nextIndex NOT IN visited THEN
        visited.ADD(nextIndex)
        queue.ENQUEUE(nextIndex)
      END IF
    END FOR
  END WHILE

  RETURN FALSE
END FUNCTION
```

Code Implementation:

```
public static bool CanJumpBFS(int[] nums)
{
    if (nums.Length <= 1) return true;

    var visited = new HashSet<int>();
    var queue = new Queue<int>();
    queue.Enqueue(0);
    visited.Add(0);

    while (queue.Count > 0)
    {
        int index = queue.Dequeue();
        int maxJump = nums[index];

        for (int jump = 1; jump <= maxJump; jump++)
        {
            int nextIndex = index + jump;

            if (nextIndex == nums.Length - 1)
```

```
                return true;

            if (nextIndex < nums.Length && !visited.Contains(nextIndex))
            {
                visited.Add(nextIndex);
                queue.Enqueue(nextIndex);
            }
        }
    }

    return false;
}
```

95.2 [Greedy - Maximum Reach]

Pseudocode:

```
FUNCTION CanJumpGreedy(nums)
  maxReach ← 0

  FOR i = 0 TO nums.length - 1 DO
    IF i > maxReach THEN
      RETURN FALSE  // Can't reach this index
    END IF

    IF i == nums.length - 1 THEN
      RETURN TRUE  // Reached the end
    END IF

    maxReach ← MAX(maxReach, i + nums[i])
  END FOR

  RETURN TRUE
END FUNCTION
```

Code Implementation:

```
public static bool CanJumpGreedy(int[] nums)
{
    int maxReach = 0;

    for (int i = 0; i < nums.Length; i++)
    {
        if (i > maxReach)
            return false;  // Can't reach this index

        if (i == nums.Length - 1)
            return true;  // Reached the end

        maxReach = Math.Max(maxReach, i + nums[i]);
    }

    return true;
}
```

---

96. Interval Scheduling Maximization

Constraints

- List of intervals (start, end)
- Maximize number of non-overlapping intervals
- Sort by end time ascending
- Greedy selection

Given a list of intervals, find the maximum number of non-overlapping intervals.

Pseudocode:

```
FUNCTION MaximizeIntervals(intervals)
  IF intervals.length == 0 THEN
    RETURN 0
  END IF

  // Sort by end time
  Sort(intervals by end time ASC)

  count ← 1
  lastEnd ← intervals[0].end

  FOR i = 1 TO intervals.length - 1 DO
    IF intervals[i].start >= lastEnd THEN
      count ← count + 1
      lastEnd ← intervals[i].end
    END IF
  END FOR

  RETURN count
END FUNCTION
```

Code Implementation:

```
public class Interval
{
    public int start, end;
    public Interval(int s, int e) { start = s; end = e; }
}

public static int MaximizeIntervals(Interval[] intervals)
{
    if (intervals.Length == 0) return 0;

    Array.Sort(intervals, (a, b) => a.end.CompareTo(b.end));

    int count = 1;
    int lastEnd = intervals[0].end;
```

```
    for (int i = 1; i < intervals.Length; i++)
    {
        if (intervals[i].start >= lastEnd)
        {
            count++;
            lastEnd = intervals[i].end;
        }
    }

    return count;
}
```

---

97. Gas Station / Circuit

Constraints

- Each station has gas amount and cost to next station
- Circular route: visit all stations and return
- Single tank, refuel at each station
- Find starting station or return -1 if impossible

Start at a gas station and visit all stations in a circular route. Each station has gas to consume and distance to next station.

Pseudocode:

```
FUNCTION CanCompleteCircuit(gas, cost)
  totalGas ← SUM(gas)
  totalCost ← SUM(cost)

  IF totalGas < totalCost THEN
    RETURN -1
  END IF

  currentGas ← 0
  startIndex ← 0

  FOR i = 0 TO gas.length - 1 DO
    currentGas ← currentGas + gas[i] - cost[i]

    IF currentGas < 0 THEN
      startIndex ← i + 1
      currentGas ← 0
    END IF
  END FOR

  RETURN startIndex
END FUNCTION
```

Code Implementation:

```
public static int CanCompleteCircuit(int[] gas, int[] cost)
{
    int totalGas = 0, totalCost = 0;
    foreach (int g in gas) totalGas += g;
    foreach (int c in cost) totalCost += c;

    if (totalGas < totalCost) return -1;

    int currentGas = 0;
    int startIndex = 0;

    for (int i = 0; i < gas.Length; i++)
    {
        currentGas += gas[i] - cost[i];

        if (currentGas < 0)
        {
            startIndex = i + 1;
            currentGas = 0;
        }
    }

    return startIndex;
}
```

---

98. Candy Distribution Problem

Constraints

- Each child gets minimum 1 candy
- Higher rating than neighbor => more candy than neighbor
- Minimize total candy distributed
- Two-pass algorithm: left-to-right, then right-to-left

Distribute candy to children where each child must get at least 1 candy, and children with higher ratings than neighbors must get more candy than those neighbors.

Pseudocode:

```
FUNCTION DistributeCandy(ratings)
  n ← ratings.length
  candy ← ARRAY of size n, initialize all to 1

  // Left to right pass: if rating[i] > rating[i-1],
  // then candy[i] = candy[i-1] + 1
  FOR i = 1 TO n - 1 DO
    IF ratings[i] > ratings[i-1] THEN
      candy[i] ← candy[i-1] + 1
    END IF
  END FOR

  // Right to left pass: ensure candy[i] > candy[i+1]
  // if ratings[i] > ratings[i+1]
  FOR i = n - 2 DOWN TO 0 DO
```

```
    IF ratings[i] > ratings[i + 1] THEN
      candy[i] ← MAX(candy[i], candy[i+1] + 1)
    END IF
  END FOR

  RETURN SUM(candy)
END FUNCTION
```

Code Implementation:

```
public static int DistributeCandy(int[] ratings)
{
    int n = ratings.Length;
    int[] candy = new int[n];

    // Initialize all children with 1 candy
    for (int i = 0; i < n; i++)
        candy[i] = 1;

    // Left to right: if rating increases, candy increases
    for (int i = 1; i < n; i++)
    {
        if (ratings[i] > ratings[i - 1])
        {
            candy[i] = candy[i - 1] + 1;
        }
    }

    // Right to left: ensure descending ratings satisfied
    for (int i = n - 2; i >= 0; i--)
    {
        if (ratings[i] > ratings[i + 1])
        {
            candy[i] = Math.Max(candy[i], candy[i + 1] + 1);
        }
    }

    int total = 0;
    foreach (int c in candy)
        total += c;

    return total;
}
```